

ADR-001 — Embedding Model: all-MiniLM-L6-v2

Field	Detail
Decision	Use <code>sentence-transformers/all-MiniLM-L6-v2</code> as the sole embedding model for the entire pipeline
Date	April 2025
Context	The project needed a fast, locally-runnable embedding model to convert document chunks and user queries into dense vectors for semantic similarity search. The model had to run on-device (CUDA-enabled) and produce consistent-dimension outputs throughout the pipeline.
Options Considered	<code>all-MiniLM-L6-v2</code> (384-dim, fast, lightweight) · <code>all-mpnet-base-v2</code> (768-dim, higher quality) · <code>text-embedding-3-small</code> via OpenAI API (cloud, latency, cost) · <code>bge-small-en-v1.5</code> (similar size)
Chosen Solution	<code>all-MiniLM-L6-v2</code>
Rationale	Fast inference on CUDA (GPU-accelerated). Only 6 transformer layers — sub-100ms encode per batch. 384-dim vectors are compact enough for efficient HNSW indexing. Pretrained on large semantic similarity corpora, adequate for domain-specific retrieval. No API calls required — fully local.
Impact	Embedding dimension fixed at 384 across the entire project. Changing the model at any point would require re-ingesting all documents (the HNSW index dimension cannot change in-place). Lives in <code>embedding_manager.py</code> .

ADR-002 — Vector Store: ChromaDB

Field	Detail
Decision	Use ChromaDB with local persistent storage as the vector database
Date	April 2025
Context	Needed a vector store for storing 384-dim embeddings of document chunks and supporting fast approximate nearest-neighbor (ANN) retrieval. Required Python-native API, metadata filtering support, and local persistence without a running server process.
Options Considered	ChromaDB (local, Python-native, HNSW, metadata support) · FAISS (fast, no metadata out-of-the-box, no persistence) · Pinecone (managed cloud, latency, cost, data leaves machine) · Weaviate (more complex, requires daemon)
Chosen Solution	ChromaDB with <code>persist_directory=../data/vector_store</code>
Rationale	Simplest Python-native API (no daemon). Local persistence (two components: <code>chroma.sqlite3</code> for metadata + UUID folder for HNSW binary). HNSW index gives ~95–99% recall at sub-second latency even as corpus grows. Metadata support is essential for source filtering. FAISS rejected: lacks metadata without add-ons. Pinecone rejected: cloud-only, adds latency and cost.
Impact	Two collections used: <code>documents</code> (source chunks) and <code>learned_qa</code> (self-learning memory). Both components (<code>chroma.sqlite3</code> + UUID folder) must be kept together — losing either breaks the store. All retrieval in <code>retriever.py</code> , <code>vector_store.py</code> , and <code>self_learner.py</code> depends on this.

ADR-003 — Classic RAG → Agentic RAG Architecture

Field	Detail
Decision	Replace the linear classic RAG pipeline with a full Agentic RAG loop
Date	April 2025
Context	The initial pipeline was a fixed single-pass: <code>embed query → retrieve → prompt → answer</code> . This approach had no ability to self-correct when retrieval was poor, no query reformulation, and no quality gate.
Options Considered	Classic RAG (simple, linear, deterministic) · Agentic RAG (LLM decides when/how to retrieve, iterative, self-correcting) · HyDE (Hypothetical Document Embeddings, generate before retrieval)
Chosen Solution	Agentic RAG with a structured iteration loop (max 6 iterations) and LangChain tool-calling
Rationale	The agent can decide <i>whether</i> to retrieve, <i>how many times</i> , <i>with what query reformulations</i> , and <i>when to stop</i> . This eliminates single-point-of-failure retrieval: if one query retrieves poor chunks, the agent reformulates and tries a different angle. The quality check gate (<code>check_answer_quality</code>) catches hallucinations before returning answers.
Impact	Defined the entire architecture of <code>agent_query.py</code> . Justified the creation of <code>tools.py</code> , <code>self_learner.py</code> , and <code>feedback_store.py</code> . Introduced the <code>MAX_ITERATIONS=6</code> and <code>MAX_TOTAL_RETRIEVALS=5</code> hard limits to prevent runaway loops.

ADR-004 — LLM Provider: Groq

Field	Detail
Decision	Use Groq as the primary LLM inference provider
Date	April 2025
Context	Needed a fast, affordable inference provider for an agentic loop that makes 3–10 LLM calls per query (retrieval iterations + compression + judging). Latency per call directly affects total query response time.
Options Considered	Groq (LPU hardware, very fast, cheap, open-source models) · OpenAI (higher quality GPT models, more expensive, slower) · Together AI (open-source models, competitive pricing) · Local inference (Ollama/vLLM — no API cost but hardware constraints)
Chosen Solution	Groq with <code>llama-3.1-8b-instant</code> as primary model
Rationale	Groq runs Llama models on custom LPU silicon — dramatically faster than GPU inference (~250 tokens/sec vs ~50 tokens/sec). For an agentic pipeline making 5–10 LLM calls per query, this latency difference compounds. Both Groq and OpenAI expose OpenAI-compatible REST APIs, so switching providers later requires only changing <code>base_url</code> and <code>api_key</code> . Cost is ~10× lower than GPT-4.x equivalents.
Impact	Affects <code>llm_caller.py</code> , <code>agent_query.py</code> , and <code>.env</code> configuration. Model names must be read from <code>.env</code> (<code>MODEL_NAME</code>) to survive Groq's model deprecation cycles (both <code>gemma2-9b-it</code> and <code>llama3-8b-8192</code> were deprecated mid-project).

ADR-005 — Primary Model: llama-3.1-8b-instant (with llama-3.3-70b-versatile for production)

Field	Detail
Decision	Use llama-3.1-8b-instant as the working/development model; identify llama-3.3-70b-versatile as the production-grade target
Date	April–May 2025
Context	After the deprecation of gemma2-9b-it and llama3-8b-8192 during development, needed to select a stable model. Two Groq models were evaluated on the criteria of latency, tool-use reliability, context window, and instruction-following.
Options Considered	llama-3.1-8b-instant (128K/8K context, extremely fast, moderate tool reliability) · llama-3.3-70b-versatile (128K/32K context, slower, much better tool-use and instruction-following) · llama-3.1-8b-instruct (similar to instant but better at multi-step reasoning, not on Groq)
Chosen Solution	llama-3.1-8b-instant for development; llama-3.3-70b-versatile recommended for production
Rationale	The 8b-instant model is adequate for rapid iteration and testing. It handles the orchestration speed required by the agentic loop. However, its instruction-following degrades at ~16K prompt tokens, and it occasionally misses tool constraints (e.g., firing all tool calls simultaneously including premature check_answer_quality). The 70B model is significantly better for NAC/DC/LBC compression quality and faithful merging, but is slower and more expensive. For production agentic use where answer faithfulness matters, the 70B model is the correct choice.
Impact	Observed quality gaps in Batch testing (Batches 6, 8, 10): poor analytical reasoning, acronym confusion, weak synthesis — all traceable to 8B model limitations. A hybrid model architecture (8B for retrieval orchestration, 70B for compression/judging) identified as the strongest upgrade path. Context window limit of 8,192 output tokens is a hard constraint for the instant model.

ADR-006 — LangChain as the LLM Framework Wrapper

Field	Detail
Decision	Use LangChain's <code>ChatOpenAI</code> (not native <code>openai</code> SDK) as the wrapper for all LLM calls
Date	April 2025
Context	Needed a framework layer to handle tool/function calling, message history management, and response parsing without writing raw HTTP requests.
Options Considered	LangChain <code>ChatOpenAI</code> (provider-agnostic, tool-call handling, broad community) · Native <code>openai</code> SDK (lower-level, tighter coupling to OpenAI) · <code>langchain_groq.ChatGroq</code> (Groq-specific, less portable) · No framework (raw requests) (maximum control, maximum boilerplate)
Chosen Solution	<code>langchain_openai.ChatOpenAI</code> with <code>base_url</code> pointed at Groq's OpenAI-compatible endpoint
Rationale	<code>ChatOpenAI</code> is provider-agnostic — switching between Groq, OpenAI, local vLLM, or TGI only requires changing <code>base_url</code> and <code>api_key</code> . The tool-call handling (schema passing, response parsing, multi-turn message threading) is handled by the framework. Using <code>ChatGroq</code> was rejected because it would couple the codebase to one provider. LangChain is "glue" — not a model itself — and its overhead is acceptable given the simplification.
Impact	All three LLM instances (<code>llm</code> , <code>merge_llm</code> , <code>judge_llm</code>) are built from <code>ChatOpenAI</code> . The design proved directly useful when later testing with local TGI servers — only <code>base_url</code> and <code>http_client</code> needed to change. LangChain's <code>LLMChainExtractor</code> was subsequently <i>rejected</i> for context compression due to extra latency and dependency on LangChain's retriever format.

ADR-007 — Context Compression Pipeline: NAC → DC → LBC

Field	Detail
Decision	Implement a three-stage semantic compression pipeline: Neighbor-Aware Compression → Deduplication Compression → LLM-Based Compression
Date	May 2025
Context	Retrieved chunks were often: (a) split mid-sentence at chunk boundaries, losing document flow; (b) semantically redundant across multiple retrieval calls; (c) too verbose for the LLM's effective context window. All three problems were degrading answer quality and wasting tokens.
Options Considered	Extractive compression (SentenceTransformers cosine similarity — safe, no hallucination) · LLM-Based Compression/LBC (rewrite/summarize — ~27.6% reduction, hallucination risk) · Deduplication/DC (remove redundant sentences — low yield unless overlap is high) · Neighbor-Aware/NAC (merge consecutive same-source chunks — restores document flow) · BM25 sentence compression (lexical fallback) · LangChain LLMChainExtractor (framework-level option) · No compression (pass raw chunks)
Chosen Solution	Three-stage pipeline: NAC → DC → LBC , token-budget gated (skip if context < 500 tokens), implemented in <code>compress_context_pipeline()</code> in <code>context_compression.py</code>
Rationale	NAC addresses the chunking boundary problem first (most impactful). DC then removes redundancy that NAC may have introduced via merging. LBC finally rewrites for relevance and conciseness. LangChain <code>LLMChainExtractor</code> was rejected — extra latency and format dependency. BM25 was evaluated but is lexical-only and misses semantic paraphrases. Extractive compression is safest (no hallucination) but purely sentence-level; it was included as a fallback. LBC hallucination risk is mitigated by a faithfulness guard (see ADR-017).
Impact	<code>context_compression.py</code> implements the full pipeline. The token-budget gate (<code>COMPRESS_MIN_TOKENS = 500</code>) skips compression for small contexts to save latency. Embedding-first DC prefiltering (cosine similarity ≥ 0.92 before LLM pair comparison) identified as the next planned improvement — changes DC from $O(n^2)$ LLM calls to $O(\text{high-similarity})$

	candidates only).
--	-------------------

ADR-008 — Self-Learning Mechanism: Memory Injection over Fine-tuning

Field	Detail
Decision	Implement self-learning via memory injection into a <code>learned_qa</code> ChromaDB collection; reject naive auto-save and RLHF/fine-tuning for the current scale
Date	April–May 2025
Context	The project goal was an agent that improves over time. Three mechanisms were evaluated: knowledge base growth, weight-level fine-tuning, and reinforcement learning from user feedback.
Options Considered	Memory injection (save validated Q&A pairs back into ChromaDB — "pseudo-learning") · Fine-tuning on accumulated interactions (real weight-level learning, batch process, requires ~500+ examples) · RLHF/DPO (thumbs-up/down as reward signal — research-grade complexity, needs scale) · Naive auto-save (save every answer — rejected: hallucinations become permanent)
Chosen Solution	Memory injection with confidence-gated saving, triggered every 5 successful (<code>OK</code>) interactions via <code>SelfLearner</code> in <code>self_learner.py</code>
Rationale	Memory injection is the only option achievable at the current project scale. Naive auto-save was explicitly rejected because hallucinations would become permanently retrievable, degrading future answers. The confidence gate (only <code>OK</code> -quality interactions trigger distillation) ensures only verified knowledge enters the <code>learned_qa</code> collection. Fine-tuning is the identified <i>future upgrade path</i> — the <code>interactions.jsonl</code> log is the training dataset, but requires ~500+ validated pairs before fine-tuning is worth the effort over better prompting. RLHF is research-grade overkill at single-app scale.
Impact	<code>self_learner.py</code> , <code>feedback_store.py</code> , <code>user_thumbsdowns.json</code> , and <code>failed_variants.json</code> all exist because of this decision. The <code>LEARN_EVERY_N = 5</code> trigger constant is in <code>agent_query.py</code> . The retriever does hybrid retrieval across both <code>documents</code> and <code>learned_qa</code> collections.

ADR-009 — Structured Output Strategy: Native JSON Mode over Outlines/Guidance

Field	Detail
Decision	Use native Groq <code>response_format: json_object</code> (with schema in prompt) for structured outputs; reject Outlines and Guidance for production use with remote APIs
Date	May 2025
Context	The pipeline required reliable JSON outputs from several LLM calls (compression verdicts, merge outputs, judge verdicts, distilled Q&A pairs). LLMs frequently produced broken JSON, extra text, wrong fields, and inconsistent verdicts — causing <code>JSONDecodeError</code> exceptions and retry loops.
Options Considered	Outlines (FSM token masking — true constrained decoding, local models only) · Guidance (logit masking, same local-model constraint) · Native Groq json_schema mode (server-side enforcement, only <code>llama-3.3-70b-versatile</code>) · Native Groq json_object mode (valid JSON guaranteed, schema enforced via prompt, works with <code>llama-3.1-8b-instant</code>) · Structured prompt + <code>json.loads()</code> + retry (current baseline)
Chosen Solution	Native Groq structured output (<code>response_format: {"type": "json_object"}</code>) + Pydantic schema in prompt + <code>json.loads()</code> with retry . <code>fix_llm_output.py</code> handles multi-tier JSON repair.
Rationale	Outlines and Guidance both rely on logit/token masking, which requires <i>local model access</i> — i.e., direct access to the model's forward pass. With remote APIs like Groq, both frameworks fall back to sending <code>response_format: json_schema</code> to the API — they provide no actual constrained decoding benefit over just calling the API directly. <code>llama-3.1-8b-instant</code> does not support <code>json_schema</code> mode at all (only <code>json_object</code>). Outlines also imposes 40s–10+ minute schema compilation time for complex schemas. Net result: Outlines and Guidance add a dependency layer that only provides the same <code>response_format</code> passthrough that a direct API call already provides. Rejected. For truly constrained decoding in the future: local model hosting with Outlines + vLLM would be the path.
Impact	Explains the absence of Outlines/Guidance in the final pipeline. <code>fix_llm_output.py</code> handles broken JSON with greedy regex fallback and multi-tier repair. The retry loop (up to

	LLM_RESPONSE_RETRY_LIMIT attempts) is the production safety net.
--	--

ADR-010 — Agent State Machine: 4-Phase RETRIEVE → COMPRESS → DRAFT → JUDGE

Field	Detail
Decision	Refactor the free-form agentic loop into an explicit 4-phase state machine with controlled transitions
Date	May 2025
Context	The earlier agentic loop gave the LLM full freedom to call any tool at any time. This led to the LLM pre-emptively calling <code>check_answer_quality</code> with fabricated answers in Iteration 1 before any retrieval had occurred, batching incompatible tool calls simultaneously, and failing to transition cleanly from retrieval to answer generation.
Options Considered	Free-form tool loop (LLM decides all tool calls — flexible but unreliable with 8B models) · Explicit state machine (RETRIEVE → COMPRESS → DRAFT → JUDGE — deterministic transitions, LLM constrained per phase)
Chosen Solution	4-phase state machine: Phase 1 RETRIEVE (tool_schemas = <code>[retrieve_documents, compress_context]</code>) → Phase 2 COMPRESS (force <code>compress_context()</code> if not already called) → Phase 3 DRAFT (invoke LLM with <code>tools=None</code> for free-form prose) → Phase 4 JUDGE (call <code>check_answer_quality()</code> as a direct Python function)
Rationale	Small 8B models lose instruction-following reliability as context grows. The state machine removes ambiguity by restricting available tools per phase — the LLM cannot call <code>check_answer_quality</code> in the RETRIEVE phase because it is not in <code>tool_schemas</code> during that phase. The DRAFT phase uses <code>tools=None</code> so the model generates prose instead of tool calls. The JUDGE phase bypasses the LLM entirely — the quality check is a Python function call. The re-loop path (INSUFFICIENT → RETRIEVE) lets the agent accumulate more evidence without discarding already-compressed chunks.
Impact	Defined the final architecture of <code>agent_query.py</code> . <code>check_answer_quality</code> was removed from <code>tool_schemas</code> and

	made a direct callable (see ADR-020). The <code>compress_done</code> flag tracks whether compression has occurred for the current iteration cycle. Token-budget gate in <code>compress_context</code> skips NAC→DC→LBC for small contexts (<500 approx tokens).
--	---

ADR-011 — Retrieval Validator & Merge Validator (LLM-as-Judge)

Field	Detail
Decision	Add two LLM-as-judge validators: one to assess retrieved chunk relevance after each <code>retrieve_documents</code> call, and one to assess faithfulness of each NAC merge
Date	May 2025
Context	Retrieved chunks were often partially or fully irrelevant to the original query. Merged chunks from NAC were sometimes fabricating claims or dropping key facts. Both problems were silently degrading final answer quality with no detection mechanism.
Options Considered	Heuristic validators (cosine similarity threshold, keyword matching — fast but shallow) · LLM-as-judge (semantic relevance scoring + per-claim fabrication/drop detection — accurate but adds latency) · No validation (pass everything through — fast, low reliability)
Chosen Solution	Two validators in <code>validators.py</code> : <code>validate_retrieval()</code> (per-chunk relevance → PASS/PARTIAL/FAIL/UNKNOWN verdict) and <code>validate_merge()</code> (per-claim faithfulness → OK/UNFAITHFUL verdict with <code>fabricated_claims</code> and <code>dropped_claims</code> lists). Both use <code>judge_llm</code> .
Rationale	Semantic relevance cannot be reliably checked with cosine similarity alone — a chunk can be topically adjacent but not answer the question. LLM-as-judge provides semantic understanding. The <code>UNKNOWN</code> verdict acts as a safe fallback (keeps all chunks) to avoid over-aggressive filtering. The merge validator with per-claim granularity gives specific feedback for retry prompts, enabling targeted corrections instead of full regeneration.
Impact	<code>validate_retrieval()</code> is wired into <code>handle_retrieve_documents_call()</code> in <code>agent_query.py</code> . FAIL/PARTIAL verdicts inject a sidecar tool-role message

	guiding the agent to reformulate. <code>validate_merge()</code> rolls back to <code>source_chunks_for_merge[0]</code> on UNFAITHFUL verdict. Identified that paraphrases of the same fact should NOT be flagged (GOOD: "lower electricity costs" ≠ FABRICATED vs "minimize electricity costs").
--	---

ADR-012 — System Prompt Architecture: Split `_ROLE_AND_RULES` + `_PROCESS_INSTRUCTIONS`

Field	Detail
Decision	Split the monolithic <code>_BASE_SYSTEM_PROMPT</code> into a <code>_ROLE_AND_RULES</code> block (prepended) and a <code>_PROCESS_INSTRUCTIONS</code> block (appended last), with thumbdown/blocked-variant context sandwiched between them
Date	May 2025
Context	As the system prompt grew with injected thumbdown history (up to ~1,800 tokens in Run 3), the <code>PROCESS</code> instructions were buried in the middle of the prompt. The 8B model exhibited recency bias — <code>PROCESS</code> instructions far from the generation point were being ignored. The model began pre-emptively batching all tool calls including premature <code>check_answer_quality</code> .
Options Considered	Monolithic prompt (single block, <code>PROCESS</code> in middle — original design) · Split prompt (<code>ROLE/RULES</code> first, contextual injection middle, <code>PROCESS</code> last — closest to user message) · Per-iteration reminder (inject a short <code>PROCESS</code> reminder into each tool result — targeted but verbose)
Chosen Solution	Split prompt: <code>_ROLE_AND_RULES</code> (role + hard limits + tool descriptions) prepended → blocked variants + thumbdown history injected → <code>_PROCESS_INSTRUCTIONS</code> appended last
Rationale	Transformer attention shows recency bias — the last tokens before the user message receive the highest weight on the next-token decision (which is the tool-call decision). Placing <code>PROCESS</code> instructions last means they are always the closest context to the generation point regardless of how large the thumbdown injection grows. The "do NOT batch tool calls" constraint specifically targets the premature-batching failure mode observed in Run 3 Iteration 1.

Impact	Prompt evolution tracked in <code>Prompt_Changes.txt</code> . Final version adds OUTPUT FORMAT constraints (plain prose, no headings, max 400 words, citations inline only, no trailing citation lists) to prevent the 11,133-char repetition-degeneration failure observed when context dilution caused the model to vomit citations in a loop.
---------------	--

ADR-013 — Thumbsdown Persistence & Failed Variants Blocklist

Field	Detail
Decision	Persist user thumbsdowns to <code>user_thumbsdowns.json</code> and all query variants that appeared in a thumbsdown session to <code>failed_variants.json</code> ; inject both into the system prompt on subsequent queries
Date	May 2025
Context	Users could flag bad answers via thumbs-down. Without persistence, the agent would re-generate the same bad answer on the next identical query. The first design only blocked the final answer text; later analysis showed the agent was also reformulating queries into the same variants that had previously failed.
Options Considered	No persistence (agent learns nothing from thumbsdowns within a session) · Persist final answer only (blocks exact answer re-generation) · Persist all query variants from the failing session (blocks the entire search trajectory)
Chosen Solution	Persist both: <code>user_thumbsdowns.json</code> (full thumbsdown records with feedback text, query, answer, and all variants with their retrieved chunks) and <code>failed_variants.json</code> (normalized query → list of known-bad answers). On each new query, all variants from prior thumbsdown sessions are expanded into the <code>blocked</code> list and injected into the system prompt.
Rationale	Blocking only the final answer was insufficient — the agent was reformulating into the same search angles that had already failed, wasting retrieval budget. By blocking all variant queries from a thumbsdown session, the agent is forced to explore genuinely different angles on subsequent attempts. Minimum feedback length of 10 chars (<code>MIN_FEEDBACK_LEN</code>) prevents empty/accidental thumbsdowns from polluting the blocklist.

Impact	<code>feedback_store.py</code> manages both stores. <code>mark_last_bad()</code> rewrites the last JSONL record and updates <code>failed_variants.json</code> . The agent's <code>build_system_prompt()</code> injects up to the most recent thumbedown records. Cap on thumbedown injection (most recent 2) recommended to prevent prompt bloat beyond the 8B model's instruction-following ceiling (~1,800 tokens).
---------------	---

ADR-014 — Provider-Agnostic Design via OpenAI-Compatible API

Field	Detail
Decision	Design the entire LLM stack to be provider-agnostic via OpenAI-compatible REST API; resolve endpoint and credentials inside <code>main()</code> after <code>load_dotenv()</code> , never at module scope
Date	May 2025
Context	Mid-project, development shifted to testing with local TGI (HuggingFace Text Generation Inference) servers. The prior design had hardcoded model names and resolved <code>OPENAI_BASE_URL</code> at module import time (before <code>load_dotenv()</code> ran), causing silent fallback to OpenAI's real endpoint and 401 errors with the custom key. A separate bug caused 404 errors when <code>CUSTOM_API_BASE</code> was set to the full chat-completions URL (e.g., <code>.../v1/chat/completions</code>) — LangChain appended <code>/chat/completions</code> again, producing double-path 404s.
Options Considered	Module-scope constant resolution (simple, breaks with late <code>load_dotenv()</code>) · main()-scope resolution (correct, verbose) · lazy_import or property pattern (overcomplicated)
Chosen Solution	All three <code>ChatOpenAI</code> instances (<code>llm</code> , <code>merge_llm</code> , <code>judge_llm</code>) resolve <code>api_key</code> , <code>base_url</code> , and <code>model</code> inside <code>main()</code> immediately after <code>load_dotenv()</code> . <code>_normalize_openai_base_url()</code> helper strips trailing <code>/chat/completions</code> (and <code>/completions</code> , <code>/responses</code>) from <code>CUSTOM_API_BASE</code> before passing to <code>ChatOpenAI</code> . Startup log line <code>[LLM] endpoint=... model=...</code> acts as a sanity check.
Rationale	The <code>load_dotenv()</code> -before-resolution ordering is a classic Python gotcha. The normalization helper is idempotent — already-correct root URLs pass through unchanged. Making the startup endpoint visible in logs eliminates silent misdirection to wrong providers.

Impact	Compatible with Groq, OpenAI, Together AI, Ollama, vLLM, and any OpenAI-spec server. The <code>llm_http_fix.py</code> module (see ADR-019) extends compatibility to TGI servers with non-spec-compliant tool-call response formats. <code>CUSTOM_API_KEY</code> and <code>CUSTOM_API_BASE</code> environment variables allow hot-swapping providers without code changes.
---------------	---

ADR-015 — Chunking Parameters: 1000 chars / 200 overlap

Field	Detail
Decision	Set chunk size to 1000 characters with 200-character overlap using <code>RecursiveCharacterTextSplitter</code>
Date	April 2025
Context	Document chunking parameters directly affect retrieval precision (too large = low precision, too small = broken context) and the NAC neighbor-merging strategy (consecutive chunks need overlap to be safely re-mergeable).
Options Considered	500/100 (smaller, higher precision, more chunks) · 1000/200 (standard RAG configuration, balances precision and context) · 2000/400 (larger chunks, fewer splits, lower precision) · Semantic splitting (split on sentence/paragraph boundaries — more complex)
Chosen Solution	1000 chars / 200 overlap, <code>RecursiveCharacterTextSplitter</code> from LangChain
Rationale	1000-char chunks are the de facto RAG standard for PDFs with structured paragraphs — well-matched to the document types in the corpus. Smaller chunks improve precision but can break mid-sentence for structured technical content. The 200-char overlap ensures consecutive chunk pairs share enough context for NAC to safely merge them without losing facts at boundaries. Batch size tuned to 128–256 for GPU embedding (RTX 5050, 8GB VRAM) — beyond 512 there are diminishing returns with <code>all-MiniLM-L6-v2</code> .
Impact	<code>chunk_seq</code> metadata field (integer index within a document) is stored per chunk to support NAC's consecutive-chunk detection. Changing chunk parameters requires full re-ingestion.

ADR-016 — Dual Collection Strategy: documents + learned_qa

Field	Detail
Decision	Maintain two separate ChromaDB collections: <code>documents</code> (source corpus) and <code>learned_qa</code> (distilled Q&A pairs from self-learning)
Date	April 2025
Context	Self-learning via memory injection (ADR-008) required a place to store distilled Q&A pairs that is searchable at query time. The alternatives were: adding synthetic Q&A pairs back into the main <code>documents</code> collection (contamination risk), storing them in a separate file (not queryable via embedding search), or a dedicated collection.
Options Considered	Single <code>documents</code> collection (simpler, risk of contaminating source corpus with synthetic content) · Flat file store (not retrievable via similarity search) · Separate <code>learned_qa</code> collection (clean separation, independently queryable)
Chosen Solution	Separate <code>learned_qa</code> ChromaDB collection; retriever queries both and merges results
Rationale	Source documents and synthetic Q&A pairs have different provenance, different update cycles, and different trust levels. Keeping them separate allows per-collection tuning (e.g., different <code>top_k</code> for each). Deduplication by SHA-256 content hash prevents the same interaction from being distilled multiple times with slightly different wording.
Impact	<code>retriever.py</code> implements hybrid retrieval across both collections. <code>self_learner.py</code> manages the <code>learned_qa</code> collection. <code>SelfLearner.run_distillation()</code> upserts by content hash. Bug identified (see <code>Bugs.md</code>): <code>load_good(limit=10)</code> was returning already-distilled interactions, causing re-distillation — fix requires a watermark or <code>distilled: true</code> flag per JSONL record.

ADR-017 — LLM-Based Compression Faithfulness Guard (retention ratio ≥ 0.35)

Field	Detail
Decision	Add a faithfulness guard to LBC: if the compressed chunk's length is less than 35% of the original, reject the compression and keep the original
Date	May 2025
Context	LLM-Based Compression (LBC) can over-aggressively summarize chunks, dropping key facts or substituting paraphrased claims that change meaning. A hallucinated compression that replaces a specific measurement with a vague generalization is worse than no compression at all.
Options Considered	No guard (accept all LBC output — maximum compression, maximum hallucination risk) · Retention ratio threshold (reject outputs with retention $< X\%$ — simple, adjustable) · Full faithfulness check via judge LLM (most accurate, adds latency and cost per chunk)
Chosen Solution	Retention ratio threshold of 0.35 (if <code>len(compressed) / len(original) < 0.35</code> , keep original). Applied in <code>llm_based_compression()</code> in <code>context_compression.py</code> .
Rationale	A retention ratio below 35% is a strong signal of over-summarization — legitimate compression of a 1000-char chunk should not produce fewer than 350 chars of substantive content. The threshold is a fast, deterministic heuristic that adds zero latency. For chunks where LBC would over-compress, keeping the original is always safer than accepting a potentially hallucinated summary.
Impact	~27.6% average context reduction measured in testing (chunks that pass the guard). Chunks rejected by the guard are kept as-is and counted as <code>skipped</code> in the compression log. The judge LLM is used for the merge validator (ADR-011) but not per-chunk LBC (too expensive).

ADR-018 — Two-LLM Design: Separate merge_llm and judge_llm

Field	Detail
Decision	Instantiate three separate <code>ChatOpenAI</code> objects (<code>llm</code> , <code>merge_llm</code> , <code>judge_llm</code>) rather than reusing one instance for all roles
Date	May 2025
Context	Different pipeline stages have different reliability requirements. The main agent <code>llm</code> needs fast multi-turn tool-calling. The compression merge LLM needs to produce valid JSON faithfully. The judge LLM needs to return structured verdicts with per-claim granularity. These requirements suggested different configurations (temperature, model size).
Options Considered	Single LLM instance for all roles (simpler, less configuration, same quality for all calls) · Two instances (separate judge from main agent) · Three instances (separate agent, merge, and judge)
Chosen Solution	Three instances — all currently pointed at the same Groq endpoint and model, but independently configurable via <code>.env</code>
Rationale	Separation enables future hot-swapping: <code>judge_llm</code> can be upgraded to <code>llama-3.3-70b-versatile</code> (better for faithfulness evaluation) while <code>llm</code> stays on <code>llama-3.1-8b-instant</code> (faster for orchestration). <code>merge_llm</code> may benefit from a lower temperature for deterministic JSON output. Having them separate now costs nothing and enables a hybrid model architecture later without refactoring.
Impact	All three instances are constructed in <code>main()</code> immediately after <code>load_dotenv()</code> . A hybrid setup (8B for agent/orchestration, 70B for merge + judge) is identified as the primary production upgrade path based on Batch 6/8/10 quality analysis.

ADR-019 — TGI HTTP Compatibility Layer (llm_http_fix.py)

Field	Detail
Decision	Add a custom <code>httpx.Client</code> hook to intercept and repair TGI server responses before LangChain's Pydantic parser sees them
Date	May 2025
Context	During testing with a local HuggingFace TGI server, LangChain's response parsing raised Pydantic validation errors. Root cause: TGI returns <code>function.arguments</code> as a raw JSON object (<code>{"query": "..."} </code>) instead of the OpenAI-spec-required JSON-encoded string (<code>{"query\\": "\\\"...\\\""} </code>). Additionally, TGI crashed with 500 errors on multi-turn conversations when the assistant message had an empty <code>content</code> field alongside <code>tool_calls</code> .
Options Considered	Patch TGI server (not feasible — external dependency) · Post-hoc LangChain monkey-patch (fragile, version-coupled) · Custom <code>httpx.Client</code> response hook (intercepts raw HTTP before Pydantic, clean, portable) · Abandon TGI and use only Groq/OpenAI (limits local inference option)
Chosen Solution	<code>build_tolerant_http_client()</code> in <code>llm_http_fix.py</code> — an <code>httpx.Client</code> subclass with a response hook that coerces <code>arguments</code> from dict to <code>json.dumps(arguments)</code> string, and a request hook that strips empty <code>content</code> fields from multi-turn tool-call messages. Wired via <code>ChatOpenAI(http_client=build_tolerant_http_client())</code> .
Rationale	The <code>httpx.Client</code> hook intercepts the raw HTTP response before LangChain's parsing layer, making the fix completely invisible to the rest of the codebase. The two quirks (response serialization + request empty-content) are TGI-specific, so the hook is only applied when <code>CUSTOM_API_BASE</code> points to a local TGI endpoint. The fix makes the entire pipeline compatible with any OpenAI-spec server regardless of minor non-compliance.
Impact	Makes the pipeline portable across Groq, OpenAI, Together AI, Ollama, vLLM, and TGI without code changes. <code>llm_http_fix.py</code> is only active when the TGI endpoint is configured; it does not affect Groq or OpenAI traffic.

ADR-020 — `check_answer_quality` Removed from Tool Schemas; Called as Direct Function

Field	Detail
Decision	Remove <code>check_answer_quality</code> from <code>tool_schemas</code> passed to the LLM; call it directly as a Python function in the JUDGE phase
Date	May 2025
Context	Under the free-form tool loop, the 8B model frequently called <code>check_answer_quality</code> prematurely — in Iteration 1 before any retrieval had occurred — with a fabricated answer synthesized from training knowledge rather than retrieved chunks. Including it in <code>tool_schemas</code> gave the model the <i>option</i> to call it at any time, and the model exploited that option incorrectly.
Options Considered	Keep in <code>tool_schemas</code> (model controls when to call it — unreliable with 8B) · Remove from schemas, call directly (orchestrator controls the call — deterministic) · Separate schema set per phase (available only in JUDGE phase schema — possible but adds complexity)
Chosen Solution	Remove <code>check_answer_quality</code> from <code>tool_schemas</code> . Export <code>make_check_answer_quality(judge_llm)</code> as a top-level factory function. The orchestrator calls it directly in Phase 4 (JUDGE) as <code>check_answer_quality(draft_answer, context, query)</code> .
Rationale	The quality check is a deterministic orchestration step — it should always happen at a specific point (after drafting), never optionally at the model's discretion. Removing it from schemas makes it impossible for the model to call prematurely. The factory pattern keeps <code>judge_llm</code> encapsulated. On <code>check_answer_quality</code> tool-call failure (HTTP 400), the orchestrator now has a <code>try/except</code> fallback that emits the draft answer rather than aborting entirely (fixed after Q36 in Batch testing, where a tool-call failure during answer validation caused a preventable blank response).
Impact	Directly fixed the "all tool calls at once including premature <code>check_answer_quality</code> " failure observed in Dry Run Batch testing. The quality check is now guaranteed to run exactly once per JUDGE phase entry, with the complete compressed context and final draft as inputs.

ADR-021 — Centralized Configuration & Feature-Flag Gates (config.py + prompts.py)

Field	Detail
Decision	Move every pipeline constant into a single <code>config.py</code> module, every prompt string into a single <code>prompts.py</code> module, and add 18 boolean feature-flag toggles in <code>config.py</code> that gate optional pipeline stages
Date	2026-06-09
Context	Constants were scattered across at least seven modules (<code>agent_query.py</code> , <code>tools.py</code> , <code>context_compression.py</code> , <code>ingest.py</code> , <code>vector_store.py</code> , <code>feedback_store.py</code> , <code>retriever.py</code>) — tuning any single threshold required hunting for it across files. Prompts were embedded inline in each module, so a prompt revision required editing the module that used it. There was also no way to disable individual pipeline stages (sub-query generation, NAC, DC, LBC, validators, repair tiers, draft, quality check, distillation) without commenting out code — making it impossible to measure each stage's contribution to answer quality or latency.
Options Considered	Status quo (constants per-module, no toggles — simplest but unmaintainable) · Centralise constants only (one <code>config.py</code> , but pipeline stages still hard-wired on) · Centralise constants + add boolean toggles (single source of truth + per-stage on/off) · Full DI/factory pattern (each stage a swappable class — overkill at this scale) · CLI flags / environment variables (verbose at the callsite, not durable across runs)
Chosen Solution	Two modules: <code>config.py</code> holds all constants and 18 named boolean feature flags (<code>ENABLE_*</code>); <code>prompts.py</code> holds every prompt string plus the <code>THIN / THICK</code> log separators. Every module that previously hard-coded a constant or a prompt now imports it from these modules. Gateable code paths are guarded by <code>if ENABLE_...: checks</code> ; the per-stage <code>OUTPUT_FIX</code> flags AND the global <code>ENABLE_GLOBAL_LLM_OUTPUT_FIX</code> master switch must both be on for an output-fix pass to run.
Rationale	Single-source-of-truth for both constants and prompts means tuning is one-file. Boolean feature flags make the pipeline explicitly composable — the same code path can run with full validation + repair on top, or stripped down to bare retrieval + answer, just by editing flags. This is the prerequisite for ablation

	testing (see ADR-022). The master <code>ENABLE_GLOBAL_LLM_OUTPUT_FIX</code> provides a kill-switch independent of the per-stage flags, useful when comparing "with vs without any repair anywhere". Keeping prompts in their own module (rather than co-located with constants) prevents <code>config.py</code> from becoming a 500-line wall of strings.
Impact	<code>agent_query.py</code> , <code>validators.py</code> , <code>context_compression.py</code> , and <code>self_learner.py</code> all import from <code>config</code> and <code>prompts</code> instead of holding local copies. The flag wiring is partial right now: <code>ENABLE_AUTO_DISTILLATION</code> is defined and toggled by <code>run_combinations.py</code> but <code>agent_query.py</code> never reads it (Bugs.md BUG-021), and the validators' "flag off" fall-through branch crashes (Bugs.md BUG-020). Both will be fixed in the next pass. Beyond this, the centralisation is foundational for ADR-022 (ablation harness) and for future per-environment overrides (e.g., dev vs prod constants via <code>os.getenv</code>).

ADR-022 — Progressive Flag-Ladder Ablation Harness (`run_combinations.py`)

Field	Detail
Decision	Build a deterministic progressive-ladder test harness that runs the same query 10 times against <code>agent_query.py</code> , stepping evenly from all-18-flags-OFF to all-ON, with one log file per run
Date	2026-06-09
Context	The 18 feature flags introduced in ADR-021 made every stage of the pipeline optional, but without a structured way to exercise combinations, the flags were a knob without a measurement. Open questions included: which stages contribute most to answer quality? which add the most latency? which combinations crash? are any stages currently no-ops? A manual run-per-combination approach ($2^{18} = 262,144$ combinations) is intractable; full coverage is unnecessary; what's needed is a representative sweep that's reproducible across days and obvious to read in the logs.
Options Considered	Random combinations (cheap but irreproducible — same bug could appear differently across days) · Full Cartesian sweep (262,144 runs — intractable) · One-stage-on-at-a-time (18 runs — measures each stage in isolation but misses interaction)

	effects) · Progressive ladder all-OFF → all-ON (N runs, deterministic order, exposes cumulative effects and ordering-dependent crashes) · Latin-hypercube / fractional factorial (statistically efficient but harder to read in logs)
Chosen Solution	Progressive ladder of N=10 runs (configurable via <code>--runs</code>). Step 0 is all-OFF, step N-1 is all-ON, intermediate steps enable <code>round(step * 18 / (N - 1))</code> flags in fixed left-to-right <code>BOOL_FLAGS</code> order. Each run patches <code>config.py</code> in-place by regex, subprocesses <code>agent_query.py</code> with the query piped via stdin (<code>PYTHONUTF8=1</code> set), captures stdout + stderr, restores <code>config.py</code> in a finally, and writes one timestamped log per run to <code>app/run_logs/run_NNN_<ts>.txt</code> . Logs include the full flag combo at the top so each run is self-describing.
Rationale	Determinism wins over coverage at this stage. The progressive ladder makes it obvious from the log filenames alone which flags are on at each step, so a crash at run 4 immediately points to the flag(s) enabled between run 3 and run 4. The fixed <code>BOOL_FLAGS</code> order means the same run-N reflects the same configuration across days, which is essential for tracking regressions. Subprocess isolation (rather than in-process flag mutation) means a crash in one run cannot pollute state for the next — a critical property for the validator-crash class of bugs uncovered by the very first ladder run.
Impact	First 10-step ladder run uncovered two bugs: validators crash when their <code>*_OUTPUT_FIX</code> flag is off (BUG-020), and <code>ENABLE_AUTO_DISTILLATION</code> is wired but never read (BUG-021). Run 10 (all-ON) took ~5.7 hours, establishing a latency upper bound — the per-stage validators and repair tiers stack up significantly when all enabled. The harness will be re-used after each bug fix to confirm the fix lands at the right step in the ladder and to track total-time deltas as repair tiers are tuned.

ADR-023 · Separate Output Repair from Structured Parsing

Field	Detail
Decision	Even when LLM output repair is disabled, all structured responses must still be parsed into Python objects through <code>parse_to_python()</code> before entering downstream logic.
Date	2026-06-10
Context	The new configuration architecture introduced both a global output-repair flag and stage-specific output-repair flags. Dry-run testing revealed that many validators and compression stages implicitly assumed that <code>fix_llm_output()</code> had already converted responses into structured objects. When repair was disabled, raw strings reached downstream code that immediately called <code>.get()</code> , iterated results, or expected dictionary schemas. This caused multiple crashes despite the configuration combination being theoretically valid.
Options Considered	Keep repair mandatory everywhere (simplest implementation but makes output-repair flags meaningless) · Allow raw strings when repair is disabled (breaks downstream contracts and creates inconsistent behavior) · Maintain a separate lightweight parsing layer (preserves structured contracts while allowing repair experimentation)
Chosen Solution	Route all non-repaired structured outputs through <code>parse_to_python()</code> . <code>fix_llm_output()</code> remains responsible for malformed-output recovery, while <code>parse_to_python()</code> handles normal JSON-to-Python conversion when repair is disabled.
Rationale	Output repair and structured parsing solve different problems. Repair addresses malformed LLM output; parsing converts valid structured data into application objects. Disabling one should not implicitly disable the other. Maintaining a consistent downstream contract simplifies every validator and compression stage.
Impact	Eliminated multiple crashes uncovered by <code>run_combinations.py</code> . Output-repair flags now genuinely represent optional behavior instead of hidden architecture-breaking switches. Enabled reliable testing of repaired vs non-repaired execution paths.

ADR-024 · Treat `_merge_similar_chunks()` As Shared Infrastructure

Field	Detail
Decision	Reclassify <code>_merge_similar_chunks()</code> from feature-specific code to shared infrastructure and validate all callers whenever merge behavior changes.
Date	2026-06-10
Context	While investigating dry-run failures, it became clear that <code>_merge_similar_chunks()</code> was being called from both retrieval deduplication and Neighbor-Aware Compression (NAC). Earlier modifications assumed a single caller and unintentionally introduced behavior that only worked correctly in one execution path. The function had effectively become a shared architectural dependency without being treated as one.
Options Considered	Maintain separate merge implementations (avoids coupling but duplicates logic) · Continue sharing the implementation without special treatment (high regression risk) · Treat the function as shared infrastructure and validate all consumers (slightly more testing overhead but clearer architecture)
Chosen Solution	Retain the shared implementation but formally treat it as infrastructure used by multiple subsystems. Future changes must consider both retrieval and compression callers.
Rationale	The merge logic solves the same problem in both locations and duplication would create long-term maintenance costs. The actual issue was not reuse itself but failing to recognize the architectural significance of the reuse.
Impact	Reduced future regression risk, clarified ownership of merge behavior, and established broader testing requirements whenever merge logic is modified.

ADR-025 · Implement Exponential Backoff with Retry-After Support

Field	Detail
Decision	Introduce exponential backoff, jitter, and provider-supplied retry-after support for all transient LLM rate-limit failures.
Date	2026-06-10
Context	Extended dry-run execution and ladder testing triggered repeated Groq rate-limit failures. Existing retry behavior immediately retried requests after a 429 response, resulting in repeated failures, wasted requests, and exhausted retry budgets. Dry-run logs showed identical requests failing multiple times in rapid succession despite provider guidance indicating recovery windows.
Options Considered	Immediate retries (simple but ineffective under sustained rate limits) · Fixed delay retries (better but ignores provider guidance) · Exponential backoff only (industry-standard but still ignores explicit recovery windows) · Exponential backoff with retry-after support (maximizes compatibility with provider behavior)
Chosen Solution	Implement exponential backoff with configurable base delay, maximum delay, jitter, and automatic honoring of provider-supplied retry-after values whenever present.
Rationale	Providers frequently expose information about when retries are likely to succeed. Ignoring that information wastes requests and increases failure rates. Exponential backoff combined with retry-after support provides the most reliable recovery behavior while minimizing unnecessary traffic.
Impact	Improved reliability during heavy dry-run execution, reduced repeated 429 failures, introduced predictable recovery behavior, and added dedicated automated tests covering retry-after handling, backoff growth, and retry exhaustion scenarios.

ADR-026 · Use Configuration Combination Testing as a Permanent Validation Layer

Field	Detail
Decision	Treat <code>run_combinations.py</code> as a permanent architectural validation tool rather than a one-time debugging utility.
Date	2026-06-10
Context	Initial feature-combination testing exposed several failures that had never appeared during normal development. Many bugs only manifested when specific validators, compression stages, or output-repair layers were disabled. Traditional testing focused almost entirely on the fully enabled execution path and therefore missed large portions of the actual architecture.
Options Considered	Return to normal testing after bug fixes (simpler but risks future regressions) · Occasional manual combination testing (limited coverage and inconsistent execution) · Treat configuration-combination execution as a permanent validation mechanism (higher runtime cost but much better architectural visibility)
Chosen Solution	Keep <code>run_combinations.py</code> as an ongoing validation tool and use it whenever major architectural changes are introduced.
Rationale	The first large-scale combination run immediately uncovered multiple architecture-level issues, demonstrating that configuration-space validation provides information unavailable from conventional tests.
Impact	Established a repeatable process for detecting hidden coupling, unsupported execution paths, and feature-flag regressions as the system evolves.

ADR-027 · Centralize Runtime Behavior Behind Feature Flags

Field	Detail
Decision	Continue moving configurable pipeline behavior into <code>config.py</code> and treat feature flags as the primary mechanism for enabling, disabling, and benchmarking architecture components.
Date	2026-06-10
Context	Prior implementations contained hardcoded behavior scattered across retrieval, validation, compression, generation, and learning modules. This made it difficult to isolate the contribution of individual pipeline stages and complicated experimentation. The migration toward a centralized configuration architecture exposed some bugs but also significantly improved observability and testability.
Options Considered	Keep behavior embedded within individual modules (simpler locally but difficult to test globally) · Partial flag coverage (reduces complexity but limits experimentation) · Centralize major pipeline behaviors through configuration flags (more complexity but enables systematic evaluation)
Chosen Solution	Maintain centralized feature flags for retrieval, validation, compression, answer generation, quality checking, output repair, and learning subsystems.
Rationale	The ability to isolate, benchmark, and validate individual stages outweighs the additional implementation complexity introduced by configuration management.
Impact	Enabled ladder testing, architecture experimentation, per-stage benchmarking, and the discovery of several latent bugs that would otherwise have remained hidden.

ADR-028 · Two-Track Retrieval Architecture: Keep documents and learned_qa Separate End-to-End

Field	Detail
Decision	Maintain <code>documents</code> and <code>learned_qa</code> as two independent retrieval channels throughout the entire pipeline (retrieval, validation, dedup/merge, accumulation, compression) and combine them only at the LLM context boundary.
Date	2026-06-11
Context	The original architecture (ADR-016) created two ChromaDB collections but merged their results into a single ranked stream inside <code>RAGRetriever.retrieve()</code> before any downstream code saw them. Stress-test analysis identified this as a structural risk: distilled QA pairs and source documents have different provenance, different trust levels, and different conflict semantics, but merging them at retrieval erased the distinction. Source-document chunks could outrank learned-QA chunks that should have taken precedence; conversely, a stale or low-quality learned-QA entry could displace the correct source-document chunk. Validators, the dedup/merge pass, and the compression pipeline all saw mixed lists, so the agent had no place where it could express "prefer learned QA on conflict." The single <code>accumulated_chunks</code> state encoded the same flaw in the orchestrator.
Options Considered	Status quo — single merged stream (simplest, but provenance is lost). · Score boost for learned QA at the merger (cheap, but score arithmetic is fragile and doesn't survive validators dropping chunks). · Soft prompt-only precedence (instruct the LLM to prefer learned QA — was already in place but couldn't be enforced since the LLM sees only one labelled list). · Two independent tracks end-to-end (the chosen option — provenance, validation, compression, and persistence all preserve the track distinction; precedence is enforced by ordering + explicit conflict rule at the answer-context boundary).
Chosen Solution	<code>RAGRetriever.retrieve_separate()</code> queries the two collections independently and returns <code>{"documents": [...], "learned_qa": [...]}</code> . Agent state holds <code>accumulated_document_chunks</code> and <code>accumulated_learned_qa_chunks</code> . <code>validate_track()</code> , <code>accumulate_track()</code> , and <code>compress_track()</code> helpers run the per-track pipeline twice without cross-track sort/dedup/merge. The

	two lists combine only inside <code>format_precedence_context_for_llm()</code> at the LLM context boundary, and they persist as separate fields in interaction records, thumbdown variants, and distillation inputs.
Rationale	Provenance survives validation and compression intact, so precedence can be enforced by the architecture rather than by hoping the LLM honors a prompt rule. Bugs that depend on one track polluting the other (e.g. learned-QA scores filtered out by a documents-tuned threshold, or a documents merge dropping a learned-QA chunk) become impossible by construction. The cost is duplicated validator/compressor calls per query — acceptable given how cheap the validator calls are relative to the agent's overall token budget.
Impact	Closed the long-standing architectural gap documented in <code>Architecture.md</code>. Required coordinated changes across <code>retriever.py</code>, <code>tools.py</code>, <code>agent_query.py</code>, <code>context_compression.py</code>, <code>feedback_store.py</code>, <code>self_learner.py</code>, <code>config.py</code>, and <code>prompts.py</code>, plus the new <code>learned_qa_store.py</code>. Enabled ADR-030 (precedence rule) and ADR-031 (cosine-canonical learned-QA collection). Exposed a follow-on cluster of issues that the existing single-stream architecture had hidden — see <code>Bugs.md</code> BUG-031..BUG-037.

ADR-029 · Learned-QA Precedence at Conflict; Combine Only at the Answer-Context Boundary

Field	Detail
Decision	When document and learned-QA chunks would carry conflicting information, the final answer must prefer the learned-QA response. The rule is implemented by ordering the two sections in the LLM context (learned QA first), labelling them HIGH PRIORITY and SECONDARY , and prepending an explicit [CONFLICT RESOLUTION RULE] header.
Date	2026-06-11
Context	Once ADR-029 separated the two tracks, the question of how to express precedence became architectural rather than implicit. Learned-QA pairs are produced by the self-learner from interactions that already passed quality checking, so they are usable evidence about how the user wants questions answered. Source documents are the ultimate ground truth for factual claims, but they may be stale, ambiguous, or contradicted by a more recent learned correction. The system needs a deterministic rule for the conflict case — not a heuristic per-query score comparison, which is fragile.
Options Considered	Documents-first (default to source as ground truth — but defeats the point of self-learning, which is to capture corrections and preferred phrasings). · Score-weighted blending (compute a confidence per track and blend — adds latency, complicates explainability, and the per-track scores aren't directly comparable across collections anyway). · Final-answer validator that rejects answers contradicting learned QA (most rigorous but doubles the judge cost and adds a new failure mode). · Prompt-level precedence rule + ordered sections (the chosen option — cheap, explicit, and the LLM's instruction-following on conflict-resolution rules is reliable in observed runs).
Chosen Solution	<code>format_precedence_context_for_llm(learned_qa_chunks, document_chunks)</code> is the single point where the two tracks are combined into LLM-consumable text. It emits, in fixed order: a [CONFLICT RESOLUTION RULE] instruction, a [LEARNED QA CONTEXT - HIGH PRIORITY] section, then a [DOCUMENT CONTEXT - SECONDARY] section. Empty tracks are omitted. The same function is used for the DRAFT context and (currently) for the JUDGE context. The agent role/process prompts in <code>prompts.py</code> repeat the precedence rule in

	retrieval, compression, drafting, judging, and final-answer instructions, so the rule is reinforced wherever the LLM might be tempted to override it.
Rationale	Conflict precedence is a property of the answer, not of the retrieval scoring, so the right place to express it is the answer context — where the LLM actually generates. Prompt-level rules at a single combine point are auditable: one function call, one rule string, one ordering. Repeating the rule across role/process prompts hedges against the model attending to one phase's instructions and ignoring another's.
Impact	Eliminates the ambiguity that existed before separation, where merge-order in the retrieval ranker effectively chose precedence by accident. Empirically untested for true conflict cases in the dry runs to date (no document-vs-learned conflict has been observed), so the rule is correctly instructed but not yet proven under real conflict pressure. Final-answer generation is not currently re-validated for conflict-precedence compliance — recorded as BUG-032 , with a planned final-answer validator as the remediation.

ADR-030 · Canonical Cosine learned_qa Collection with Safe L2→Cosine Migration

Field	Detail
Decision	Funnel all access to the <code>learned_qa</code> Chroma collection through a single factory (<code>learned_qa_store.get_or_create_learned_qa_collection</code>) that guarantees the collection uses <code>hnsw:space = cosine</code> , and that safely migrates any pre-existing L2 collection in place with snapshot/restore/rollback.
Date	2026-06-11
Context	Dry-run inspection showed that the live <code>learned_qa</code> collection was persisted with the Chroma default L2 distance, while <code>documents</code> was created with cosine (per ADR-002 / Research topic 13). Both collections were filtered with the same <code>score_threshold = 0.5</code> and ranked by <code>similarity_score = 1 - dist</code> . On unit-normalized vectors that formula has different semantics for cosine and L2, so learned-QA results were ranked and filtered on incompatible score arithmetic — making the precedence rule from ADR-030 unsafe in practice. A second contributing problem was startup ordering: <code>agent_query.py</code> created the collection without <code>hnsw:space</code> (defaulting to L2) before <code>self_learner.py</code> requested cosine on the same name, so the cosine metadata was effectively a no-op against an already-L2 collection. Chroma does not migrate distance metric on metadata change — the underlying HNSW index has to be rebuilt.
Options Considered	Delete and re-ingest (correct but destroys ~374 distilled QA pairs — high cost and undermines the point of self-learning). · Live with mixed metrics and tune thresholds per collection (complicates the precedence rule and the validators). · Rename the existing collection and create a fresh cosine one without restore (loses data). · Snapshot → recreate → restore → verify → swap atomically, with rollback (the chosen option — zero data loss, idempotent on already-cosine collections, fails closed on any error).
Chosen Solution	<code>learned_qa_store.get_or_create_learned_qa_collection(client, name, batch_size)</code> . If the collection doesn't exist it is created with canonical cosine metadata. If it exists and is already cosine it is returned as-is. If it exists with any other space, the helper snapshots every record (ids + embeddings + metadatas + documents) in <code>batch_size</code> pages, renames the original to a UUID-

	suffixed backup, creates a new collection at the canonical name with cosine metadata, restores the snapshot, verifies that <code>count() == len(records)</code>, and only then deletes the backup. Any exception triggers rollback: the partial replacement is deleted (if it was created), the backup is renamed back to the canonical name, and the original error is re-raised. The factory is wired into both <code>agent_query.py</code> startup and <code>self_learner.py</code>, so there is only one canonical collection handle in the system.
Rationale	Production data must not be lost during a metric change. Two-phase migration with a verified count and rollback is the standard pattern. Centralising collection creation also eliminates the original startup-ordering bug — there is no longer any path that creates the collection at any other distance.
Impact	Live migration successfully moved 374 L2 entries to 374 cosine entries with all IDs preserved. The precedence rule from ADR-030 now operates on comparable score arithmetic. Reduced surface area: only <code>learned_qa_store.py</code> knows how the learned-QA collection is created; every other module imports the factory. Regression test added in <code>test_retrieval_separation.py</code> for the L2→cosine migration path.

ADR-031 · Centralized, Config-Controlled RETRIEVAL_TOP_K and RETRIEVAL_TOP_L; LLM No Longer Sets top_k

Field	Detail
Decision	Move retrieval-depth control out of the <code>retrieve_documents</code> tool schema and into two configuration constants in <code>config.py</code> : <code>RETRIEVAL_TOP_K</code> for the <code>documents</code> collection and <code>RETRIEVAL_TOP_L</code> for the <code>learned_qa</code> collection. The LLM cannot override either at call time.
Date	2026-06-11
Context	Before this change, <code>top_k</code> was a parameter on the <code>retrieve_documents</code> tool schema. The 8B model would set it idiosyncratically — sometimes 3, sometimes 10 — which made retrieval depth a function of the model's mood rather than an operating parameter. Stress-test analysis flagged this as one of the causes of unpredictable over-retrieval (see Research topic 28): in worst-case combinations of <code>MAX_TOTAL_RETRIEVALS</code> and a model-chosen <code>top_k</code> , accumulated raw chunks could approach 100 before compression. The two-track architecture also required a second depth control for the learned-QA channel, which the prior single- <code>top_k</code> schema had no way to express.
Options Considered	Keep <code>top_k</code> on the tool schema, add <code>top_l</code> too (preserves the same root problem and adds another knob for the model to misuse). · Drop <code>top_k</code> from the schema, replace with a single config constant (was the first iteration during this work, but the new two-track architecture demanded two distinct constants). · Two config constants, schema with no depth parameters at all (the chosen option — <code>RETRIEVAL_TOP_K</code> for documents, <code>RETRIEVAL_TOP_L</code> for learned QA, both env-overridable, neither exposed to the LLM). · Adaptive top-k based on prior retrieval quality (interesting but premature — there's no signal yet to drive it).
Chosen Solution	<code>config.py</code> declares <code>RETRIEVAL_TOP_K = int(os.getenv("RETRIEVAL_TOP_K", "5"))</code> and <code>RETRIEVAL_TOP_L = int(os.getenv("RETRIEVAL_TOP_L", "5"))</code> . <code>RAGRetriever.retrieve_separate()</code> consumes both as defaults. The <code>retrieve_documents</code> tool schema and docstring no longer expose any depth parameter. Working defaults are 5/5; both are env-overridable for ablation runs.

Rationale	Operating parameters belong to operators, not to a tool-using model. Centralising them in <code>config.py</code> makes them visible in ladder runs, ablation harnesses, and reproducibility audits, and removes a class of "the model retrieved too many chunks" variance from the dry-run logs. Two separate constants are needed because the two collections serve different roles — small numbers of high-precedence learned QA, larger candidate pools of source documents — and may diverge as tuning continues.
Impact	Retrieval depth is now deterministic per run. Ablation runs via <code>run_combinations.py</code> can sweep over <code>RETRIEVAL_TOP_K</code> and <code>RETRIEVAL_TOP_L</code> independently. Tool-call surface area reduced — one fewer parameter for the model to get wrong. Regression test added in <code>test_retrieval_separation.py</code> covering configured per-track limits.

ADR-032 · Centralized Logging Via logger_config.py (No Per-File Logging Setup)

Field	Detail
Decision	Replace every <code>print()</code> / <code>file.write()</code> diagnostic statement across the project with <code>logging</code> module calls routed through a single configuration module (<code>logger_config.py</code>). Every module imports <code>logging</code> and does <code>logger = logging.getLogger(__name__)</code> at module scope; configuration (handlers, levels, formatters, file targets) lives only in <code>logger_config.py</code> and is applied once per process via an idempotent <code>setup_logging()</code> call at each entry point.
Date	2026-06-15
Context	Before this change, the project produced ~370 unstructured <code>print()</code> calls across 14+ production modules, plus a handful of direct <code>open()/write()</code> blocks (notably for <code>llm_data_check.txt</code> and <code>llm_json_tries.txt</code> from <code>fix_llm_output.py</code>). Output had no level filtering, no timestamps, no per-module attribution, and no way to retain a full run's trace short of capturing terminal scrollback. Repeated dry runs were already losing diagnostic information when the operator forgot to redirect stdout or the batch-run UI overwrote earlier output. Two organisational approaches were on the table: configure <code>logging</code> per-module (each file builds its own handler and formatter) or configure it once centrally and have each module just <code>getLogger(__name__)</code> .
Options Considered	Status quo — keep <code>print()</code> everywhere (simplest, but no level filtering, no retained traces, and the diagnostic file writes in <code>fix_llm_output.py</code> were already inconsistent with that pattern). Per-module <code>logging.basicConfig</code> calls (decentralized, but each file would duplicate handler lists, formatters, and file paths — the configuration footprint grows with every new module). Central <code>logger_config.py</code> module with one <code>setup_logging()</code> function (the chosen option — configuration lives in one place; each module's only required change is <code>print(x) → logger.info(x)</code>). Structured-logging library (<code>structlog</code>, <code>loguru</code>) (more features but adds a non-stdlib dependency and a learning curve for diagnostic-quality output that stdlib <code>logging</code> already produces).
Chosen Solution	New module <code>logger_config.py</code> exporting <code>setup_logging(log_dir, app_name, console_level)</code> . The function is idempotent via a <code>CONFIGURED_ATTR</code> flag on the root logger so it's safe to call from any of the entry points (<code>agent_query.py</code> , <code>ingest.py</code> ,

	run_batch.py, query.py, test_output_fixes.py, test_llm_caller.py, run_combinations.py, top-level main.py). It installs one console handler at INFO and one per-run debug file handler at DEBUG ({app_name}_{YYYYMMDD_HHMMSS}.debug.log under run_logs/). Diagnostic loggers llm_data_check and llm_json_tries are configured with propagate=True so their output reaches both the console at effective level and the debug file at DEBUG. A custom DynamicStdoutHandler re-points to sys.stdout at emit time so batch-run tee/redirect setups don't lose console output. Every module does logger = logging.getLogger(name) at module scope. print() is preserved exclusively for interactive REPL prompts, the batch-run progress UI, the test-console formatter, and JSON/JSONL data writes.
Rationale	A central setup module is the minimum-code-change route to gaining timestamps, per-module attribution, level filtering, and retained per-run trace files. Per-file setup would have multiplied the configuration footprint by the number of modules and required updating each module again whenever a handler or formatter changed. getLogger(name) is the stdlib-recommended pattern and gives every log line a free module-name tag without any per-module configuration. The choice to drop the originally-planned separate .error.log came from a dry-run observation: errors are already visible on console at ERROR level <i>and</i> captured in the per-run debug file, so the dual-file split doubled the log-management surface for no extra signal. The choice to keep DEBUG out of the console came from a subsequent observation that third-party libraries (httpcore, httpx, groq. base_client) flood the terminal with thousands of DEBUG lines per query — useful in a file, unusable on stdout.
Impact	One new module (logger_config.py), one call site per entry point (setup_logging(app_name="...")), and a mechanical print() → logger.<level>() conversion across ~14 production modules. Per-run debug files now retain the full DEBUG trace (including third-party HTTP traffic and Groq SDK retry timings) so dry-run analysis no longer depends on terminal scrollback. Surfaced two bugs during rollout — BUG-038 (mechanical conversion silently dropped some print/write blocks) and BUG-039 (severity-level mismatches inherited from the conversion). Both closed same-day. The original agent_query.error.log and agent_query.debug.log files from the brief dual-file design are now stale and can be deleted. Future work: an audit of remaining print() callsites in run_batch.py and query.py to confirm the

	preserved ones are genuinely user-facing rather than diagnostic.
--	--

ADR-033 · Defer LangGraph Adoption; Keep Hand-Rolled State Machine

Field	Detail
Decision	Continue with the hand-rolled phase state machine in <code>agent_query.py</code> for the foreseeable future. Do not port the pipeline to LangGraph. Leave the existing <code>langgraph</code> declaration in <code>requirements.txt</code> as a transitional artefact until a follow-on cleanup pass either removes it (if no migration is planned in the near term) or imports it (if a future ADR reverses this decision).
Date	2026-06-15
Context	The pipeline has grown into a multi-phase loop (RETRIEVE → COMPRESS → DRAFT → JUDGE, with iteration budgets, retry paths, thumbdown injection, two-track per-stage handling, and protocol-enforcement nudges). LangGraph is the natural off-the-shelf framework for this kind of workflow: it provides a typed <code>StateGraph</code> , named nodes and conditional edges, native checkpointing for resume-after-failure, parallel fan-out branches with automatic merge, human-in-the-loop pause/resume, and time-travel inspection of past checkpoints. The question was whether the current orchestrator's complexity has crossed the threshold where adopting LangGraph would pay off, or whether the hand-rolled state machine is still cheaper to maintain than the framework migration would cost.
Options Considered	Port <code>agent_query.py</code>'s state machine to a LangGraph <code>StateGraph</code> now (clean architecture, native parallel branches, native checkpointing, but a substantial rewrite of all phase orchestration, protocol enforcement, two-track helpers, and the test harnesses, with no guaranteed runtime speedup). · Adopt LangGraph only for the parts that benefit most (parallel sub-query retrieval, parallel per-track validation — but partial adoption means two orchestration patterns coexist in the codebase). · Build a minimal in-house checkpointing layer (JSON or SQLite checkpoint per phase boundary — gets the resume-after-failure benefit without LangGraph). · Defer; keep the hand-rolled state machine (the chosen option — the present orchestrator is already working, well-traced under the new logging architecture, and the remaining pipeline issues — BUG-

	031..037 from the post-separation dry run — are about correctness, not orchestration).
Chosen Solution	Defer LangGraph adoption. The state machine in <code>agent_query.py</code> remains the orchestrator. The <code>langgraph</code> package stays in <code>requirements.txt</code> as a declared-but-unused dependency until a follow-on cleanup pass either removes it or a future ADR reverses this decision.
Rationale	LangGraph does not make any individual node faster — its real value is in <i>parallel fan-out</i>, <i>conditional early exit</i>, <i>durable checkpoint recovery</i>, and <i>human-in-the-loop pause/resume</i>. For this project's specific phase sequence, the compression/draft/judge tail is strictly sequential (NAC must finish before DC; DC before LBC; LBC before draft; draft before judge) so the framework cannot parallelise the longest path. The genuine parallel opportunities (multiple sub-queries during RETRIEVE; per-track validation during VALIDATE) would benefit from concurrency but can be implemented with <code>asyncio.gather</code> or a thread pool without the architectural cost of a framework port. Durable checkpoint recovery requires a persistence backend (SQLite, Postgres, JSON files) that the project doesn't currently use; <code>InMemorySaver</code> would only recover within a live process, which the current <code>try/except</code> retry paths and the exponential-backoff logic in <code>llm_caller.py</code> (ADR-026) already handle for the dominant failure modes (Groq 429, transient HTTP errors). The framework's value rises sharply once the orchestrator includes human-in-the-loop review steps, long-running workflows that may span deployment cycles, or genuinely parallel retrieval strategies — none of which are present today.
Impact	No code changes today. The decision is documented so that future contributors don't re-litigate the same evaluation. <code>langgraph</code> remains a declared dependency until a follow-on cleanup removes it. The decision is explicitly revisitable: when (a) parallel sub-query retrieval is implemented, or (b) human-in-the-loop review enters the pipeline, or (c) long-running workflows that must survive process restarts become a requirement, this ADR should be re-opened.

ADR-034 · Pause In-Progress LangGraph Migration; Build Fundamentals via a Throwaway Practice Graph First

Field	Detail
Decision	Stop the direct, file-by-file migration of <code>agent_query.py</code> 's state machine onto LangGraph that had already begun (<code>app/AgentState.py</code>), delete the in-progress migration files, and instead build a small, disposable practice graph (<code>tempFile.py</code>) wired to the project's real retrieval components to learn <code>StateGraph</code> mechanics before resuming any production port.
Date	2026-06-16
Context	ADR-034 (2026-06-15) deferred LangGraph adoption pending a concrete reason to revisit it, but exploration of the framework's syntax continued the next day and escalated past syntax review into an actual migration attempt: <code>app/AgentState.py</code> was created as a full <code>TypedDict</code> mirror of the production agent's runtime state (<code>query</code> , <code>messages</code> , <code>phase</code> , both accumulated chunk tracks, token counters, etc.), alongside a toy script (<code>agent_workflow.py</code>) used to debug basic <code>StateGraph</code> compile/visualize mechanics. This put migration-shaped code into the <code>app/</code> package before there was hands-on familiarity with conditional edges, graph compilation, or visualization tooling — the toy script alone surfaced three mechanical mistakes in one sitting (missing <code>main</code> guard, Jupyter-only <code>IPython.display</code> used in a terminal script, calling <code>.get_graph()</code> on the uncompiled <code>StateGraph</code> instead of the compiled app).
Options Considered	Continue the direct migration (keep building out <code>AgentState.py</code> and port <code>agent_query.py</code> node-by-node) — fastest path to a finished migration, but ADR-034 was never actually reversed by a concrete driver (no parallel sub-queries, no human-in-the-loop step, no durable-checkpoint need), so building it now would be migrating for the framework's sake rather than a demonstrated benefit. · Abandon LangGraph entirely, delete everything including the practice work — consistent with ADR-034's deferral, but discards genuinely useful syntax and mechanics knowledge gained while exploring. · Pause the production-shaped files; keep practicing on small, disposable graphs wired to real project objects (the chosen option) — preserves ADR-034's deferral, avoids leaving half-finished migration code in <code>app/</code> , and builds the operational familiarity (conditional edges, compilation, visualization, checkpointing) needed to execute a future migration correctly if one is ever

	undertaken.
Chosen Solution	Deleted <code>app/AgentState.py</code> and <code>agent_workflow.py</code> . Created <code>tempFile.py</code> at the project root (outside <code>app/</code> , importing from it) — a minimal two-node graph (<code>retrieve</code> → <code>generate_answer</code>) using the project's actual <code>EmbeddingManager</code> , <code>VectorStore</code> , <code>RAGRetriever</code> , and <code>learned_qa_store</code> collection, with a conditional edge to <code>END</code> when retrieval returns nothing. This file is explicitly non-architectural and will be deleted once it has served its learning purpose. No production module was modified.
Rationale	ADR-034 already concluded LangGraph offers no benefit to this project's strictly sequential compression/draft/judge tail, and that the genuine opportunities (parallel sub-query retrieval, human-in-the-loop, durable checkpointing) aren't present requirements today. Starting a full migration anyway, before those triggers exist and before a single graph had been run by hand, risked the same class of mistake the toy script immediately exposed (state-vs-config confusion, compiled-vs-uncompiled graph confusion) — but inside the production codebase instead of a disposable file. Practicing against real project objects, rather than a fully synthetic example, gives a closer rehearsal of the eventual migration without the cost of being wrong inside <code>agent_query.py</code> .
Impact	No functional change to the running agent. <code>app/AgentState.py</code> and <code>agent_workflow.py</code> no longer exist. <code>tempFile.py</code> exists at the project root as scratch code, not a tracked architectural module, and should not be treated by future ADRs as a migration artifact — it will be deleted. ADR-034's deferral stands unchanged; this ADR documents the process decision (learn-by-building-small before porting live code) rather than reversing the substantive defer-or-adopt question. Findings from the hands-on session are captured in <code>Research.md</code> topic 31.

ADR-035 · Adopt LangGraph for the Production Pipeline via a Parallel `app_workflow/` Package

Field	Detail
Decision	Implement the full production RAG pipeline in LangGraph inside a new <code>app_workflow/</code> package, running in parallel with the existing <code>app/</code> package. <code>app/</code> remains untouched and functional throughout; <code>app_workflow/</code> is built incrementally until it reaches feature parity.
Date	2026-06-17
Context	ADR-034 (2026-06-15) deferred LangGraph adoption because the compression/draft/judge tail is strictly sequential and the genuine parallel opportunities (sub-query retrieval, per-track validation) had not yet been built. ADR-035 (2026-06-16) paused a premature direct migration and substituted a disposable practice sandbox (<code>tempFile.py</code>). By 2026-06-17, the sandbox had demonstrated the core LangGraph mechanics (parallel <code>Send</code> fan-out, <code>Annotated</code> fan-in reducer, conditional edges, <code>draw_mermaid_png</code> visualization) against real project objects. The two-track retrieval architecture introduced on 2026-06-11 (ADR-029) also surfaced a concrete parallel opportunity: document and learned-QA tracks can be validated, dedup-merged, and compressed independently and simultaneously — exactly the fan-out/fan-in pattern LangGraph is built for. These two facts together — working hands-on knowledge of LangGraph mechanics, and a confirmed parallel workload — are the triggers ADR-034 identified as the conditions for reversing the deferral.
Options Considered	Direct migration — replace <code>app/agent_query.py</code> in-place (fastest path to a single codebase, but risks breaking the working production agent during incremental work). · Fork <code>app/</code> and migrate the fork (avoids risk to the live agent but creates two diverging copies with no clear boundary). · New parallel package <code>app_workflow/</code> that imports from <code>app/</code> for services (the chosen option — clean separation; <code>app/</code> remains a fallback at any point; <code>app_workflow/</code> can import <code>app/</code>'s service modules rather than duplicating them). · Abandon LangGraph, implement parallelism with <code>asyncio.gather</code> in <code>agent_query.py</code> (lower adoption cost, but foregoes native fan-out/fan-in wiring, conditional early-exit edges, and the graph-topology visualisation the team has found useful for dry-run debugging).

Chosen Solution	New <code>app_workflow/</code> package. Service modules are copied from <code>app/</code> into <code>app_workflow/services/</code> for now (and <code>sys.path</code> patched so <code>app/</code> bare imports resolve from <code>app_workflow/</code>). Node files live in <code>app_workflow/nodes/</code> , one per pipeline stage. Graph wiring in <code>app_workflow/graph.py</code> . <code>app/</code> is never modified.
Rationale	The parallel package approach satisfies the two new triggers from ADR-034 without any risk to the working production agent. It also provides a natural gate: <code>app_workflow/</code> must reach feature parity with <code>app/agent_query.py</code> (including feedback recording, thumbdown persistence, and self-learning) before <code>app/</code> can be retired. Running both in parallel makes that parity comparison testable. The LangGraph fan-out/fan-in model is a better architectural fit for the two-track pipeline than hand-rolled helpers: <code>Annotated[list[dict], operator.add]</code> on both <code>retrieved_document_chunks</code> and <code>retrieved_learned_qa_chunks</code> gives parallel accumulation for free, and the two compression track pipelines (documents: <code>NAC→DC→LBC</code> ; learned_qa: <code>DC→LBC</code>) converge at <code>combine_tracks</code> via native LangGraph fan-in rather than a manual merge call.
Impact	<code>app/</code> continues to run unchanged. <code>app_workflow/</code> is the new development target. Two known parity gaps remain open after the initial build (June 17–18): <code>user_thumbdowns.json</code> <code>variants</code> field is always empty; <code>failed_variants.json</code> is never written by <code>app_workflow/</code> . Both must be closed before <code>app/</code> can be retired. Introduces a <code>services/</code> duplication between the two packages that should be consolidated in a follow-on cleanup once parity is confirmed.

ADR-036 · Split `app_workflow/nodes/nodes.py` Into Per-Node Files; `config.py` Promoted to Package Root

Field	Detail
Decision	Replace the monolithic <code>app_workflow/nodes/nodes.py</code> (all node functions in one file) with one file per pipeline stage, each importing only its required services. Promote <code>config.py</code> from <code>services/</code> to the <code>app_workflow/</code> root so both <code>graph.py</code> and node files can import constants without a circular path through <code>services/</code> .
Date	2026-06-18
Context	The day-1 <code>nodes/nodes.py</code> grew to include all REPL command nodes, query variant generation, retrieval, all six compression nodes (NAC/DC/LBC × 2 tracks), dedup-merge, combine-tracks, and answer generation in a single file. It had already accumulated import issues (undefined loggers, stale unused imports, inconsistent relative paths) by the first run. The <code>config.py</code> inside <code>services/</code> created a circular-import risk: <code>graph.py</code> needs routing constants, which live in <code>config.py</code> , which is inside the <code>services/</code> package that also imports from <code>graph.py</code> indirectly via node files.
Options Considered	Keep monolithic <code>nodes.py</code> (lower initial overhead but poor for incremental development — adding or modifying one node requires reading the entire file). · One file per node (the chosen option — mirrors the <code>app/</code> module convention and the standard LangGraph modular layout discussed in the June 17 fundamentals session). · Group by pipeline phase (e.g. <code>retrieval_nodes.py</code> , <code>compression_nodes.py</code>) — a middle ground, but phases don't map cleanly to import dependencies.
Chosen Solution	One file per conceptual group: <code>user_input.py</code> , <code>commands.py</code> , <code>query_variants.py</code> , <code>retrieve.py</code> , <code>validate_retrieval.py</code> , <code>dedup_merge.py</code> , <code>nac.py</code> , <code>dc.py</code> , <code>lbc.py</code> , <code>combine_tracks.py</code> , <code>generate_answer.py</code> , <code>no_context_answer.py</code> . <code>nodes/__init__.py</code> re-exports all public node functions. <code>nodes/nodes.py</code> deleted. <code>config.py</code> moved from <code>services/config.py</code> to <code>app_workflow/config.py</code> .
Rationale	Per-file nodes match the architectural separation the team already uses for <code>app/</code> production modules, are consistent with the modular LangGraph layout learnt during the June 16–17 fundamentals sessions, and make it straightforward to find and modify any single pipeline stage without reading hundreds of

	unrelated lines. Moving <code>config.py</code> to the package root resolves the circular-import risk and makes <code>RETRIEVAL_TOP_K</code> , <code>MAX_TOTAL_RETRIEVALS</code> , and the <code>ENABLE_*</code> flags importable from a stable location that both <code>graph.py</code> and any node file can reference.
Impact	<code>nodes/nodes.py</code> no longer exists; <code>nodes/__init__.py</code> is the import surface for graph wiring. <code>app_workflow/config.py</code> is the canonical location for all <code>app_workflow/-</code> level configuration constants. Any future node additions follow the one-file-per-node convention. No change to <code>app/</code> .

ADR-037 · Split `generate_answer` into `generate_draft` / `check_answer_quality` / `auto_distillation`, Mirroring `app/agent_query.py`'s Budget-Gated JUDGE Loop

Field	Detail
Decision	Replace the single <code>generate_answer</code> node (which had grown to handle drafting, quality-checking, and answer finalisation in one function) with four nodes — <code>generate_draft</code> , <code>check_answer_quality</code> , <code>generate_answer</code> (now a thin terminal node), and <code>auto_distillation</code> — connected by conditional edges that reproduce <code>app/agent_query.py</code> 's JUDGE-phase budget check (<code>budget_ok = ENABLE_SUB_QUERY_GENERATION</code> and <code>retry_count < LLM_RESPONSE_RETRY_LIMIT</code>) rather than refining the draft inline inside the quality-check node.
Date	2026-06-19
Context	<code>generate_answer</code> was the last unsplit pipeline stage in <code>app_workflow/</code> , controlling three independently-flagged behaviors (<code>ENABLE_ANSWER_DRAFT_CREATION</code> , <code>ENABLE_ANSWER_QUALITY_CHECK</code> , <code>ENABLE_AUTO_DISTILLATION</code>) from inside one function body — the same kind of monolith ADR-037 had already split <code>nodes.py</code> to avoid at the file level. A first pass wired three new nodes (<code>draft_answer</code> , <code>check_answer_quality</code> , <code>auto_distillation</code>) with simple boolean-flag edges, but <code>check_answer_quality</code> refined the draft internally on an <code>INSUFFICIENT</code> verdict — which doesn't have a way to re-run retrieval with new query variants, unlike <code>app/agent_query.py</code> 's JUDGE phase, which loops back to RETRIEVE when budget remains and only falls back to a best-effort answer when it doesn't.
Options Considered	Keep quality-check-with-inline-refinement (the first pass) — simpler graph, but an <code>INSUFFICIENT</code> verdict can only be patched by asking the same LLM to try again with the same context, never by fetching new evidence; this diverges from <code>app/</code> 's actual self-correction behavior. · Route <code>INSUFFICIENT</code> back to <code>generate_query_variants</code> when budget remains (the chosen option) — reproduces the real retry semantics from <code>app/agent_query.py</code> , using the graph's existing retry edge and <code>LLM_RESPONSE_RETRY_LIMIT</code> rather than inventing a new bound. · Add a dedicated <code>refine_draft</code> node — closer to a true 1:1 port of the inline refinement, but duplicates logic already covered by re-running the existing <code>generate_draft</code> node with

	fresh context after a retry loop.
Chosen Solution	generate_draft.py only produces state["draft"]. check_answer_quality.py only runs the judge and stores state["quality_verdict"] — no refinement. routes.py's route_after_quality_check computes budget_ok and returns "retry" (→ generate_query_variants) when INSUFFICIENT with budget remaining, or "generate_answer" (best-effort) otherwise. generate_answer.py is the single terminal node: uses state["draft"] when present, derives quality from quality_verdict, and always logs feedback. auto_distillation.py is a thin one-liner calling self_learner.run_distillation(). The originally-named draft_answer.py was deleted and replaced by generate_draft.py during the redesign.
Rationale	The graph already has a retry edge (generate_query_variants ← retrieval/compression failure paths) and a budget bound (LLM_RESPONSE_RETRY_LIMIT) built for exactly this purpose — re-using it for the quality-check loop avoids introducing a second, parallel retry mechanism with separate semantics. Keeping check_answer_quality refinement-free also keeps each node single-purpose (judge vs. retry vs. finalise), continuing the per-node-single-responsibility convention from ADR-037 and Research topic 32 (node vs. service boundary).
Impact	state.py gained draft: NotRequired[str] and quality_verdict: NotRequired[str]. routes.py gained route_after_generate_draft, route_after_quality_check, route_after_generate_answer. graph.py's answer pipeline is now combine_tracks → generate_draft → check_answer_quality → generate_answer → auto_distillation → END, each arrow conditional on its governing ENABLE_* flag. generate_draft.py and generate_answer.py also converted their llm.invoke() calls to llm_invoke() for consistency with the rest of app_workflow/.

ADR-038 · Subprocess-Isolated benchmark.py for app/ vs app_workflow/ Comparison, Abandoned as Unreliable Under Shared Rate Limits

Field	Detail
Decision	Build a single <code>benchmark.py</code> at the project root that runs a fixed query against either <code>app_workflow.main</code> or <code>app.agent_query</code> (selected via a <code>workflow/agent</code> CLI argument) three times each, in isolated subprocesses, appending timing + answer results to a shared <code>benchmark_results.txt</code> . After repeated 429 rate-limit storms persisted even with a 2-minute inter-run delay, abandon automated side-by-side benchmarking in favour of manually running each pipeline separately, with pauses, and checking output by hand.
Date	2026-06-19
Context	With <code>app_workflow/</code> having reached rough feature parity with <code>app/</code> after the June 17–19 build-out, a direct timing and answer-quality comparison was needed. Both projects define a bare <code>config</code> module at their respective roots, so importing both into a single Python process causes module-name collisions — <code>app.config</code> and <code>app_workflow.config</code> cannot coexist as the same imported name without one shadowing the other when both are added to <code>sys.path</code> simultaneously.
Options Considered	In-process comparison (import both pipelines into one script) — simplest code, but blocked by the <code>config</code> module collision described above; would require renaming or restructuring one project's modules just for benchmarking, which wasn't acceptable. · Subprocess isolation per system (the chosen option) — each worker subprocess only puts its own project's directories on <code>sys.path</code> , so no collision occurs; results are passed back via sentinel-delimited stdout (<code>%%BENCH_ELAPSED%%=, %%BENCH_ANS_START%% / %%BENCH_ANS_END%%</code>) parsed by the orchestrator. · Two separate benchmark scripts, one per system — avoids the collision without subprocesses, but loses the single-file, single-invocation convenience and makes the side-by-side average comparison harder to keep in sync.
Chosen Solution	One <code>benchmark.py</code> that doubles as orchestrator and worker: <code>python benchmark.py workflow</code> or <code>python benchmark.py agent</code> runs only the requested system 3 times via

	subprocess.run([sys.executable, "benchmark.py", "--_mode=...", QUERY]) and appends a self-contained results block to <code>benchmark_results.txt</code> (revised from an initial version that ran both systems automatically and overwrote the file). A 2-minute <code>time.sleep()</code> was inserted between consecutive runs within one invocation to let Groq's per-minute rate limits recover.
Rationale	Subprocess isolation is the only option that avoids the <code>config</code> module collision without restructuring either production project, and keeps the benchmark tool fully external to both pipelines (no changes needed inside <code>app/</code> or <code>app_workflow/</code> to support it). Append-mode output lets <code>python benchmark.py workflow</code> and <code>python benchmark.py agent</code> be run independently (e.g. on different days) while still building up a comparable <code>benchmark_results.txt</code>.
Impact	<code>benchmark.py</code> exists as a 321-line standalone tool at the project root; neither <code>app/</code> nor <code>app_workflow/</code> were modified to support it. Despite the 2-minute inter-run delay, side-by-side runs of both pipelines still triggered Groq 429 rate-limit retry storms (confirmed in <code>agent_query_20260619_185829_debug.log</code> and <code>rag_langgraph_20260619_183828_debug.log</code>), making wall-clock comparisons unreliable for today's session. No further automated-benchmark work was done after this; manual pipeline-by-pipeline checking with pauses was used instead. The benchmark tool itself is not deprecated — only today's attempt to run it back-to-back without a larger gap between systems.

ADR-039 · Raise LLMRateLimitAbortError Instead of Sleeping When Backoff Exceeds Maximum

Field	Detail
Decision	When <code>llm_invoke()</code> 's computed exponential-backoff delay would exceed <code>LLM_RATE_LIMIT_MAX_DELAY_SECONDS</code> (1800 s), raise <code>LLMRateLimitAbortError(delay)</code> immediately rather than sleeping for the full interval.
Date	2026-06-20
Context	June 19 benchmark runs produced Groq 429 storms where retry delays kept compounding — at the extreme, <code>llm_invoke()</code> would have entered a 30-minute sleep. In extended dry-run sessions (no interactive user) this is merely wasteful; once HTTP APIs were added (ADR-041), an indefinitely sleeping handler blocks the server for every other caller with no visible feedback and no way for the client to know whether to wait or abort.
Options Considered	Sleep indefinitely (original behavior, no cap) — requires no new exception type, but a 30-minute handler occupies a server thread/coroutine slot with no client feedback. · Cap sleep to <code>LLM_RATE_LIMIT_MAX_DELAY_SECONDS</code> — bounds the wait but still blocks for up to 30 minutes, which is unacceptable for an HTTP API. · Abort and propagate via exception (chosen) — caller decides how to respond; CLI callers can print an actionable "retry after X seconds" message; HTTP handlers can return a 429 with <code>Retry-After</code> ; the server remains responsive for other requests immediately.
Chosen Solution	<code>LLMRateLimitAbortError(delay: float)</code> — a lightweight Exception subclass carrying the would-be delay. Added to <code>app/llm_caller.py</code> and <code>app_workflow/services/llm_caller.py</code> . <code>LLM_RATE_LIMIT_MAX_DELAY_SECONDS = 1800.0</code> added to both <code>config.py</code> files. <code>app/agent_query.py</code> , <code>app_workflow/main.py</code> , <code>app/main.py</code> , <code>app/api.py</code> , and <code>app_workflow/api.py</code> all catch <code>LLMRateLimitAbortError</code> and surface it as an actionable message or HTTP 429.
Rationale	Aborting is the only option that keeps the HTTP server responsive under sustained rate limiting. The 1800 s threshold is generous — a delay of that magnitude almost certainly signals a quota suspension rather than a transient burst, so sleeping would just waste time. CLI callers receive a clear "retry after N seconds" message they can act on. The exception carries <code>delay</code> as

	a float so callers can pass it directly to <code>Retry-After</code> without recomputing it.
Impact	Any call stack that passes through <code>llm_invoke()</code> can now raise <code>LLMRateLimitAbortError</code> in addition to the existing <code>RateLimitError</code> from the Groq client. Callers that did not previously catch anything must now handle this error or let it propagate to the top-level handler. HTTP 429 responses now carry a <code>Retry-After</code> header derived from <code>e.delay</code> .

ADR-040 · Expose Both Pipelines Over HTTP via FastAPI (app/api.py and app_workflow/api.py)

Field	Detail
Decision	Add a FastAPI HTTP server to each pipeline package (app/api.py on port 8000, app_workflow/api.py on port 8001) exposing five endpoints: POST /query, POST /feedback/bad, GET /stats, POST /learn, POST /quit.
Date	2026-06-20
Context	Both pipelines were CLI-only. Running them programmatically required spawning a subprocess (as benchmark.py demonstrated) or importing the package directly — both fragile options. Adding an HTTP layer enables language-agnostic clients, browser-based UIs, and automated testing without restructuring either package. June 19's benchmark work underscored that the pipelines needed a more stable programmatic interface.
Options Considered	Flask — synchronous, requires threading for concurrent requests, no built-in request validation, no auto OpenAPI docs. · FastAPI (chosen) — async-native, asynccontextmanager lifespan handles one-time startup cost (embedding load, vector-store open, LLM client init) cleanly, Pydantic request/response validation, auto-generated /docs (Swagger) and /redoc out of the box. · Django REST Framework — heavyweight, ORM-oriented, overkill for a single-module inference server. · CLI subprocess wrapper — keeps both pipelines CLI-only; adds latency and complexity for the caller without gaining any structured request/response protocol.
Chosen Solution	FastAPI with asynccontextmanager lifespan for startup/teardown; asyncio.Lock(_query_lock) to serialise concurrent /query calls (shared agent_state / GraphState dicts are not re-entrant); asyncio.to_thread() to dispatch blocking LLM/pipeline calls from async handlers. app/api.py initialises all pipeline objects directly in the lifespan (three ChatGroq instances, EmbeddingManager, VectorStore, RAGRetriever, FeedbackStore, SelfLearner). app_workflow/api.py delegates to the existing services/services.py registry and calls build_graph() at startup. Both servers catch LLMRateLimitAbortError and return HTTP 429 with Retry-After. POST /quit sends SIGTERM to the process for clean teardown.

Rationale	FastAPI's async model avoids the overhead of a thread pool while still handling multiple in-flight connections; <code>asyncio.to_thread()</code> bridges the blocking pipeline calls without blocking the event loop. The lifespan context manager guarantees that expensive one-time initialisations (embedding model load, ChromaDB open) happen exactly once at startup and are cleaned up at shutdown, eliminating the per-request cost that made <code>benchmark.py</code>'s subprocess approach slow. Pydantic request models (<code>QueryRequest</code>, <code>FeedbackRequest</code>) enforce types at the boundary without adding any code beyond the model definition. <code>app_workflow/api.py</code>'s reserved-word guard (<code>_COMMAND_INPUTS</code>) prevents CLI command strings from being passed as queries, which would silently produce wrong answers.
Impact	Two new server entry points: <code>app/api.py</code> (run with <code>uvicorn api:app --port 8000</code> from <code>app/</code>) and <code>app_workflow/api.py</code> (run with <code>uvicorn api:app --port 8001</code> from <code>app_workflow/</code>). <code>app/</code> paths remain CWD-relative; <code>os.chdir(_APP_DIR)</code> at module load ensures they resolve regardless of uvicorn's working directory. <code>app_workflow/</code> paths use <code>file</code>-derived <code>PROJECT_ROOT</code> (unchanged from the BUG-045 fix). <code>API_ENDPOINTS.txt</code> documents all endpoints with request/response schemas and <code>curl</code> examples. Neither pipeline's core logic was modified.

ADR-041 · ThreadPoolExecutor-Based Timeout Wrapper for Blocking I/O Operations

Field	Detail
Decision	Wrap every long-running blocking I/O call (<code>llm.invoke()</code> , <code>collection.query()</code> , <code>model.encode()</code>) in a <code>concurrent.futures.ThreadPoolExecutor(max_workers=1)</code> thread and enforce a per-module deadline via <code>future.result(timeout=N)</code> . Caught timeouts raise a typed exception (<code>LLMResponseTimeoutError</code> , <code>RetrievalTimeoutError</code> , <code>EmbeddingEncodingTimeoutError</code>) rather than propagating <code>concurrent.futures.TimeoutError</code> directly.
Date	2026-06-22
Context	Unguarded calls to Groq's LLM API, ChromaDB's <code>.query()</code> , and SentenceTransformers' <code>model.encode()</code> have no enforced upper time bound. Under network degradation, provider-side queuing, or GPU contention, any of these can stall indefinitely. During the June 22 benchmark session, individual LLM calls were observed taking up to 75 s and the total app/ pipeline wall-clock reached 28 min 28 s. Without a timeout guard, a stalled call blocks the entire pipeline — and, for the HTTP API layer added in ADR-041, the Uvicorn server thread — with no recourse for the caller.
Options Considered	· <code>signal.alarm(N)</code> (POSIX signals) — raises <code>SIGALRM</code> after N seconds, hard-interrupting the calling thread. Not available on Windows (the project's development platform) and cannot interrupt calls that hold the GIL, which includes C extensions (NumPy inside SentenceTransformers, ChromaDB's native layer). · Per-library timeout parameters — Groq's <code>ChatOpenAI</code> constructor accepts a <code>request timeout</code>; ChromaDB's <code>.query()</code> and SentenceTransformers' <code>encode()</code> do not expose timeout parameters. Partial coverage only — unacceptable for a consistent cross-module contract. · <code>ThreadPoolExecutor</code> + <code>future.result(timeout=N)</code> (chosen) — works on Windows, wraps any blocking call regardless of whether the library exposes a timeout, and expresses the deadline on the calling side without modifying the call itself.
Chosen Solution	<code>concurrent.futures.ThreadPoolExecutor(max_workers=1)</code> submits the blocking call and immediately calls <code>future.result(timeout=N)</code> . The calling thread unblocks after N seconds if the future has not completed; the background thread continues to completion but its result is discarded. A per-module

	typed exception is raised from <code>concurrent.futures.TimeoutError</code> and caught at the same or one frame above. LLM and retrieval timeouts return safe fallbacks (<code>LLMResult(ok=False)</code>, <code>[]</code>) to allow pipeline continuation; embedding timeouts are re-raised because there is no meaningful embedding vector fallback. Thresholds set to $2\times$ the empirical p95 for LLM (150 s); $\sim 14\times$ p95 for retrieval (10 s); $\sim 500\times$ p95 for embedding (10 s) — margins calibrated after the June 22 benchmark runs (BUG-051).
Rationale	The <code>ThreadPoolExecutor</code> pattern is the only option that: (a) works on Windows, (b) is library-agnostic, and (c) requires no changes inside third-party calls. The key caveat — the background thread is not killed, only abandoned — is acceptable here because Groq calls eventually complete or fail with a provider error, ChromaDB is local and fast, and SentenceTransformers encodes from memory with no network I/O, so resource leakage is bounded. Typed exception classes give callers a precise signal without exposing <code>concurrent.futures</code> internals to every call site.
Impact	<code>app/llm_caller.py</code>, <code>app/retriever.py</code>, <code>app/embedding_manager.py</code>, <code>app_workflow/services/llm_caller.py</code>, <code>app_workflow/services/retriever.py</code>, <code>app_workflow/services/embedding_manager.py</code> — each gained one try/except block around its I/O call, one typed exception class, and <code>import concurrent.futures</code>. Three new constants added to both <code>config.py</code> files: <code>LLM_RESPONSE_TIMEOUT_SECONDS</code>, <code>RETRIEVAL_TIMEOUT_SECONDS</code>, <code>EMBEDDING_ENCODING_TIMEOUT_SECONDS</code>. The pipeline can now surface a <code>TIMEOUT</code> result kind rather than hanging indefinitely, and the HTTP API layer remains responsive even when a downstream call stalls.

ADR-042 · Per-Run JSON Timing Files via a Module-Level Singleton Timing Tracker

Field	Detail
Decision	Introduce a module-level singleton timing tracker in each pipeline package (<code>app/timing_tracker.py</code> , <code>app_workflow/services/timing_tracker.py</code>) that records per-stage wall-clock durations using <code>time.perf_counter()</code> and flushes results to a JSON file after every <code>record()</code> call. The JSON file is named identically to the per-run debug log (same stem, <code>.debug.json</code> extension) and lives alongside it in <code>run_logs/</code> .
Date	2026-06-23
Context	June 22's benchmark produced per-pipeline total wall-clock averages (13:51 vs 28:28) but no per-stage breakdown. Understanding which stages account for the gap — and how much of any stage's time comes from LLM calls, retries, backoff, or framework overhead — requires probes that capture durations at each node boundary. The existing DEBUG log carries LLM timing fragments via <code>llm_caller.py</code> 's per-attempt lines, but they are buried in unstructured text, require manual parsing, and cover only LLM calls — not embedding, retrieval, compression, or draft generation. A structured, machine-readable output alongside the existing log was needed to turn benchmark run output into directly plottable per-category numbers without a log-parser.
Options Considered	· Parse timing from the existing DEBUG log — zero new code, but requires a brittle log-parser for every analysis and cannot capture stages that do not currently emit timing messages. · Append a timing section to the same DEBUG log — minimal file proliferation, but mixes structured numeric data with unstructured diagnostic prose, making both harder to work with. · Write a separate structured JSON file per run (chosen) — keeps the debug log human-readable and the timing data machine-readable without coupling them; the identical naming scheme makes it unambiguous which JSON belongs to which run. · Persist timing to a shared SQLite or CSV — supports cross-run aggregation natively, but adds a schema concern and requires coordination if two server processes run simultaneously; the per-file approach inherits the natural isolation the per-run log file already provides.
Chosen Solution	Module-level singleton with a <code>threading.Lock</code> . Nine fixed

	categories match the nine stages probed across both pipelines. <code>set_json_path(path)</code> / <code>initialize(path)</code> called once per run from <code>logger_config.py</code> immediately after the <code>.debug.log</code> file is created — the JSON is written to disk at that point with empty lists, so it exists even if the run crashes before any stage completes. <code>record(category, duration)</code> appends and flushes synchronously on every call — no batching. Wall-clock scope for each probe covers the entire node execution including retries, exponential-backoff delays, LLM output-fix attempts, and failed attempts, so the recorded duration equals what a caller waiting on that stage would observe. For <code>app_workflow/services/logger_config.py</code>, <code>importlib.import_module("app_workflow.services.timing_tracker")</code> is used rather than a relative import to guarantee the caller obtains the canonical <code>sys.modules</code> singleton that node files also reference — see BUG-052.
Rationale	Synchronous flush-on-every-record is deliberately chosen over batching: a run that crashes mid-way still produces a partially-populated JSON with all completed stages visible, which is more useful for diagnosing long-running failures than a file that only appears when the run reaches a teardown hook. The module-level singleton (rather than a class instance passed around) avoids threading issues from multiple imports and ensures that all instrumented call sites in separate files share the same accumulated state without an explicit dependency injection step. <code>time.perf_counter()</code> is preferred over <code>time.time()</code> because it is monotonic and unaffected by system clock adjustments — important when measuring sub-second operations like embedding and retrieval.
Impact	Two new files: <code>app/timing_tracker.py</code>, <code>app_workflow/services/timing_tracker.py</code>. Both <code>logger_config.py</code> files modified to initialise the tracker after log-file creation. All pipeline stage modules in both packages instrumented with <code>time.perf_counter()</code> probes. Every run now produces a companion <code>.debug.json</code> alongside its <code>.debug.log</code> in <code>run_logs/</code>. <code>ENABLE_AUTO_DISTILLATION</code> set to <code>False</code> in both <code>config.py</code> files to prevent self-learning overhead from distorting timing measurements during benchmark runs. <code>run_app_queries.py</code> and <code>run_workflow_queries.py</code> added at the project root to automate multi-query benchmark sessions.

ADR-043 · FIFO Serialization Gate in app/llm_caller.py to Prevent Token-Quota Starvation Under Concurrent Load

Field	Detail
Decision	Replace the random-jitter exponential backoff in <code>app/llm_caller.py</code> with a thread-safe FIFO serialization gate: a <code>queue.Queue</code> of per-thread <code>threading.Event</code> objects that ensures only one thread calls Groq at a time, in strict arrival order. A thread that receives a 429 retains the gate during its token-reset-window sleep rather than re-enqueuing.
Date	2026-06-24
Context	BUG-053: submitting two simultaneous <code>POST /query</code> requests to <code>app/api.py</code> causes one to silently fail — <code>llm_invoke()</code> returns an error after exhausting all <code>LLM_RATE_LIMIT_MAX_ATTEMPTS</code> retries, every one of which was a 429. <code>asyncio.to_thread()</code> dispatches both <code>run_agent()</code> calls into the thread pool concurrently, so both threads reach <code>llm_invoke()</code> at the same time. The first thread's LLM calls consume the Groq token-per-minute quota; the second thread hits 429 on every attempt and exhausts the retry budget. The prior <code>async with _query_lock:</code> in the <code>/query</code> handler appeared to serialize at the HTTP layer but was released before <code>to_thread()</code> dispatched the blocking work into the thread pool — <code>asyncio.Lock</code> is not a cross-thread mutex and provided no actual serialization between pipeline executions. The failure was silent: no exception propagated to the HTTP boundary, so the caller received an erroneous answer with no error signal.
Options Considered	<code>threading.Semaphore(1)</code> (rejected) — limits concurrency to one caller at entry, but a 429-holding thread must release the semaphore before sleeping its retry delay; any waiting thread immediately acquires it and can burn more tokens before the original 429-holder re-enters. No ordering priority: the 429-holder competes equally with new arrivals and can cycle indefinitely through 429s while newer threads starve it. · <code>asyncio.Lock</code> at the <code>/query</code> handler level (rejected, already present but ineffective) — <code>async with _query_lock:</code> covers the async handler body; <code>asyncio.to_thread()</code> hands the blocking work to a thread pool thread immediately, so the lock is released before the pipeline executes. Provides no serialization at the <code>llm_invoke()</code> level where contention actually occurs. · <code>queue.Queue</code> FIFO gate inside <code>llm_invoke()</code> (chosen) — each calling thread enqueues a <code>threading.Event</code> and blocks until the

	front-of-queue thread sets it. A thread that gets a 429 holds the gate (does not enqueue again), sleeps the full reset window from Groq headers, and retries. Gate released only on successful call completion. Strict FIFO ensures no starvation and guarantees the reset-window sleep is uninterrupted by any competing thread.
Chosen Solution	Three module-level globals: <code>_llm_queue</code>: <code>queue.Queue[threading.Event]</code>, <code>_llm_gate_lock</code>: <code>threading.Lock</code>, <code>_llm_active</code>: <code>bool</code>. <code>_gate_acquire(caller_tag)</code> takes the gate immediately if idle or enqueues a <code>threading.Event</code> and blocks on <code>.wait()</code>. <code>_gate_release_to_next()</code> pops the oldest event and calls <code>.set()</code> (waking the next thread) or marks the gate free. <code>_llm_invoke()</code> calls <code>_gate_acquire()</code> once before the retry loop; 429s hold the gate and use <code>_wait_for_token_window()</code> (reads <code>_token_reset_until</code>, a shared monotonic deadline written by any 429-receiver) to sleep the remaining reset window before retrying; success calls <code>_apply_cooldown()</code> then <code>_gate_release_to_next()</code>. <code>_apply_cooldown()</code> sleeps <code>_cooldown_floor</code> seconds (derived from <code>x-ratelimit-remaining-tokens</code> headers) when the queue is non-empty, doubling on 429-recovery, halving after 3 clean successes (<code>_clean_success_streak</code>), resetting to 0 on empty queue; clamped to <code>[MIN_COOLDOWN_TIME, MAX_COOLDOWN_TIME]</code>. <code>_parse_groq_duration()</code> parses Go-style duration strings ("1m30s", "7.66s", "500ms"). <code>_groq_wait_seconds()</code> reads headers in priority: <code>x-ratelimit-reset-tokens</code> → <code>x-ratelimit-reset-requests</code> → <code>retry-after</code>. <code>LLM_RATE_LIMIT_BACKOFF_JITTER_SECONDS</code> removed from <code>config.py</code>. <code>async</code> with <code>_query_lock</code>: block removed from <code>app/api.py</code>'s <code>/query</code> handler. <code>MIN_COOLDOWN_TIME = 0.0</code>, <code>MAX_COOLDOWN_TIME = 30.0</code> added to <code>app/config.py</code>.
Rationale	A FIFO queue is the only primitive that combines strict ordering (a 429-holder is not disadvantaged relative to new arrivals), gate retention on failure (the reset-window wait is paid once and uninterrupted), and correct cross-thread semantics (<code>threading.Event</code> + <code>threading.Lock</code> are thread-safe; <code>asyncio.Lock</code> is not). The adaptive cooldown layer adds a secondary defence: even well-spaced calls can briefly exceed the per-minute token limit due to token-count variability per call; a floor derived from <code>x-ratelimit-remaining-tokens</code> headers absorbs this residual without requiring a full 429-cycle wait.
Impact	<code>app/llm_caller.py</code> — FIFO gate replaces jitter backoff; new module-level symbols: <code>_llm_queue</code>, <code>_llm_gate_lock</code>, <code>_llm_active</code>, <code>_token_reset_until</code>, <code>_cooldown_floor</code>,

	<code>clean_success_streak</code>, <code>gate_acquire</code>, <code>gate_release_to_next</code>, <code>wait_for_token_window</code>, <code>apply_cooldown</code>, <code>parse_groq_duration</code>, <code>groq_wait_seconds</code>. <code>app/api.py</code> — <code>async</code> with <code>query_lock</code>: removed; <code>request_id</code> (uuid4) generated per request and passed to <code>run_agent()</code> and <code>FeedbackStore.log()</code>. <code>app/config.py</code> — <code>MIN_COOLDOWN_TIME</code>, <code>MAX_COOLDOWN_TIME</code> added; <code>LLM_RATE_LIMIT_BACKOFF_JITTER_SECONDS</code> removed. <code>app/logger_config.py</code> — <code>ContextVar[str]</code>, <code>set_request_id()</code>, <code>get_request_id()</code>, <code>RequestIdFilter</code> added. <code>app/agent_query.py</code> — <code>run_agent()</code> gains <code>request_id: str = ""</code> parameter. <code>app/feedback_store.py</code> — <code>log()</code> and <code>save_thumbsdown()</code> accept and persist <code>request_id</code>. Branch <code>setup_async_lock</code> merged into master on 2026-06-25; <code>app_workflow/services/llm_caller.py</code> ported in the same session — see ADR-045.
--	--

ADR-044 · Port FIFO Serialization Gate + Adaptive Cooldown to `app_workflow/services/llm_caller.py`

Field	Detail
Decision	Apply the identical FIFO serialization gate and adaptive cooldown system from <code>app/llm_caller.py</code> (ADR-044) to <code>app_workflow/services/llm_caller.py</code> , bringing both pipelines to full parity on concurrent-request rate-limit handling.
Date	2026-06-25
Context	ADR-044 introduced the FIFO gate in <code>app/llm_caller.py</code> to fix BUG-053 — concurrent threads consuming Groq token quota and causing silent retry exhaustion. <code>app_workflow/services/llm_caller.py</code> used exponential backoff with jitter and no gate, making it susceptible to the same failure mode under concurrent <code>/query</code> requests to <code>app_workflow/api.py</code> (port 8001). After BUG-053's fix was validated in <code>app/</code> and the branch merged, parity demanded the same mechanism be applied to the LangGraph pipeline.
Options Considered	Port FIFO gate from <code>app/</code> verbatim (chosen) — the two <code>llm_caller.py</code> files already share the same interface (<code>llm_invoke</code>, <code>LLMResult</code>, <code>LLMErrorKind</code>); the gate is a drop-in addition at the same points in the call graph. · Leave <code>app_workflow/</code> with its own backoff strategy (rejected) — this is the same root failure

	mode that BUG-053 fixed in <code>app/</code>; diverging the two implementations would permanently require two maintenance tracks for the same mechanism. · Centralise <code>llm_caller.py</code> into a shared module (deferred) — a valid long-term cleanup, but requires restructuring both packages' import chains; out of scope for this session.
Chosen Solution	Ported all FIFO gate components to <code>app_workflow/services/llm_caller.py</code>: <code>gate_acquire</code>, <code>gate_release_to_next</code>, <code>apply_cooldown</code>, <code>groq_wait_seconds</code>, <code>parse_groq_duration</code>, <code>token_reset_until</code>, <code>cooldown_floor</code>, <code>clean_success_streak</code>, <code>llm_queue</code>, <code>llm_gate_lock</code>, <code>llm_active</code>. LLM_RATE_LIMIT BACKOFF JITTER SECONDS removed from <code>app_workflow/config.py</code>; <code>MIN_COOLDOWN_TIME / MAX_COOLDOWN_TIME</code> added. <code>max_retries=0</code> set on all ChatGroq instances in both packages. Handler-level lock removed from <code>app_workflow/api.py</code>; per-request <code>uuid4 request_id</code> generated and propagated to graph state and <code>FeedbackStore.log()</code>. <code>ContextVar[str] request_id_var + RequestIdFilter</code> ported to <code>app_workflow/logger config.py</code>. <code>request_id: NotRequired[str]</code> added to <code>GraphState</code> in <code>app_workflow/state.py</code>. <code>request_id</code> persisted by <code>app_workflow/feedback_store.py</code>.
Rationale	Both HTTP endpoints (<code>app/api.py</code> port 8000, <code>app_workflow/api.py</code> port 8001) accept concurrent requests and dispatch them via <code>asyncio.to_thread()</code>. Any concurrent-load fix applied to one must apply to the other to maintain parity. Running both pipelines to compare performance requires both to behave identically under load.
Impact	Both pipeline packages now use the FIFO gate as their sole concurrency mechanism for Groq calls; <code>asyncio.Lock</code> handler-level serialization is removed from both <code>api.py</code> files. <code>app_workflow/ state</code> carries a <code>request_id: NotRequired[str]</code> field; <code>FeedbackStore.log()</code> persists it; every log record in the LangGraph pipeline includes <code>[request_id]</code> via <code>RequestIdFilter</code>. Both <code>llm_caller.py</code> files are fully in parity. BUG-053 closed.

ADR-045 · MongoDB Replaces JSON/JSONL Flat-File Feedback Stores in Both Pipelines

Field	Detail
Decision	Replace the three flat-file feedback stores (<code>interactions.jsonl</code> , <code>user_thumbsdowns.json</code> , <code>failed_variants.json</code>) with three MongoDB collections (<code>feedback_interactions</code> , <code>user_thumbsdowns</code> , <code>failed_variants</code>) in both <code>app/</code> and <code>app_workflow/</code> . Introduce a new <code>db.py</code> module in each package as the sole MongoDB client and index initialization point.
Date	2026-06-27
Context	The flat-file stores accumulated three correctness problems as the system scaled: (1) <code>interactions.jsonl</code> is append-only; querying "the last interaction" or "the interaction with this request_id" requires reading the full file — $O(n)$ with no indexing. (2) <code>failed_variants.json</code> and <code>user_thumbsdowns.json</code> are read in full at startup and held in a process-level dict (<code>_ctx["failed_variants_store"]</code>), making the in-process dict the canonical state. Under multi-worker deployments or process restarts between a <code>/query</code> and <code>/feedback/bad</code> call, this dict is gone. (3) The <code>/feedback/bad</code> endpoint's <code>mark_last_bad()</code> relied on the in-memory last-result sidecar (<code>last_result / last_variants_with_chunks</code>), which is only valid if <code>/feedback/bad</code> is called in the same process, immediately after the <code>/query</code> that produced the result. The request-id tracing added on 2026-06-24/25 (ADR-044/045) provided a natural join key that could make <code>/feedback/bad</code> process-agnostic if the interaction were stored in a queryable, persistent store.
Options Considered	SQLite — zero new dependency (stdlib <code>sqlite3</code>), queryable by <code>request_id</code>, persistent across processes. But SQLite's write concurrency is file-lock based; concurrent uvicorn workers would contend on write. · Keep JSON/JSONL + improve — add an in-process index over the JSONL for $O(1)$ <code>request_id</code> lookups; use file locking for concurrent writes. Viable but invents a mini-database; maintenance burden grows with query complexity. · MongoDB (chosen) — native indexing (unique, ascending, <code>\$addToSet</code>), multi-document transactions via sessions for atomic two-step thumbsdown writes, client-side connection pooling, and no file-locking contention under concurrent workers. <code>pymongo</code> was already a natural choice given the document-oriented feedback schema.

Chosen Solution	New <code>app/db.py</code> and <code>app_workflow/services/db.py</code> modules with lazy singleton <code>MongoClient</code>, <code>get_db()</code> (reads <code>MONGODB_URI</code> and <code>MONGODB_DB_NAME</code> from env), <code>ensure_indexes()</code> (idempotent, guarded by <code>_indexes_ensured</code> flag), and three public collection accessors. <code>FeedbackStore</code> rewritten to use <code>insert_one</code> / <code>find_one</code> / <code>update_one</code> on MongoDB collections. <code>api.py</code> updated to call <code>failed_variants_col()</code> directly instead of maintaining an in-process dict.
Rationale	MongoDB's document model maps directly onto the existing JSON feedback schema with zero schema migration: each JSONL line becomes one document. Unique indexes on <code>request_id</code> and <code>normalized_query</code> provide $O(1)$ lookups by key. <code>\$addToSet</code> upsert for <code>failed_variants</code> prevents duplicate variant strings without requiring a read-modify-write cycle. Multi-document sessions enable the atomic thumbdown write path (ADR-048). The <code>MONGODB_URI</code> / <code>MONGODB_DB_NAME</code> env vars allow both pipelines to share the same database instance in the current single-machine setup or split onto separate instances in a future multi-machine deployment.
Impact	<code>interactions.jsonl</code>, <code>user_thumbdowns.json</code>, and <code>failed_variants.json</code> are no longer written. Two new files added (<code>app/db.py</code>, <code>app_workflow/services/db.py</code>). <code>FeedbackStore</code> in both packages fully rewritten. <code>app/api.py</code> loses the <code>failed_variants_store</code> dict and JSON helpers; gains direct <code>failed_variants_col()</code> calls. MongoDB must be running (replica-set mode for transactions — see ADR-048) before either API server starts. <code>MONGODB_URI</code> defaults to <code>mongodb://localhost:27017</code>; <code>MONGODB_DB_NAME</code> defaults to <code>rag_db</code>.

ADR-046 · Request-ID-Based /feedback/bad Replaces In-Memory Last-Result Lookup

Field	Detail
Decision	Change <code>app/api.py</code> 's <code>POST /feedback/bad</code> endpoint to accept <code>request_id: str</code> as a required field in the request body, and look up the interaction in MongoDB by that ID via <code>feedback_store.mark_bad(request_id, feedback)</code> , rather than calling <code>feedback_store.mark_last_bad()</code> which relied on the in-memory last-result sidecar.
Date	2026-06-27
Context	The original <code>/feedback/bad</code> endpoint called <code>mark_last_bad()</code> , which fetched variants from the process-level <code>last_result</code> dict (or <code>last_variants_with_chunks</code> sidecar in <code>app_workflow/</code>) populated by the most recent <code>/query</code> call. This had two correctness constraints: (1) the call had to arrive in the same process that handled the <code>/query</code> (no multi-worker safety); (2) the call had to arrive before the next <code>/query</code> overwrote <code>last_result</code> . These constraints were never formally documented in <code>API_ENDPOINTS.txt</code> , making them invisible to API callers. The addition of <code>request_id</code> tracing (ADR-044/045) and the MongoDB migration (ADR-046) together removed both constraints: interactions are now persisted by <code>request_id</code> immediately at <code>/query</code> time, with their variants embedded, and are queryable in any process for as long as the document exists.
Options Considered	Keep <code>mark_last_bad()</code> as-is — no API change, but retains the hidden process-affinity and ordering constraints indefinitely. · Accept <code>request_id</code> as optional, fall back to <code>mark_last_bad()</code> if absent — backward compatible but preserves the old failure mode for callers that omit the field. · Require <code>request_id</code>, use MongoDB lookup (chosen) — breaks the process-affinity and ordering constraints cleanly; the <code>request_id</code> has been included in every <code>/query</code> response since ADR-044, so callers already have it.
Chosen Solution	<code>FeedbackRequest</code> Pydantic model updated to include <code>request_id: str</code> as a required field. <code>POST /feedback/bad</code> calls <code>feedback_store.mark_bad(request_id, feedback)</code> . <code>mark_bad</code> does <code>interactions_col().find_one({"request_id": request_id})</code> , then opens a MongoDB session transaction to mark the interaction <code>USER_THUMBSDOWN</code> and (if feedback is long enough) write a thumbsdown record. <code>API_ENDPOINTS.txt</code> updated to document <code>request_id</code> as required and to remove the "must call

	immediately after <code>/query</code> " constraint.
Rationale	Making <code>request_id</code> required is preferable to optional because an optional field leaves the old failure mode in place for callers that forget to send it. Every <code>/query</code> response already includes <code>request_id</code> (since ADR-044), so requiring it is not a burden on callers that already consume the field. The MongoDB lookup makes the endpoint safe for any number of concurrent workers and for calls arriving seconds, minutes, or hours after the original <code>/query</code> .
Impact	<code>FeedbackRequest</code> model in <code>app/api.py</code> gains <code>request_id: str</code> . <code>mark_last_bad()</code> is no longer called by <code>/feedback/bad</code> ; it remains in <code>FeedbackStore</code> for internal use by CLI-mode callers (e.g. <code>app_workflow/s cmd_bad</code> node which still uses the last-result sidecar). <code>API_ENDPOINTS.txt</code> updated. Callers that were not passing <code>request_id</code> will receive an HTTP 422 validation error until they update.

ADR-047 · MongoDB Session Transactions for Atomic Two-Step Thumbdown Write

Field	Detail
Decision	Wrap the two-step thumbdown write (mark interaction <code>USER_THUMBSDOWN</code> + insert thumbdown record) in a single MongoDB session transaction so both writes commit or both roll back, eliminating the partial-write failure mode where an interaction is marked bad but no thumbdown record is ever stored.
Date	2026-06-27
Context	The thumbdown write path has always been two distinct writes: first, update the interaction's <code>quality</code> field to <code>USER_THUMBSDOWN</code> ; second (conditional on <code>feedback length ≥ MIN_FEEDBACK_LEN</code>), insert a new document into <code>user_thumbdowns</code> . Under the old JSONL/JSON file stores, these were two separate file mutations with no rollback capability — if the process crashed after the first write, the interaction would be marked bad but no thumbdown record would ever appear. Under MongoDB without transactions, the same partial-write risk persists: if <code>update_one</code> succeeds but the process dies before <code>insert_one</code> completes, the thumbdown is lost while the interaction quality shows <code>USER_THUMBSDOWN</code> . MongoDB multi-document transactions, available in replica-set mode, eliminate this risk by making both writes atomic.
Options Considered	Accept partial writes — the original behavior; the interaction is marked bad even if the thumbdown record is lost, which is the safer of the two partial states (the user's intent is recorded), but downstream thumbdown-injection logic never fires for those queries. · Idempotent retry loop — detect partial state on next call and repair, but this requires a read-before-write and a custom reconciliation policy. · MongoDB session transaction (chosen) — built-in atomicity with no application-level reconciliation needed; available in replica-set mode which is already required for the project.
Chosen Solution	Both <code>mark_last_bad()</code> and <code>mark_bad(request_id)</code> in <code>FeedbackStore</code> open a session with <code>get_client().start_session()</code> and call <code>session.start_transaction()</code> . The <code>update_one</code> and the <code>append_thumbdown insert_one</code> both receive <code>session=session</code> . <code>append_thumbdown</code> gains an optional <code>session=None</code> parameter so it can participate in the caller's transaction when provided, or

	run without one when called standalone. The same pattern is applied in both <code>app/feedback_store.py</code> and <code>app_workflow/services/feedback_store.py</code> .
Rationale	MongoDB transactions are the simplest correct solution: one <code>with</code> block, no custom reconciliation, no retry policy. The replica-set requirement is already imposed by the project (a standalone <code>mongod</code> does not support multi-document transactions), so this decision adds no new infrastructure constraint. Wrapping <code>append_thumbsdown</code> with an optional <code>session=</code> parameter rather than always opening a new session inside it is the correct pattern for functions that may be called from either a transactional or non-transactional context.
Impact	<code>mark_last_bad()</code> and <code>mark_bad()</code> in both <code>FeedbackStore</code> classes now require a replica-set MongoDB instance; they will raise <code>InvalidOperation</code> if called against a standalone <code>mongod</code> . <code>append_thumbsdown</code> gains <code>session=None</code> parameter in both packages. MongoDB must be started with <code>replSetName: rs0</code> in <code>mongod.cfg</code> and <code>rs.initiate()</code> run once before either API server uses the thumbsdown path.

ADR-048 · Adaptive Branch Budget Heuristic for Query Variant Count in app_workflow/

Field	Detail
Decision	Introduce <code>adaptive_branch_budget(query)</code> — a zero-LLM heuristic that returns 1, 2, or 3 based on query word count and conjunction keyword detection — and inject the budget into the variant-generation system prompt so the LLM cannot exceed it.
Date	2026-06-29
Context	<code>generate_query_variants</code> previously instructed the LLM to generate "2 to 3" variants regardless of query complexity, then passed all variants through retrieval. Short, unambiguous queries (e.g. "what is ABA?", 4 words) were generating 3 variants that were semantically near-identical, wasting retrieval budget on redundant sub-queries. Longer conjunction queries ("compare ABA and DIR approaches", 7 words with "compare") genuinely benefit from 3 variants covering each clause and contrast angle. A zero-LLM heuristic was preferred over a learnable model to avoid adding latency, cost, or a second LLM call before retrieval has even begun.
Options Considered	Always use 3 variants (prior behavior) — maximum coverage but wastes budget on short/unambiguous queries and saturates post-retrieval dedup with near-identical chunks. · Always use 1 variant (no expansion) — eliminates redundancy but loses the coverage benefit of multi-angle sub-queries that is the entire point of variant generation. · LLM-decided budget — add a first-pass LLM call to estimate needed complexity; adds latency and cost for every query before any retrieval occurs. · Rule-based heuristic (chosen) — word count is a reliable proxy for query specificity; conjunction keywords reliably signal multi-clause queries that benefit from parallel sub-queries. Zero LLM cost, sub-millisecond execution.
Chosen Solution	<code>adaptive_branch_budget(query)</code> : strip and split on whitespace; if <code>len(words) <= BRANCH_BUDGET_SHORT_WORD_COUNT</code> (4) return 1; if any word is in <code>BRANCH_BUDGET_CONJUNCTION_KEYWORDS</code> return 3; if <code>len(words) <= BRANCH_BUDGET_MEDIUM_WORD_COUNT</code> (8) return 2; else return 3. The budget is passed to <code>_build_system_prompt(max_variants)</code> which appends a

	RETRIEVAL BUDGET: <n> line to the system prompt so the LLM is explicitly instructed not to exceed the computed limit. <code>generate_query_variants()</code> calls <code>adaptive_branch_budget()</code> first, then <code>pre_retrieval_filter()</code> (ADR-050), before returning variants to the graph. Four new constants in <code>app_workflow/config.py</code>: <code>BRANCH_BUDGET_SHORT_WORD_COUNT</code>, <code>BRANCH_BUDGET_MEDIUM_WORD_COUNT</code>, <code>BRANCH_BUDGET_CONJUNCTION_KEYWORDS</code>, and the existing <code>PRE_RETRIEVAL_SIM_THRESHOLD</code> (added in the same session).
Rationale	Word count is the cheapest available signal for query complexity. Conjunction keywords ("and", "or", "both", "versus", "vs", "compare", "difference", "how", "why", "explain") are reliably associated with multi-clause or comparative intent. The override to 3 on conjunction detection is conservative: it fires only when the query explicitly signals multi-angle intent, not on every long query. The system prompt injection closes the loop: the LLM is notified of its budget and is less likely to generate more variants than will survive pre-retrieval filtering.
Impact	<code>app_workflow/nodes/query_variants.py</code> — <code>adaptive_branch_budget()</code> added; <code>_build_system_prompt()</code> gains <code>max_variants: int</code> parameter. <code>generate_query_variants()</code> calls <code>adaptive_branch_budget()</code> before LLM and passes the result to <code>_build_system_prompt()</code>. Four new constants in <code>app_workflow/config.py</code>. <code>app/</code> is unaffected.

ADR-049 · Pre-Retrieval Cosine Similarity Filter for Query Variant Deduplication

Field	Detail
Decision	Introduce <code>pre_retrieval_filter(variants, query_embedding)</code> in <code>app_workflow/nodes/query_variants.py</code> that (1) removes inter-variant near-duplicates using pairwise cosine similarity at <code>PRE_RETRIEVAL_SIM_THRESHOLD = 0.95</code> , then (2) ranks surviving variants by cosine similarity to the original query embedding and caps the list to the adaptive budget from ADR-049.
Date	2026-06-29
Context	The LLM's variant generation is stochastic and occasionally produces semantically near-identical sub-queries (e.g. "motor delays in autism" and "motor skill delays in ASD" from the same parent query). When both reach retrieval, they compete for the same chunks, the retrieve fan-in accumulates near-duplicate content, and the post-retrieval validation and compression stages must process the redundancy — wasting LLM calls. A pre-retrieval filter eliminates duplicates before any retrieval cost is paid. See Research topic 38 for the Jaccard-vs-cosine analysis that justified using cosine here.
Options Considered	No dedup before retrieval (prior behavior) — all variants reach retrieve; duplicates discovered by <code>post_retrieval_filter_node</code> (ADR-051) or absorbed by the compression pipeline. Cost is paid for every redundant retrieve call and its downstream validation. · Jaccard token-overlap dedup — computationally cheap but fails on paraphrase pairs (different vocabulary, same meaning) which are the dominant failure mode. Rejected on evidence from Research topic 38. · Cosine similarity on embeddings (chosen) — the query embedding is already computed by <code>EmbeddingManager</code> at the start of <code>generate_query_variants</code> . Embedding each short variant adds <5 ms on CUDA. Cosine captures semantic equivalence regardless of surface wording.
Chosen Solution	<code>pre_retrieval_filter(variants, query_embedding)</code> : embed all variants in one batch via <code>services.embedding_manager.generate_embedding()</code> ; compute pairwise cosine matrix; iterate in order — mark a variant as duplicate if its cosine to any prior survivor is $\geq$ <code>PRE_RETRIEVAL_SIM_THRESHOLD</code> ; rank remaining

	survivors by cosine to the original query embedding (descending); return the top-budget entries. The original query embedding is passed in from generate_query_variants() which already holds it. PRE_RETRIEVAL_SIM_THRESHOLD = 0.95 added to app_workflow/config.py.
Rationale	0.95 cosine is a conservative high-similarity threshold: at this level, pairs are almost certainly paraphrases with no distinct retrieval angle. The rank-by-query-similarity step ensures the retained variants are the ones most semantically aligned to the user's actual intent rather than arbitrary survivors. Two-phase design (dedup then rank-and-cap) is important: dedup first ensures we aren't accidentally dropping a high-quality unique variant by ranking before removing duplicates.
Impact	app_workflow/nodes/query_variants.py — pre_retrieval_filter() added; generate_query_variants() calls it after LLM generation and before returning. app_workflow/config.py — PRE_RETRIEVAL_SIM_THRESHOLD = 0.95 added. No other files modified. app/ unaffected.

ADR-050 · Post-Retrieval Deduplication by Retrieved-Chunk Frozenset Fingerprint

Field	Detail
Decision	Introduce <code>post_retrieval_filter_node</code> in a new <code>app_workflow/nodes/post_retrieve.py</code> module as a barrier node inserted in the graph between the retrieve fan-in and the two parallel validation nodes. The node identifies variants whose retrieved chunk sets are byte-for-byte identical (fingerprinted as <code>frozenset((content, source))</code> tuples) and strips the duplicate variant's chunk contributions from the accumulated retrieval lists. Results are written to new <code>GraphState</code> fields <code>post_filtered_document_chunks</code> and <code>post_filtered_learned_qa_chunks</code> .
Date	2026-06-29
Context	Even after pre-retrieval cosine filtering (ADR-050), two variants with cosine similarity below the 0.95 threshold may retrieve the same top-K chunks — especially for narrow domain corpora where the vector store has limited coverage. In those cases, the <code>retrieve</code> fan-in accumulates duplicate chunk copies, and the validation and compression stages must process and eventually discard the redundancy at higher cost. A post-retrieval dedup pass that checks actual chunk content catches these cases at the lowest possible cost (no LLM call, pure set operations).
Options Considered	No post-retrieval dedup — rely on NAC (cosine-based chunk-level dedup inside compression) to handle duplicate chunks. Cost: NAC is a compression node; duplicates arriving at NAC must first survive retrieval validation (one LLM call per track) and dedup-merge (another LLM call pair), paying LLM costs for content that will eventually be dropped. · Content-hash dedup inside <code>validate_retrieval.py</code> — simple deduplicate-by-content logic inline in each validator node. Mixes chunk-identity concerns with validation concerns; harder to unit-test and maintain separately. · Dedicated barrier node (chosen) — clean separation of concerns; runs before any LLM call in the post-retrieve phase; writes to clearly named <code>post_filtered_*</code> state fields so downstream nodes can distinguish filtered from unfiltered content explicitly.
Chosen Solution	<code>post_retrieval_filter_node(state)</code> : build per-variant chunk fingerprints as <code>frozenset((c["content"], c["source"]))</code> for <code>c</code> in <code>chunks_for_variant</code> for both document and learned-QA tracks;

	iterate variants in order — mark as duplicate if its frozenset exactly matches a prior survivor's frozenset; collect chunks from surviving variants only; write to <code>post_filtered_document_chunks</code> and <code>post_filtered_learned_qa_chunks</code>. Graph rewired: <code>retrieve</code> → <code>post_retrieval_filter_node</code> → {<code>validate_document_retrieval</code>, <code>validate_learned_qa_retrieval</code>}. Validation nodes read <code>post_filtered_*</code> first, falling back to raw <code>retrieved_*</code> accumulators when the filter node has not run. Two new <code>NotRequired[list[dict]]</code> fields added to <code>GraphState</code> in <code>state.py</code>. <code>post_retrieval_filter_node</code> exported from <code>nodes/ __init__.py</code>.
Rationale	Frozenset equality on <code>(content, source)</code> pairs is a deterministic, zero-cost duplicate check. It only fires when two variants truly retrieved the same set of chunks — a strong signal that the retrieval trajectories are equivalent, and one is fully redundant. The barrier-node placement is important: it runs once after all parallel <code>Send</code> branches have completed their fan-in, before any further LLM processing begins. The <code>NotRequired</code> annotation on the new state fields ensures backward compatibility: code paths that bypass the node (e.g. tests or future graph variants) still receive valid (empty-<code>None</code>) field state.
Impact	New file: <code>app_workflow/nodes/post_retrieve.py</code>. <code>app_workflow/state.py</code> — two new <code>NotRequired</code> fields. <code>app_workflow/graph.py</code> — new node + rewired edges. <code>app_workflow/nodes/validate_retrieval.py</code> — reads <code>post_filtered_*</code> first. <code>app_workflow/nodes/ __init__.py</code> — new export. <code>app/</code> unaffected.

ADR-051 · Separate Per-Collection Similarity Thresholds for Document and Learned-QA Retrieval

Field	Detail
Decision	Replace the single <code>score_threshold</code> parameter in <code>app_workflow/services/retriever.py</code> 's <code>retrieve_separate()</code> with two independent parameters: <code>doc_score_threshold</code> (default <code>DOCUMENTS_MIN_SIMILARITY</code>) and <code>learned_score_threshold</code> (default <code>LEARNED_QA_MIN_SIMILARITY</code>), each configurable independently in <code>app_workflow/config.py</code> .
Date	2026-06-29
Context	<code>retrieve_separate()</code> previously applied one <code>score_threshold</code> value to both the <code>documents</code> and <code>learned_qa</code> collections. Log analysis (Research topic 39) revealed that the two collections have different score-distribution profiles: <code>documents</code> chunks (source PDFs/textfiles, 1,181 chunks) span a wider cosine range against typical queries; <code>learned_qa</code> chunks (distilled Q&A pairs semantically tuned to prior queries) cluster at higher cosine scores. Using a single threshold either leaves noisy document chunks in (threshold too low) or discards relevant learned-QA entries (threshold too high). Separate thresholds allow each collection to be tuned to its own score distribution independently.
Options Considered	Single shared threshold (prior behavior) — simpler config, but conflates two distinct retrieval populations with different score characteristics. · Per-query dynamic threshold (adaptive, e.g. derived from the score histogram) — more accurate but requires reading the score distribution after retrieval, adding latency and complexity. · Fixed separate thresholds (chosen) — zero runtime cost, directly config-driven, tunable per collection from log evidence without changing any retrieval logic.
Chosen Solution	<code>retrieve_separate(query, top_k, top_l, doc_score_threshold=DOCUMENTS_MIN_SIMILARITY, learned_score_threshold=LEARNED_QA_MIN_SIMILARITY)</code> — each threshold is applied independently to its collection's ranked list before returning. <code>DOCUMENTS_MIN_SIMILARITY = 0.5</code> and <code>LEARNED_QA_MIN_SIMILARITY = 0.5</code> added to <code>app_workflow/config.py</code> as initial conservative values (log-based tuned targets 0.55 and 0.58 documented in Research topic 39 for

	a future update). Call sites in <code>app_workflow/nodes/retrieve.py</code> and <code>app_workflow/services/tools.py</code> updated to pass the named keyword arguments explicitly.
Rationale	The two collections have structurally different embedding populations. Documents are indexed as raw text chunks; their cosine scores against a short user query span a wide range and require a lower floor to avoid dropping relevant but specific chunks. Learned-QA entries are synthetic Q&A pairs whose embeddings are inherently closer to conversational queries; their score floor can be set higher without recall loss. Separate constants make the tuning explicit, visible, and auditable in <code>config.py</code> rather than hardcoded inside retrieval logic.
Impact	<code>app_workflow/services/retriever.py</code> — <code>retrieve_separate()</code> signature changed (backward-incompatible: callers that passed <code>score_threshold</code> as a positional arg must switch to keyword args). <code>app_workflow/nodes/retrieve.py</code> and <code>app_workflow/services/tools.py</code> — call sites updated. <code>app_workflow/config.py</code> — <code>DOCUMENTS_MIN_SIMILARITY</code> and <code>LEARNED_QA_MIN_SIMILARITY</code> added; the old <code>MIN_SIMILARITY</code> constant is no longer used by <code>retrieve_separate()</code> (it may still be referenced by other callers; not removed). <code>app/</code> unaffected.

ADR-052 · Evidence-Based Tuning of RETRIEVAL_TOP_K, RETRIEVAL_TOP_L, DOCUMENTS_MIN_SIMILARITY, and LEARNED_QA_MIN_SIMILARITY From A/B Log Comparison

Field	Detail
Decision	Update four retrieval config constants in <code>app_workflow/config.py</code> from their conservative initial values to values derived from a controlled five-log A/B comparison: <code>RETRIEVAL_TOP_K</code> 5 → 4, <code>RETRIEVAL_TOP_L</code> 5 → 4, <code>DOCUMENTS_MIN_SIMILARITY</code> 0.50 → 0.53, <code>LEARNED_QA_MIN_SIMILARITY</code> 0.50 → 0.57.
Date	2026-06-30
Context	ADR-052 (2026-06-29) introduced per-collection similarity thresholds and set both to 0.50 as a conservative initial value, with Research topic 39 identifying 0.55 and 0.58 as tentative tuned targets based on score-distribution shape. On 2026-06-30, the new <code>extract_logs.py</code> utility was used to run a controlled A/B comparison across five debug logs: the same two representative queries (one simple in-domain; one complex cross-domain with three query variants) were each executed under three config combinations — <code>K=5/0.50/0.50</code> (previous default), <code>K=4/0.55/0.60</code> (aggressive reduction), and <code>K=4/0.53/0.57</code> (intermediate). LLM call counts, INSUFFICIENT verdict frequencies, rate-limit-pause counts, and answer correctness were compared across the resulting five debug logs. The Research topic 39 tentative recommendation (0.55/0.58) was not a tested combination; the comparison surfaced 0.53/0.57 as empirically optimal over both extremes tested.
Options Considered	Keep 0.50/0.50 (<code>K=5</code> , original default) — simple in-domain query: 148 LLM calls, 19+ rate-limit pauses, 1 INSUFFICIENT retry. Complex cross-domain query: 205 calls, 3 INSUFFICIENT verdicts, 2 full retrieval retries. Lowest per-chunk filter rate; highest total LLM cost by 3–5×. · Use 0.55/0.60 (<code>K=4</code> , aggressive) — simple query: 42 LLM calls, 3 pauses, 0 INSUFFICIENT. Complex 3-variant "ASD" query: 37 calls, 0 INSUFFICIENT — but 2 of 3 query variants retrieved zero chunks because the threshold was too high for multi-domain corpus coverage; the LLM hallucinated the domain meaning of "ASD" rather than admitting insufficient context. Lower cost on simple queries; correctness regression on multi-domain queries. · Use 0.53/0.57 (<code>K=4</code> , intermediate, <code>langgraph_api_20260630_125641.debug.log</code>) — simple query:

	30 LLM calls, 3 pauses, 0 INSUFFICIENT. Complex "ASD" query: 30 calls, 0 INSUFFICIENT, no retrieval starvation, correct domain answer. Best on all three dimensions: cost, verdict quality, accuracy.
Chosen Solution	RETRIEVAL_TOP_K = 4, RETRIEVAL_TOP_L = 4, DOCUMENTS_MIN_SIMILARITY = 0.53, LEARNED_QA_MIN_SIMILARITY = 0.57 in app_workflow/config.py. All four values remain env-overridable.
Rationale	The 0.53/0.57 pair is the only combination tested that simultaneously avoids low-quality chunk inflation (0.50 failure mode) and retrieval starvation on multi-domain queries (0.55/0.60 failure mode). The cross-domain hallucination observed at 0.55/0.60 is a correctness regression — the LLM confidently produced a wrong answer rather than routing to no_context_answer — which is strictly worse than a higher LLM call count. Evidence was derived from production debug logs with real queries, not synthetic test data. The A/B log comparison methodology is reusable and repeatable for future threshold adjustments as the corpus grows.
Impact	Supersedes the tentative 0.55/0.58 targets from Research topic 39. RETRIEVAL_TOP_K and RETRIEVAL_TOP_L each reduced from 5 to 4, cutting the maximum retrieved-chunk count per variant by one. Combined effect on observed production queries: 3–5× reduction in LLM calls vs. the 0.50/0.50 default with no observed accuracy regression across the two tested query types. Config constants remain env-overridable; the A/B comparison methodology (Research topic 40) is the reference for any future threshold revisit.

ADR-053 · LLM Provider Expansion: Hugging Face Inference Providers Router Adopted Over Google Colab Tunnel

Field	Detail
Decision	Reject Google Colab + tunnel (ngrok/localtunnel) as a hosting path for production judge_llm/repair-tier calls; adopt the Hugging Face Inference Providers router (https://router.huggingface.co/v1 , OpenAI-compatible) as a second LLM provider alongside Groq, accessed via langchain_openai.ChatOpenAI
Date	2026-07-01
Context	The sub-8B latency-reduction investigation (Research topic 41, 2026-06-30) needed a second, genuinely free (no subscription/billing) LLM source for the judge and JSON-repair tiers, since Groq has no general-purpose chat model under 8B parameters. A Colab-notebook-plus-tunnel approach (running Qwen/Gemma/Mistral/Phi-3 models on a free Colab GPU, exposed via pyngrok or localtunnel) was proposed as a way to get free access to a wider range of small open models.
Options Considered	Colab + tunnel — genuinely free GPU, wide model choice, but violates Colab's Terms of Service for unattended/server-like usage backing a production pipeline, has no stable endpoint (tunnel URL changes every session restart, session dies after ~90 min idle / ~12h max), and GPU availability is not guaranteed. Confirmed in production use (app_workflow/services/llm_setup.py's LocalJudgeLLM hitting a loca.lt URL): ReadTimeout on slow responses and 502/503 errors when the Colab-side server was overloaded — see Status.md 2026-07-01. · Hugging Face Inference Providers router — persistent, stable URL; OpenAI-compatible /v1/chat/completions; free tier available without a paid subscription; drops into the existing ChatOpenAI-based LLM construction pattern already used for Groq. Limitation: only a curated subset of the HF Hub is exposed through the router (e.g. mistralai/Mistral-7B-Instruct-v0.3 was rejected with model_not_supported). · Hugging Face Spaces — considered as a fallback hosting option (persistent, ToS-compliant), not pursued once the router proved sufficient. · Local Ollama — considered in the earlier options survey (Research topic 41 follow-up), not pursued this session; would eliminate rate-limit engineering entirely but requires dedicated local hardware.

Chosen Solution	Hugging Face Inference Providers router via <code>langchain_openai.ChatOpenAI(base_url=HF_API_BASE, api_key=HF_TOKEN, model=...)</code>, wired into <code>app_workflow/services/llm_setup.py</code> for both <code>judge_llm</code> and the new <code>json_fix_llm</code>. The Colab+tunnel path, though briefly deployed and exercised in production, was abandoned once its failure modes materialized.
Rationale	The router gives the project a second, stable, OpenAI-compatible provider with zero infrastructure to operate or babysit — the same trade-off that made Groq attractive in ADR-004, but for models Groq doesn't host. The Colab approach's operational cost (manual URL updates, ToS risk, non-guaranteed GPU) was confirmed empirically rather than just theoretically: the same timeout/502/503 failure classes anticipated in the pre-adoption discussion were the exact failures observed once the tunnel was actually wired into the pipeline.
Impact	<code>app_workflow/services/llm_setup.py</code>, <code>app_workflow/services/llm_caller.py</code> (OpenAI SDK exception handling — see ADR-055 for the same-day cleanup). New <code>HF_TOKEN/HF_API_BASE</code> env vars. <code>app/</code> untouched. The router's curated-subset limitation (confirmed via the Mistral-7B-Instruct-v0.3 rejection) means any future model swap on this provider must be checked against router availability before being wired in — <code>useful_but_not_valuable_files/discover_available_models.py</code> was written for exactly this check.

ADR-054 · Dedicated json_fix_llm Model Decoupled from Per-Call llm Override

Field	Detail
Decision	Give the JSON-repair tier (<code>_LLM_Json_Repair()</code> , <code>_Verify_And_Correct()</code> in <code>app_workflow/services/fix_llm_output.py</code>) a single dedicated module-level <code>json_fix_llm</code> instance, and remove the <code>llm: Any = None</code> parameter these functions previously accepted (and the public <code>fix_llm_output()</code> wrapper exposed) from their signatures entirely
Date	2026-07-01
Context	While wiring the new HuggingFace-hosted <code>json_fix_llm</code> (see ADR-054) into <code>fix_llm_output.py</code> , inspection showed the existing <code>llm</code> parameter on <code>_LLM_Json_Repair()</code> was overwritten by <code>llm = json_fix_llm</code> on the line immediately after the function signature — meaning every one of the 8 call sites across <code>dc.py</code> , <code>lbc.py</code> , <code>nac.py</code> , <code>self_learner.py</code> , and <code>validators.py</code> that passed <code>llm=...</code> had been passing a value that was silently discarded. The parameter was dead code left over from before the module had its own dedicated repair model.
Options Considered	Keep the <code>llm</code> parameter and actually honor it — would let each call site choose its own repair model, but no call site had a reason to diverge, and it reintroduces the original problem this session was trying to solve (repair calls silently using whatever <code>llm</code> happened to be in scope, rather than a deliberately chosen small dedicated model). · Remove the parameter, always use module-level <code>json_fix_llm</code> (chosen) — matches the fact that every call site already received the same overwritten value in practice; makes the repair tier's model choice a single, auditable point of configuration in <code>llm_setup.py</code> rather than scattered call-site arguments that don't do anything.
Chosen Solution	Removed <code>llm</code> from <code>_LLM_Json_Repair()</code> , <code>_Verify_And_Correct()</code> , and <code>fix_llm_output()</code> 's public signature in <code>app_workflow/services/fix_llm_output.py</code> . Both repair functions now call <code>json_fix_llm</code> directly, retaining their existing ChatGroq auto-instantiation fallback for the case where <code>json_fix_llm</code> itself is <code>None</code> . All 8 call sites had their now-meaningless <code>llm=...</code> argument removed. <code>app/fix_llm_output.py</code> was not touched — its <code>llm</code> parameter is still load-bearing there (no overwrite bug present in that file).

Rationale	A parameter that is unconditionally overwritten on the next line is worse than no parameter at all — it signals to every caller and future maintainer that repair-model selection is configurable per call site, when it has not been since <code>json_fix_llm</code> was introduced. Removing it makes the actual architecture (one dedicated repair model, chosen once in <code>llm_setup.py</code>) visible in the function signatures instead of contradicted by them.
Impact	<code>app_workflow/services/fix_llm_output.py</code> — signature changes to <code>_LLM_Json_Repair()</code> , <code>_Verify_And_Correct()</code> , <code>fix_llm_output()</code> . Call-site cleanup in <code>dc.py</code> , <code>lbc.py</code> , <code>nac.py</code> , <code>self_learner.py</code> , <code>validators.py</code> (4 sites). <code>app/fix_llm_output.py</code> and its call sites are unaffected and remain on the pre-existing pattern.

ADR-055 · Answer-Quality Judge Upgraded to Structured Multi-Verdict Output (GROUNDED / PARTIALLY_FABRICATED / OVERCLAIMED / OFF_TOPIC / UNKNOWN)

Field	Detail
Decision	Replace <code>GROUNDING_PROMPT</code> 's free-text <code>OK</code> / <code>INSUFFICIENT</code> — <code><sentence></code> output with a structured JSON verdict object, evaluated against three rules (exhaustive per-sentence traceability, relevance, and a new completeness/calibration rule) instead of two, and make judge-call failure default to <code>"UNKNOWN"</code> instead of the previous fail-open <code>"OK"</code>
Date	2026-07-02
Context	The prior judge graded only "relevance" and whether "key claims" (not all claims) were traceable to retrieved chunks. This let an answer built from one true grounded claim padded with several fabricated ones pass as <code>OK</code> , because the judge had no mandate to check every sentence and no axis at all for whether the answer's scope/confidence matched how much evidence it was actually given. Judge-call failure (timeout, malformed response) also silently defaulted the verdict to <code>"OK"</code> — a fail-open gate protecting the entire self-learning knowledge base with a single lenient check.
Options	Keep the binary free-text verdict and append more instruction text (cheap, but doesn't close the "key claims" sampling loophole

Considered	— the judge still has no mandate to check every sentence) · reuse the codebase's own compression-safety judge pattern already proven for the identical underlying fabrication risk (<code>DC_JUDGE</code>-style: exhaustive per-sentence check + structured JSON output with a fabricated-claims list) · build a dedicated trained reward model / PRM (rejected — no training data yet, see Research topic 10)
Chosen Solution	Rewrote <code>GROUNDING_PROMPT</code> to require sentence-by-sentence traceability (not just "key claims"), added Rule 3 for completeness/calibration (answer scope and confidence must match evidence volume), and changed the output to JSON: <code>verdict</code> (one of <code>GROUNDED</code>/<code>PARTIALLY_FABRICATED</code>/<code>OVERCLAIMED</code>/<code>OFF_TOPIC</code>/<code>UNKNOWN</code>), <code>unsupported_claims: []</code>, <code>scope_mismatch</code>, <code>overall reason</code>. <code>check_answer_quality.py</code> parses this via a new <code>fix_llm_output("grounding_judge", ...)</code> schema (<code>GroundingJudgeSchema</code>, added to the existing 8-schema registry), validates the verdict against a <code>VALID_VERDICTS</code> set, and falls back to <code>"UNKNOWN"</code> on any judge-call or parse failure. <code>QUALITY_PASS_VERDICT = "GROUNDED"</code> is the single source of truth for pass/fail.
Rationale	Reuses a judge pattern already validated elsewhere in the codebase for the identical underlying risk rather than inventing new judge architecture. Splitting <code>OVERCLAIMED</code> (every claim technically true, but scope/confidence exceeds the evidence) from <code>PARTIALLY_FABRICATED</code> (some claims outright invented) lets future routing treat the two differently — a strip-and-caveat pass versus a full retry — even though today's routing still treats every non-<code>GROUNDED</code> verdict the same way. Failing closed to <code>UNKNOWN</code> removes a fail-open gate that let a transient judge failure silently pass every answer.
Impact	<code>app_workflow/services/prompts.py</code> (<code>GROUNDING_PROMPT</code> rewritten), <code>app_workflow/services/fix_llm_output.py</code> (new <code>GroundingJudgeSchema</code> + <code>"grounding_judge"</code> registry entry, new <code>ENABLE_ANSWER_QUALITY_OUTPUT_FIX</code> config flag), <code>app_workflow/nodes/check_answer_quality.py</code> (rewritten to consume the JSON verdict and log <code>unsupported_claims</code> count), <code>app_workflow/routes.py</code> (<code>route_after_quality_check</code> now branches on <code>verdict == QUALITY_PASS_VERDICT</code>). <code>generate_answer.py</code> and <code>api.py</code> both had a second, independent <code>.startswith("INSUFFICIENT")</code> assumption on the same state field, fixed for consistency in the same pass. Differentiated routing per non-passing verdict (e.g. strip-and-caveat for

	OVERCLAIMED vs. full retry for PARTIALLY FABRICATED) is not yet implemented — today all non-GROUNDED verdicts route the same way.
--	---

ADR-056 · combine_tracks Registered with LangGraph defer=True as a True Fan-In Barrier

Field	Detail
Decision	Register the combine_tracks node with LangGraph's defer=True option instead of relying on default fan-in behavior
Date	2026-07-02
Context	combine_tracks has two predecessors of unequal depth — the document track (NAC → DC → LBC, 3 stages) and the learned-QA track (DC → LBC, 2 stages) — so they land in different LangGraph supersteps. LangGraph's default fan-in only merges predecessors completing in the <i>same</i> superstep, so the node fired once per completing track rather than once total. See Bugs.md BUG-062.
Options Considered	Manually gate the node body with a "both tracks marked complete" flag (works, but pushes barrier logic into node code and has to be reimplemented if a third parallel track is ever added) · restructure the learned-QA track to artificially match the document track's depth with a no-op stage (changes pipeline topology to work around a scheduling detail, not a real requirement) · use LangGraph's built-in defer=True node option, designed for exactly this situation
Chosen Solution	graph.py's add_node("combine_tracks", combine_tracks, defer=True). LangGraph holds the node's execution until every other pending task in the run has finished, then runs it exactly once.
Rationale	defer=True is the framework's own answer to "a node has multiple predecessors of unequal path length" — simpler and more robust than hand-rolled completion-flag logic, and it generalizes automatically if a third track is ever added to the graph.
Impact	app_workflow/graph.py. Confirmed via manual log inspection (and graphify update .) that combine_tracks now fires once per

	request with both <code>document_chunks</code> and <code>learned_qa_chunks</code> populated, resolving the "multiple drafts per request" and "draft built from partial context" symptoms in Bugs.md BUG-062.
--	--

ADR-057 · `generate_answer`: Draft Is Synthesis Input, Not the Final Answer

Field	Detail
Decision	When a draft exists, <code>generate_answer</code> must call the LLM once more with the draft as working material to produce the actual final answer, and use the literal draft only as a fallback if that synthesis call fails
Date	2026-07-02
Context	The intended DRAFT → (synthesis) → FINAL design was already implemented correctly in <code>app/agent_query.py</code> 's <code>generate_final_answer</code> (always calls the LLM once more, passing the draft as working material alongside the compressed context, falling back to the literal draft only on failure), but <code>app_workflow/nodes/generate_answer.py</code> instead returned <code>state["draft"]</code> verbatim whenever present, skipping the synthesis call entirely. See Bugs.md BUG-063.
Options Considered	Keep the short-circuit (cheapest, but defeats the purpose of having a separate DRAFT phase and a quality judge that grades text nobody re-checks afterward) · port <code>app/agent_query.py</code> 's exact two-branch synthesis-prompt pattern (draft-present vs. draft-absent) · introduce a separate "refine" node between <code>generate_draft</code> and <code>generate_answer</code> (adds a graph node for something that fits naturally inside <code>generate_answer</code> 's existing branch)
Chosen Solution	Added <code>GENERATE_ANSWER_FROM_DRAFT_PROMPT</code> (alongside the existing <code>GENERATE_ANSWER_PROMPT</code> for the no-draft path) in <code>prompts.py</code> . <code>generate_answer.py</code> now: draft exists → call the LLM with <code>GENERATE_ANSWER_FROM_DRAFT_PROMPT</code> (draft + full context) and use that result as the answer; synthesis call fails (<code>result.ok</code> is <code>False</code>) → fall back to the literal draft rather than returning empty; no draft → call the LLM with <code>GENERATE_ANSWER_PROMPT</code> directly from flattened context (unchanged).

Rationale	Matches the already-correct design in the sibling <code>app/</code> pipeline exactly, rather than inventing a new pattern. Preserves the fallback (draft-as-final) for the case the design already anticipates — a failed synthesis call — instead of silently returning nothing.
Impact	<code>app_workflow/services/prompts.py</code> (new <code>_GENERATE_ANSWER_FROM_DRAFT_PROMPT</code>), <code>app_workflow/nodes/generate_answer.py</code> (rewired branch logic). Motivated adding <code>_GENERATE_DRAFT_PROMPT</code> (see ADR-059) since a synthesis step is only as honest as the draft it's given.

ADR-058 · Conservative-Grounding Prompt Pattern Applied to All Generation-Stage Prompts (Answer, Draft, Answer-From-Draft)

Field	Detail
Decision	Replace the single-line <code>generate_answer</code> prompt and <code>generate_draft</code> 's inline "Be thorough and complete" prompt with three structured, anti-hallucination prompts (<code>_GENERATE_ANSWER_PROMPT</code> , <code>_GENERATE_ANSWER_FROM_DRAFT_PROMPT</code> , <code>_GENERATE_DRAFT_PROMPT</code>)
Date	2026-07-02
Context	The prior <code>generate_answer</code> prompt — "Answer the following question using only the provided context" — constrains <i>where</i> facts come from but not <i>how much</i> to claim relative to how much evidence exists; observed failure mode was a single thin retrieved chunk producing a confident, fully itemized, largely fabricated answer. <code>generate_draft.py</code> 's prompt was worse in this respect — it explicitly instructed the model to "be thorough and complete," directly encouraging padding.
Options Considered	Append a short "don't hallucinate" caveat to the existing one-line prompts (cheap, but Research topic 19 already found vague negative instructions are far less effective than named anti-patterns plus worked examples for steering model behavior) · apply the structured prompt pattern already proven in this codebase for the equivalent judge-side risk (<code>_REDUNDANCY_JUDGE_PROMPT</code> 's role-sentence → output-contract → worked-GOOD/BAD-examples → named-

	anti-patterns → explicit-output-format structure) to the generation side instead of only the judging side
Chosen Solution	Added <code>_GENERATE_ANSWER_PROMPT</code> , <code>_GENERATE_ANSWER_FROM_DRAFT_PROMPT</code> , and <code>_GENERATE_DRAFT_PROMPT</code> to <code>prompts.py</code> , all requiring: explicit FULL/PARTIAL/INSUFFICIENT self-classification of context coverage before answering; claims restricted to verbatim or direct paraphrase of context only (no inference, extrapolation, or "typical fact" filling); an explicit rule that one relevant chunk is not license for a full itemized answer; and paired GOOD/BAD worked examples per rule. <code>_GENERATE_DRAFT_PROMPT</code> uses the same grounding rules but frames the output as a working draft ("this will be reviewed and rewritten, but must already be honest") rather than a final answer. <code>generate_draft.py</code> rewired to <code>_GENERATE_DRAFT_PROMPT.format(query=..., context=...)</code> .
Rationale	Named anti-patterns plus worked GOOD/BAD pairs are the pattern this project's own prior research (Research topic 19) found most reliable for steering instruction-tuned models away from a specific failure mode — reused here on the generation side rather than an ad hoc caveat.
Impact	<code>app_workflow/services/prompts.py</code> (three new prompts), <code>app_workflow/nodes/generate_answer.py</code> and <code>app_workflow/nodes/generate_draft.py</code> (wired to the new prompts instead of inline strings). Directly enables ADR-058's draft-as-synthesis-input flow — a synthesis step is only as trustworthy as the draft and context it is given.

ADR-059 · LLM Backend Consolidation: Custom OpenAI-Compatible Endpoint for llm/judge_llm/json_fix_llm; Groq Narrowed to Tool-Calling Only (llm_tool)

Field	Detail
Decision	Point all three reasoning-heavy LLM instances in <code>app_workflow/services/llm_setup.py</code> — <code>llm</code> , <code>judge_llm</code> , <code>json_fix_llm</code> — at a single custom OpenAI-compatible endpoint via three new env vars (<code>CUSTOM_API_BASE</code> , <code>CUSTOM_API_KEY</code> , <code>CUSTOM_API_MODEL_NAME</code> , all defaulting to <code>"llama-3.1-8b-instruct"</code>); retain the previous Groq ChatGroq instance under a new name, <code>llm_tool</code> , reserved exclusively for tool-calling traffic (query-variant generation)
Date	2026-07-03
Context	The 2026-07-01 session (ADR-054) adopted the Hugging Face Inference Providers router as a second LLM provider for <code>judge_llm/json_fix_llm</code> , alongside Groq for <code>llm</code> . That router experiment surfaced a high JSON-repair engagement rate (100% of successful <code>judge_llm</code> calls needed repair — Research topic 44) and a curated-subset model-availability constraint (Research topic 43). Separately, <code>GEN_MODEL_NAME</code> was found misconfigured with an HF-Hub-style path feeding the Groq-only <code>llm</code> client (Bugs.md BUG-064), highlighting the risk of juggling two differently-named-and-curated providers across the same config surface. A custom (self-hosted / local) OpenAI-compatible LLM server became available as a third option, prompting a consolidation decision.
Options Considered	Keep the July 1 split (HF router for <code>judge_llm/json_fix_llm</code> , Groq for <code>llm</code>) — two providers to keep in sync, and Research topic 44 already found the router-hosted model needed repair on effectively every call. · Move everything back to Groq (pre-July-1 state) — reintroduces a single point of dependency on Groq's free-tier rate limits across every stage, and Groq is deprecating <code>llama-3.1-8b-instant</code> on 2026-08-16 (Bugs.md BUG-056), which was the original motivation for exploring alternatives (Research topic 41). · Point all three reasoning LLMs at one custom OpenAI-compatible endpoint, keep Groq only for tool-calling (chosen) — one env-var-driven endpoint to manage for <code>llm/judge_llm/json_fix_llm</code> ; Groq retained narrowly for the one capability (tool schemas) not yet confirmed to work on the custom endpoint.

Chosen Solution	llm, judge_llm, and json_fix_llm are each constructed as ChatOpenAI(base_url=os.getenv("CUSTOM_API_BASE"), api_key=os.getenv("CUSTOM_API_KEY"), model=os.getenv("CUSTOM_API_MODEL_NAME", "llama-3.1-8b-instruct"), max_retries=0), differing only in per-role temperature/max_tokens (llm: 0.1/2048; judge_llm/json_fix_llm: 0.0/1024). The former Groq ChatGroq(model_name=os.getenv("GEN_MODEL_NAME", "llama-3.1-8b-instant")) instance is retained verbatim but renamed llm_tool, and app_workflow/nodes/query_variants.py's generate_query_variants() is rewired to import and call it instead of llm.
Rationale	Follows the provider-agnostic design already established in ADR-006/ADR-014 (ChatOpenAI as the universal wrapper — swapping providers is a base_url/api_key change, not a rewrite). Isolating tool-calling onto the one LLM instance already confirmed to support it (Groq) avoids blocking the whole consolidation on first confirming tool-call support against the new custom endpoint. Routing llm/judge_llm/json_fix_llm through one endpoint also removes the two-provider-parity burden the July 1 HF-router adoption introduced — one model-curation constraint, one exception-handling profile, one repair-rate profile to reason about instead of two.
Impact	app_workflow/services/llm_setup.py rewritten — judge_llm and json_fix_llm no longer read HF_TOKEN/HF_API_BASE (added under ADR-054); those env vars are not removed from .env but are no longer consumed by this module. Three new env vars: CUSTOM_API_BASE, CUSTOM_API_KEY, CUSTOM_API_MODEL_NAME. app_workflow/nodes/query_variants.py updated to import llm_tool instead of llm. app_workflow/config.py: ENABLE_ANSWER_QUALITY_OUTPUT_FIX = True re-added under ENABLE_ANSWER_QUALITY_CHECK (previously added and removed same-day on 2026-07-02). Supersedes ADR-054's provider choice for judge_llm/json_fix_llm. app/ is unaffected — it continues to use Groq exclusively per ADR-004/ADR-005.

ADR-060 · Query-Variant Generation: Direct JSON List Output Instead of Tool-Calling

Field	Detail
Decision	Stop using LangChain tool-calling (<code>retrieve_documents</code> tool schema) to extract query variants from the LLM; instead prompt for and parse a plain JSON array (<code>[{"query": "..."}, ...]</code>) via <code>fix_llm_output</code>
Date	2026-07-06
Context	<code>generate_query_variants()</code> originally called the LLM with <code>tools=[RETRIEVE_DOCUMENTS_TOOL_SCHEMA]</code> purely as a structured-output mechanism — the tool was never actually invoked to retrieve anything at this stage (retrieval happens later, per variant, in the parallel <code>retrieve</code> fan-out). The tool schema was being repurposed solely to force the LLM into emitting <code>{"query": "..."}-shaped</code> arguments the node could extract from <code>tool_calls</code> . This meant the query-variant node depended on tool-calling support and reliability for a purpose unrelated to its name, and required whichever LLM backend served it (Groq, historically) to have working tool-call support — one more constraint layered onto the multi-provider juggling already underway (ADR-054/055/060).
Options Considered	Keep the <code>retrieve_documents</code> tool-schema trick (works today on Groq, but ties variant generation to tool-call support and to a schema whose name — "retrieve documents" — describes an action this node never performs) · switch to native JSON-array output, parsed through the project's own existing <code>fix_llm_output</code> repair pipeline (mirrors how every other structured LLM output in the project — merge, judge verdicts, distilled QA pairs — is already handled, and removes the tool-calling dependency entirely)
Chosen Solution	<code>generate_query_variants()</code> no longer passes <code>tools=</code> to <code>llm.invoke()</code> . The prompt (<code>_ROLE_AND_RULES/_PROCESS_INSTRUCTIONS</code> in <code>prompts.py</code>) was rewritten to ask for a bare JSON array with a first-character anchor, and the response is parsed via <code>fix_llm_output("query_variants", raw, llm=llm_tool)</code> against a new <code>QueryVariantSchema</code> , gated by a new <code>ENABLE_QUERY_VARIANTS_OUTPUT_FIX</code> flag (falling back to <code>parse_to_python()</code> when repair is disabled, per the

	Research topic 22 separation of repair vs. parsing). <code>services/tools.py</code> — which existed solely to define the now-unused <code>RETRIEVE_DOCUMENTS_TOOL_SCHEMA</code> — was deleted.
Rationale	Removing tool-calling from this node collapses two independent structured-output mechanisms (LangChain tool schemas vs. <code>fix_llm_output</code>'s JSON registry) down to one, consistent with every other LLM call in the project. It also decouples query-variant generation from tool-call reliability/availability on whatever backend serves it — relevant given the project has already had to route around tool-call-format quirks on non-Groq backends (ADR-019) and was mid-consolidation across three different LLM providers this same week (ADR-054, ADR-060).
Impact	<code>app_workflow/nodes/query_variants.py</code>, <code>app_workflow/services/prompts.py</code> (<code>_ROLE_AND_RULES/PROCESS_INSTRUCTIONS</code> rewritten), <code>app_workflow/services/fix_llm_output.py</code> (new <code>QueryVariantSchema</code>, 10th registry entry), <code>app_workflow/config.py</code> (new <code>ENABLE_QUERY_VARIANTS_OUTPUT_FIX</code> flag), <code>app_workflow/services/tools.py</code> deleted (–342 lines). Motivated a same-day benchmark (<code>Execution Time Comparison.md</code>, Research topic 46) comparing LLM-provider splits now that <code>llm_tool</code>'s only remaining justification (tool-call support) is unused by any live call site — <code>llm_setup.py</code> was rewired back toward the ADR-054 Groq+HF-router split as part of the same session, superseding ADR-060's single-custom-endpoint consolidation after 3 days.

ADR-061 · Full LLM Backend Consolidation: llm_tool, judge_llm, json_fix_llm Moved to the Custom Endpoint; Groq and the HF Router Retired

Field	Detail
Decision	Migrate <code>llm_tool</code> (previously Groq) and <code>judge_llm/json_fix_llm</code> (previously the Hugging Face Inference Providers router) onto the same <code>CUSTOM_API_BASE/CUSTOM_API_KEY</code> endpoint already serving <code>llm</code> since ADR-060; comment out the <code>ChatGroq</code> construction blocks for <code>llm_tool</code> and <code>llm</code> in place rather than deleting them
Date	2026-07-08
Context	ADR-061 (2026-07-06) reverted <code>judge_llm/json_fix_llm</code> back onto the HF router and kept <code>llm_tool</code> on Groq, leaving the provider split at Groq (<code>llm</code>, unused <code>llm_tool</code>) + HF router (<code>judge_llm</code>, <code>json_fix_llm</code>) — justified at the time by tool-calling reliability, even though the only tool-calling call site (query-variant generation) had itself been removed that same day. This session completed the <code>Execution Time Comparison.md</code> benchmark (Research topic 46) with two more configurations: Setup 4 (JSON-fix/validators/CAQ on the HF router, rest local) averaged 10:38 with 67–149 HF 402 hits per run; Setup 5 (all four roles local) averaged 6:58 with zero errors across all three queries — closely trailing Setup 2's 5:57 (Groq tool-LLM + local) as the only other zero-error configuration. A same-session debug-log audit of the exact log files these benchmark runs produced (<code>langgraph_api_20260708_124129.debug.log</code> = Setup 4 Query3/Run2; <code>langgraph_api_20260707_153607.debug.log</code> = Setup 5 Query3/Run2) additionally confirmed the HF 402 outage was silently degrading most validator calls to fallback defaults in the Setup-4-style run, while the all-local run had zero infra failures and its only defects were genuine model-reasoning issues (DC's destructive redundancy deletion, BUG-067) rather than provider unavailability.
Options Considered	Keep the Groq + HF-router split from ADR-061 (justified originally by tool-calling reliability, a justification with no remaining live call site) · move only <code>judge_llm/json_fix_llm</code> back to custom as in ADR-060, leaving <code>llm_tool</code> on Groq (partial consolidation, an unused Groq dependency remains) · move all four roles (<code>llm</code> , <code>llm_tool</code> , <code>judge_llm</code> , <code>json_fix_llm</code>) onto the custom/local endpoint, retiring both Groq and the HF router

	entirely
Chosen Solution	Full consolidation: <code>llm</code>, <code>llm_tool</code>, <code>judge_llm</code>, and <code>json_fix_llm</code> are all now <code>ChatOpenAI(base_url=CUSTOM_API_BASE, api_key=CUSTOM_API_KEY, model=CUSTOM_API_MODEL_NAME, max_retries=0)</code> in <code>app_workflow/services/llm_setup.py</code>, with per-role temperature/max tokens unchanged (<code>llm/llm_tool</code>: 0.1/2048; <code>judge_llm/json_fix_llm</code>: 0.0/1024). The <code>ChatGroq</code> blocks for <code>llm_tool</code> and <code>llm</code> are commented out in place, matching the file's existing practice of preserving prior provider wiring as a fast rollback path.
Rationale	The benchmark now directly measures what ADR-061 could only reason about qualitatively: HF-router quota exhaustion (BUG-065) is the dominant latency cost regardless of what serves the other roles, and it degrades validator judgment quality, not just speed — in the Setup-4-style run, <code>validate_merge/validate_redundancy/validate_lbc</code> returned real (non-fallback) verdicts only 1/13, 0/9, and 0/14 times respectively. With tool-calling no longer used anywhere (ADR-061), Groq's only remaining justification was inertia. Setup 5's zero-error record, combined with the debug-log audit showing its only defects were addressable model/prompt issues rather than infrastructure failures, made local-only the strongest available configuration on both speed and reliability.
Impact	<code>app_workflow/services/llm_setup.py</code> rewritten; <code>HF_TOKEN/HF_API_BASE</code> and <code>GROQ_API_KEY</code> are no longer consumed by this module (though <code>GROQ_API_KEY</code> remains required for the independent <code>ChatGroq</code> fallbacks inside <code>fix_llm_output.py</code>'s <code>LLM_Json_Repair/Verify_And_Correct</code>). <code>judge_llm/json_fix_llm</code>'s <code>model=</code> fallback defaults still read <code>"Qwen/Qwen2.5-7B-Instruct"/"Qwen/Qwen2.5-Coder-3B-Instruct"</code> — dead HF-router-era values, since <code>CUSTOM_API_MODEL_NAME</code> now resolves <code>model</code> identically for all four instances. Committed alongside the completed <code>Execution Time Comparison.md</code> benchmark (Setups 4–5) in the same commit. Supersedes ADR-060/061's provider-split reasoning — see Research topics 46, 50, and 51.

ADR-062 · Observability Backend: Arize Phoenix Adopted as Primary, Self-Hosted Under a No-Data-Egress Constraint; LangSmith Retained in Parallel

Field	Detail
Decision	Instrument the <code>app_workflow/</code> LangGraph pipeline with Arize Phoenix (self-hosted, OTel/OpenInference-based) as the primary observability backend, while keeping the existing LangSmith instrumentation active in parallel rather than removing it
Date	2026-07-09
Context	The project's tracing needs grew (the debug-log quality audits of Research topic 51, and the LangSmith conceptual mapping of Research topic 52) to the point of wanting a proper trace UI rather than manual <code>.debug.log</code> grepping. A hard constraint applies: no prompts, retrieved chunks, queries, or model outputs may leave the local network — ruling out LangSmith's default SaaS/cloud mode outright. LangSmith does support self-hosting, but only as an Enterprise-licensed, Kubernetes/on-prem-oriented product, judged too heavy/expensive for a solo local project. Self-hosted, genuinely-free alternatives were evaluated (see Research topic 54): Langfuse (Postgres + ClickHouse + Redis + blob store) and Arize Phoenix (single container).
Options Considered	LangSmith Cloud (rejected — violates the no-egress constraint) · LangSmith self-hosted Enterprise (rejected — license cost and infra weight disproportionate to a solo local project) · Langfuse self-hosted (viable, but a heavier operational footprint — the ClickHouse dependency in particular — for production-ops features not yet needed) · Arize Phoenix self-hosted (chosen — single container, zero feature gating, strongest fit for the project's actual open retrieval/compression-debugging problems)
Chosen Solution	New <code>app_workflow/services/phoenix_tracing.py</code> module: <code>setup_phoenix_tracing()</code> registers Phoenix via its OTel collector endpoint (<code>PHOENIX_COLLECTOR_ENDPOINT</code> , default <code>http://localhost:4317</code>) and calls <code>LangChainInstrumentor().instrument()</code> to auto-instrument every LangChain/LangGraph call in the pipeline. Called at startup in both <code>app_workflow/api.py</code> and <code>app_workflow/main.py</code> (the latter was initially missed — see Bugs.md BUG-069). The pre-existing LangSmith instrumentation (<code>.env</code> 's <code>LANGCHAIN_TRACING_V2/LANGCHAIN_API_KEY/LANGCHAIN_PROJECT</code>) is left active rather than removed, so the

	two backends can keep being compared on live traffic.
Rationale	Phoenix's single-container footprint and lack of feature gating make it the lowest-friction option that still satisfies the no-egress constraint; its evals/experiment/embedding-drift tooling is also the better fit for this project's actual open problems (two-track retrieval quality, compression faithfulness — Bugs.md BUG-031/BUG-035-class issues) versus Langfuse's stronger but currently-unneeded production-ops maturity (cost attribution, prompt deploy-labels). Keeping LangSmith wired in parallel costs nothing extra now that both are OTel/callback-based instrumentation layered onto the same LangChain calls, and lets the user finish evaluating LangSmith's UI limitations (see Bugs.md BUG-072) before deciding whether to keep, comment out, or fully remove that path.
Impact	New <code>app_workflow/services/phoenix_tracing.py</code> ; <code>app_workflow/api.py</code> and <code>app_workflow/main.py</code> both call <code>setup_phoenix_tracing()</code> at startup. New env vars <code>PHOENIX_COLLECTOR_ENDPOINT</code> , <code>PHOENIX_PROJECT_NAME</code> . Directly motivated ADR-064 (the log-to-trace mirroring handler) so that the project's existing logging-based diagnostic detail is visible inside whichever backend the user is inspecting. <code>app/</code> is unaffected — this instrumentation is scoped to <code>app_workflow/</code> only.

ADR-063 · `_TracingHandler`: Mirror logging Records onto Active LangSmith Runs and Phoenix Spans via Run-ID Lookup

Field	Detail
Decision	Add a third <code>logging.Handler</code> (<code>_TracingHandler</code>) to <code>app_workflow/services/logger_config.py</code> 's <code>setup_logging()</code> that mirrors every log record onto the currently active LangSmith run (<code>RunTree.add_event</code>) and the currently active Phoenix span, looked up via LangChain's <code>run_id</code> rather than OpenTelemetry's ambient "current span" context
Date	2026-07-09
Context	With Phoenix instrumentation added (ADR-063), the project's existing <code>.debug.log</code> detail (67 of 127 <code>logger.*</code> calls across <code>app_workflow/nodes/*.py</code> are <code>logger.debug</code>) still lived in a channel invisible to either trace UI (see Research topic 55). The first implementation attempt used the textbook OTEL pattern, <code>opentelemetry.trace.get_current_span()</code> , for the Phoenix side — this reliably returned a non-recording no-op span in every real run, traced (via source inspection of <code>openinference-instrumentation-langchain</code>) to that package's tracer never calling <code>context.attach()/start_as_current_span()</code> , instead tracking spans in an internal <code>_spans_by_run: Dict[UUID, Span]</code> keyed by LangChain's own run ID (Bugs.md BUG-071).
Options Considered	Drop Phoenix log-mirroring entirely, keep only the LangSmith side (loses parity between the two backends, defeats the point of running both in parallel) · monkeypatch/subclass <code>OpenInferenceTracer</code> to attach spans to OTEL context (invasive, couples the project more deeply to that package's internals, fragile across version upgrades) · reuse the <code>run_id</code> already available from the working LangSmith lookup and query the instrumentor's own internal span registry via its public <code>get_span(run_id)</code> accessor (chosen — no monkeypatching, uses a public method, needs only one extra lookup)
Chosen Solution	<code>_TracingHandler.emit()</code> : (1) calls <code>langsmith.run_helpers.get_current_run_tree()</code> ; if a run is active, calls <code>.add_event(...)</code> on it and captures its <code>run_id</code> . (2) Using that same <code>run_id</code> , calls <code>LangChainInstrumentor().tracer.get_span(run_id)</code> ; if a matching Phoenix span is found, calls <code>.add_event(...)</code> on it with <code>log.level/log.logger/log.request_id</code> attributes. Both lookups are wrapped in <code>try/except: pass</code> so tracing failures never affect

	logging. The handler runs at <code>logging.DEBUG</code> unconditionally (independent of the console handler's <code>INFO</code> level — see Bugs.md BUG-070) so no node-level detail is silently dropped before mirroring.
Rationale	Reusing the LangSmith-side <code>run_id</code> as the join key for Phoenix's internal registry is a minimal, non-invasive fix — it uses a method (<code>get_span</code>) the <code>openinference-instrumentation-langchain</code> package already exposes publicly, rather than reaching into private state or patching the library. It also means the Phoenix lookup's correctness is contingent on the LangSmith lookup already having succeeded for the same call — acceptable since both trackers are driven by the same underlying <code>LangChain</code> callback event.
Impact	<code>app_workflow/services/logger_config.py</code> (<code>TracingHandler</code> class, wiring into <code>setup_logging()</code>). Verified end-to-end: a <code>logger.info(...)</code> call inside a real traced <code>RunnableLambda</code> was confirmed present in the resulting Phoenix span's <code>.events</code> list, and a production run's LangSmith <code>RunTree</code> was confirmed via direct API query (<code>Client().read_run(run_id)</code>) to carry 43 mirrored events. Directly enabled diagnosis of Bugs.md BUG-072 (LangSmith's own UI does not render these events despite storing them) and motivated the companion <code>app_workflow/services/trace_events.py</code> CLI.

**ADR-064 · app/llm_setup.py Diverges From app_workflow/:
llm/merge_llm/judge_llm Reverted to Native ChatGroq; json_fix_llm
Moved to the HF Inference Providers Router**

Field	Detail
Decision	In <code>app/llm_setup.py</code> only, rebuild <code>llm</code>, <code>merge_llm</code>, and <code>judge_llm</code> as native <code>langchain_groq.ChatGroq</code> instances (<code>api_key=GROQ_API_KEY</code>, <code>model_name=MODEL_NAME</code> defaulting to "llama-3.1-8b-instant") instead of the <code>ChatOpenAI(base_url=CUSTOM_API_BASE, ...)</code> construction they carried since the <code>app_workflow/</code> sync; move <code>json_fix_llm</code> off <code>CUSTOM_API_BASE/CUSTOM_API_KEY/CUSTOM_API_MODEL_NAME</code> onto the Hugging Face Inference Providers router (<code>base_url=HF_API_BASE</code>, <code>api_key=HF_TOKEN</code>), reading a new dedicated <code>JSON_FIX_MODEL_NAME</code> env var (default "Qwen/Qwen2.5-Coder-3B-Instruct") in place of the shared <code>CUSTOM_API_MODEL_NAME</code>
Date	2026-07-10
Context	<code>app/llm_setup.py</code> had been brought into parity with <code>app_workflow/services/llm_setup.py</code>'s full custom-endpoint consolidation (ADR-062) via the 610c4b8 "app folder made up-to-date with the app_workflow folder" sync. This commit ("updated API keys to groq") is the first change to touch <code>app/</code> independently since that sync, and it is not mirrored back into <code>app_workflow/services/llm_setup.py</code>, which remains on the full <code>CUSTOM_API_BASE</code> configuration from ADR-062 (verified unchanged by this commit). The commit message states no further rationale; this entry documents the mechanical change and its consequences rather than a benchmark- or bug-driven decision trail, unlike ADR-054/060/061/062.
Options Considered	Keep <code>app/</code> synced to <code>app_workflow/</code>'s custom-endpoint consolidation (status quo immediately prior to this commit) · revert all four <code>app/</code> LLM roles to Groq uniformly · split roles as implemented — Groq for the three reasoning/orchestration roles (<code>llm</code>, <code>merge_llm</code>, <code>judge_llm</code>), HF router for <code>json_fix_llm</code> only
Chosen Solution	The split above, implemented directly in <code>app/llm_setup.py</code>: three <code>ChatGroq</code> instances plus one <code>ChatOpenAI</code> instance pointed at the HF router. Per-role temperature/max tokens unchanged (<code>llm</code>: 0.1/2048; <code>merge_llm</code>: 0.0/2048; <code>judge_llm</code>: 0.0/1024; <code>json_fix_llm</code>: 0.0/1024). No commented-out fallback block was

	left in place for the prior <code>CUSTOM_API_BASE</code> wiring, unlike the rollback pattern <code>app_workflow/services/llm_setup.py</code> has used for its own provider oscillations.
Rationale	Not stated in the commit. Practical effect: <code>llm/merge_llm/judge_llm</code> now depend on Groq's availability and rate limits (subject to the FIFO serialization gate in <code>llm_caller.py</code> , see ADR-044) instead of the local <code>CUSTOM_API_BASE</code> endpoint that <code>app_workflow/</code> now uses exclusively for those roles; <code>json_fix_llm</code> depends on the HF Inference Providers router, the same backend <code>app_workflow/</code> found prone to sustained-load 402s under benchmark conditions (Bugs.md BUG-065) before moving off it in ADR-062.

ADR-065 · Langfuse Adopted as a Third Tracing Backend (Callback-Based), Run Alongside Phoenix and LangSmith

Field	Detail
Decision	Add Langfuse (self-hosted) as a third observability backend for <code>app_workflow/</code> , gated by its own <code>ENABLE_LANGFUSE_TRACING</code> flag and run in parallel with Phoenix and LangSmith rather than replacing either
Date	2026-07-10
Context	ADR-063 (2026-07-09) adopted Phoenix as the primary self-hosted backend under a no-data-egress constraint and explicitly deferred Langfuse — "a candidate for later exploration if/when prompt-version management or production cost-attribution become priorities" (Research topic 54) — without hands-on evaluation. This session brought up a self-hosted Langfuse stack and wired it in to evaluate it directly rather than continuing to defer on comparison-table reasoning alone.
Options Considered	Continue deferring Langfuse and rely on Phoenix + LangSmith only (status quo since ADR-063) · adopt Langfuse and replace Phoenix as primary (premature — Phoenix had not yet been found deficient for this project's actual use) · adopt Langfuse as a third parallel backend, gated by its own flag exactly like Phoenix's <code>ENABLE_PHOENIX_TRACING</code> , so all three can keep being compared on live traffic
Chosen Solution	New <code>app_workflow/services/langfuse_tracing.py</code>

	(get_langfuse_handler() → langfuse.langchain.CallbackHandler); new config.py flag ENABLE_LANGFUSE_TRACING = True; main.py/api.py construct the handler conditionally and pass it as a LangChain callback into every rag_app.invoke(...)/run_distillation() call.
Rationale	Langfuse's self-hosted stack (Postgres + ClickHouse + Redis + MinIO) is heavier to operate than Phoenix's single container (consistent with Research topic 54's comparison), but its Scores/Datasets/Annotation-Queue/Experiment feature set (Research topic 57) is materially richer than Phoenix's for building a curated evaluation harness — the exact gap Research topic 52 identified between this project's ad hoc tooling and a proper LLMOps platform. Evaluating it hands-on, rather than continuing to defer on the comparison table alone, was judged worth the operational cost during this active backend-evaluation phase.
Impact	app_workflow/services/langfuse_tracing.py (new), app_workflow/config.py, app_workflow/main.py, app_workflow/api.py. Directly motivated ADR-067 (Langfuse's callback model, unlike LangSmith/Phoenix's ambient instrumentation, required explicit config propagation the codebase didn't have) and surfaced Bugs.md BUG-076 (Phoenix/Langfuse TracerProvider conflict when both are enabled). app/ is unaffected — this instrumentation is scoped to app_workflow/ only, matching ADR-063's precedent.

ADR-066 · Explicit config Threading Through the Entire LLM Call Chain to Support Callback-Based Tracing

Field	Detail
Decision	Add an optional <code>config: RunnableConfig</code> parameter to every graph node function and every downstream LLM-invocation helper (<code>llm_invoke()</code> , <code>invoke_once()</code> , the four validators, the <code>fix_llm_output.py</code> repair functions, <code>self_learner.py</code> 's distillation path), forwarding it explicitly all the way to <code>llm.invoke(messages, config=config, ...)</code>
Date	2026-07-10
Context	After wiring Langfuse's <code>CallbackHandler</code> into <code>rag_app.invoke(state, config={"callbacks": [langfuse_handler]})</code> (ADR-066), zero traces appeared in Langfuse despite the pipeline running normally. Diagnosis found LangGraph only auto-forwards <code>config</code> into a node if that node's function signature declares a <code>config</code> parameter — none of the ~14 node functions in <code>app_workflow/nodes/</code> did, since LangSmith (env-var <code>ambient</code>) and Phoenix (OTel auto-instrumentation patched globally at import time) never needed it. Even had node-level propagation been correct, <code>services/llm_caller.py</code> 's <code>llm_invoke()/invoke_once()</code> never accepted a <code>config</code> argument at all, and <code>invoke_once()</code> 's <code>ThreadPoolExecutor</code> -wrapped <code>future.result(...)</code> call would have broken ambient contextvar-based propagation regardless of node wiring — LangChain's callback context is not guaranteed to cross a thread hop without being passed explicitly.
Options Considered	Rely on LangGraph/LangChain's ambient config propagation and hope the <code>ThreadPoolExecutor</code> hop doesn't break it (already falsified — this was the observed failure) · thread <code>config</code> explicitly end-to-end through every node and every LLM call site (mechanical, touches ~40 call sites, but the only approach guaranteed to work given the thread-hop) · drop the <code>ThreadPoolExecutor</code> wrapping in <code>invoke_once()</code> to preserve ambient context instead (rejected — the executor wrapping exists specifically to enforce <code>LLM_RESPONSE_TIMEOUT_SECONDS</code> via <code>future.result(timeout=...)</code> , a separate and load-bearing piece of infrastructure not worth removing to fix an unrelated tracing gap)
Chosen Solution	Every node function in <code>app_workflow/nodes/*.py</code> gained an optional <code>config: RunnableConfig</code> parameter and forwards it into

	whichever service-layer call it makes. <code>services/llm_caller.py</code>'s <code>llm_invoke()/_invoke_once()</code> gained the same parameter and now call <code>llm.invoke(messages, config=config, **kwargs)</code> explicitly instead of relying on ambient propagation. <code>services/validators.py</code>'s four validators and <code>services/fix_llm_output.py</code>'s <code>LLM_Json_Repair()/_Verify_And_Correct()</code> were threaded the same way so nested repair/verification LLM calls stay traced. <code>services/self_learner.py</code>'s distillation call site was updated identically.
Rationale	This is the only mechanism guaranteed to work regardless of thread-hops, since it doesn't depend on any ambient-context propagation working correctly in the first place — the same principle already applied to this codebase's request-ID tracing (<code>logger_config.py</code>'s <code>RequestIdFilter/ContextVar</code>, added 2026-06-15) had to be revisited here because LangChain's callback config is a different, non-<code>ContextVar</code> mechanism with its own propagation rules.
Impact	~40 call sites across 11 files: <code>services/llm_caller.py</code>, <code>services/fix_llm_output.py</code>, <code>services/validators.py</code>, <code>services/self_learner.py</code>, and node files <code>query_variants.py</code>, <code>check_answer_quality.py</code>, <code>generate_answer.py</code>, <code>generate_draft.py</code>, <code>dc.py</code>, <code>lbc.py</code>, <code>nac.py</code>, <code>validate_retrieval.py</code>, <code>dedup_merge.py</code>, <code>auto_distillation.py</code>, <code>commands.py</code>. Verified end-to-end: a live query produced a full Langfuse trace with populated <code>GENERATION</code> observations (model, input/output, token usage) for every LLM call in the graph, confirmed via Langfuse's REST API. See Bugs.md BUG-075.

ADR-067 · Central Tracing Policy: `instrument_namespace()` + TraceSpec-Based Automatic Function Tracing

Field	Detail
Decision	Replace manual per-function <code>@traced operation(name)</code> decoration with a single <code>instrument_namespace(globals(), group, exclude={...})</code> call appended to the bottom of each of the 17 <code>app_workflow/</code> node/service modules, and centralize all trace-shaping policy (what counts as a traceable function, which arguments to include/exclude, how to summarize large values, what the trace name looks like) in one <code>operation_tracing.py</code> module via a <code>TraceSpec</code> dataclass registry
Date	2026-07-13
Context	The project already had three independent tracing backends (LangSmith, Phoenix, Langfuse) wired ambient/callback-style at the LangGraph node level (ADR-063/064/066/067), but visibility stopped at node boundaries — the LLM/validator/compression helper functions called <i>inside</i> each node were invisible as individual trace entries in any backend, since only the outer node (or specific <code>llm.invoke()</code> calls picked up by LangChain's own callback instrumentation) produced trace observations. Manually adding a tracing decorator to every function needing coverage would require touching every call site individually and would silently miss newly added helpers.
Options Considered	Per-function manual <code>@traced operation(name)</code> decoration — explicit and readable, but requires remembering to decorate every new function across 17 modules and doesn't scale. · <code>instrument_namespace(globals(), group)</code> called once per module (chosen) — automatically wraps every module-defined function and class method in <code>RunnableLambda</code> -based tracing, closing coverage gaps for future helpers without per-function decoration; needs an <code>exclude</code> set to skip functions already covered by the graph's own node-level tracing, avoiding double-wrapping the top-level LangGraph node function. · Leave function-level tracing permanently manual/opt-in — rejected; the 2026-07-08/07-09 debug-log quality audits (Research topics 51, 55) already relied on manually grepping <code>.debug.log</code> files for per-function detail, and a systematic solution was overdue.
Chosen Solution	<code>instrument_namespace(globals(), group, exclude={...})</code> in <code>operation_tracing.py</code> , called once at the bottom of each of 17 <code>app_workflow/</code> node and service modules (e.g. <code>query_variants.py</code>)

	appends <code>_instrument_namespace(globals(), "Query Variants", exclude={"generate_query_variants"})</code>). Each wrapped function is registered with a stable display name <code>f"{group} / {name}"</code> and traced via <code>@traced_operation</code>, which wraps the call in a <code>RunnableLambda</code> inheriting the ambient <code>LangChain/LangGraph RunnableConfig</code> so it nests correctly under the graph's existing trace hierarchy and is visible to all three backends simultaneously. A <code>TraceSpec</code> dataclass (added later the same day) — holding <code>input_builder/output_builder</code> callables — lets specific functions (retrieval, embeddings, LLM calls, validators, storage) shape what actually gets recorded instead of dumping raw arguments/return values. A <code>_summarize()</code> recursive serializer bounds trace payload size (<code>MAX_TEXT_CHARS = 2_000</code> char previews, <code>MAX_COLLECTION_ITEMS = 20</code> item caps, numpy arrays reduced to <code>shape+dtype</code>, depth-limited at 3 levels), and <code>_include_argument()</code> filters out noisy arguments (<code>self, cls, config, callbacks, client, handler</code>) from trace inputs.
Rationale	A single append-only call per module (rather than N decorators per module) means every current and future function in a traced module gets coverage automatically, directly closing the "helper functions invisible inside node boundaries" gap without ongoing maintenance burden. Centralizing the trace-shaping policy in one file, rather than scattering per-function trace customization across 17 modules, keeps the "what does a trace look like" decision auditable and consistent — matches this project's established pattern of centralizing cross-cutting policy in one file (<code>config.py</code> for feature flags, <code>prompts.py</code> for prompt strings, per ADR-021).
Impact	New <code>app_workflow/services/operation_tracing.py</code> (final size ~333 lines across two same-day passes: +99 initial, +234 expansion). Two-line <code>_instrument_namespace(...)</code> call appended to the bottom of all 17 <code>app_workflow/</code> node + service modules. Three constants (<code>EXCLUDED_ARGUMENTS</code>, <code>MAX_TEXT_CHARS</code>, <code>MAX_COLLECTION_ITEMS</code>) were initially kept private inside <code>operation_tracing.py</code> to keep the tracing setup self-contained, then moved into <code>config.py</code> on request so the tracing-policy limits are user-configurable alongside every other pipeline constant. The <code>TraceSpec/_summarize/_include_argument</code> machinery is the single point of control for trace payload shape across the entire <code>app_workflow/</code> pipeline — any future node or service function automatically inherits this policy with zero per-function code. <code>app/</code> is unaffected — this instrumentation is scoped to <code>app_workflow/</code> only, matching the precedent set by ADR-

	063/064/066/067.
--	------------------

ADR-068 · Dedicated LangfuseHandler (logging.Handler Subclass) Mirrors Every Log Record as a Langfuse Event, Independent of _TracingHandler

Field	Detail
Decision	Add a new <code>logging.Handler</code> subclass, <code>LangfuseHandler</code> , in <code>app_workflow/services/langfuse_logging.py</code> , registered as a fourth root log handler (alongside console, file, and the existing <code>_TracingHandler</code>) at <code>DEBUG</code> level, that converts every <code>LogRecord</code> into a Langfuse event observation attached to the active trace — rather than extending <code>_TracingHandler</code> (ADR-064) to also cover Langfuse
Date	2026-07-13
Context	<code>_TracingHandler</code> (ADR-064, 2026-07-09) already mirrors log records onto active LangSmith runs and Phoenix spans via a run-id lookup. Langfuse was added as a third tracing backend the next day (ADR-066/067), but its callback-based, non-ambient tracing model (Research topic 56) means the same run-id-lookup mechanism <code>_TracingHandler</code> uses for LangSmith/Phoenix doesn't apply the same way — Langfuse's Python SDK exposes log-to-trace attachment through its own <code>get_client()</code> -based event-creation API tied to whatever trace/observation is ambient in the current OpenTelemetry context (populated by Langfuse's own OTel-native SDK), not through a LangChain <code>run_id</code> registry lookup like Phoenix's <code>OpenInferenceTracer.get_span(run_id)</code> .
Options Considered	Extend <code>_TracingHandler</code> with a third branch for Langfuse — keeps all log-mirroring logic in one handler, but couples three backends' very different attachment mechanisms (run-id registry lookup for LangSmith/Phoenix vs. ambient-OTel-context event creation for Langfuse) into one function, and risks one backend's failure/latency affecting the others within a single <code>emit()</code> call. · Separate dedicated <code>LangfuseHandler</code> (chosen) — isolates Langfuse's event-creation mechanism and its own failure modes (missing/disabled Langfuse config, re-entrancy from the Langfuse SDK's own internal logging) from the existing LangSmith/Phoenix mirroring path entirely. On handler <i>level</i>: INFO (initial choice) — fewer events shipped to Langfuse, but silently drops the same class of node-level <code>logger.debug(...)</code>

	detail that BUG-070 had already found missing from <code>TracingHandler</code> (67 of 127 <code>logger.*</code> calls across <code>app_workflow/nodes/*.py</code> are <code>logger.debug</code>). · DEBUG (chosen) — ships every log line regardless of level, at the cost of thousands of extra event rows per trace on a full pipeline run.
Chosen Solution	<code>LangfuseHandler.emit(record)</code> formats the log message, lazily calls Langfuse's <code>get_client()</code>, and creates a Langfuse event carrying the formatted message, logging level (mapped to Langfuse's <code>DEBUG/DEFAULT/WARNING/ERROR</code> scale), logger name, request ID, and file/function/line metadata; Langfuse automatically associates the event with whichever trace/observation is currently active. A <code>ContextVar</code>-based re-entrancy guard prevents log records emitted <i>during</i> the Langfuse SDK's own internal calls (e.g. its export/flush machinery) from feeding back into <code>LangfuseHandler</code> and recursing. First registered in <code>logger_config.py</code> at <code>INFO</code>, then changed to DEBUG the same session (matching <code>TracingHandler</code>'s independent-of-console-level design from BUG-070) so no node-level detail is dropped before mirroring. Any Langfuse-side failure (missing client, disabled tracing, network error) is caught and silently tolerated so logging never affects the application.
Rationale	Reuses the "mirror logging onto the active trace" pattern already validated by <code>TracingHandler</code>, but as its own handler rather than a shared one — consistent with how this project has treated each new tracing backend as an independently toggleable, independently failing concern (<code>ENABLE_PHOENIX_TRACING</code>, <code>ENABLE_LANGFUSE_TRACING</code>) rather than a monolithic tracing layer. The re-entrancy guard directly anticipates the same feedback-loop risk already documented for <code>TracingHandler</code>-adjacent code — the Langfuse SDK's own <code>logging.getLogger("langfuse")</code> output could otherwise re-enter <code>LangfuseHandler.emit()</code> if not excluded. The INFO → DEBUG level change was a deliberate reversal, not an oversight: <code>DEBUG</code>-level messages carry exactly the granular pipeline detail (per-node intermediate state, retry/backoff detail, per-chunk validation reasoning) that <code>operation_tracing.py</code>'s function-level tracing (ADR-068) is meant to make visible, so filtering them out at the handler defeats the point of adding this handler at all. The thousands-of-extra-rows cost was judged acceptable because Langfuse's own trace UI supports filtering/searching observations after ingestion — the volume is a viewing-time concern, not a capture-time one, so it is cheaper to over-capture and filter on read than to under-capture and lose detail

	permanently.
Impact	New <code>app_workflow/services/langfuse_logging.py</code> (+74 lines). <code>app_workflow/services/logger_config.py</code> modified (+14/-6) to instantiate <code>LangfuseHandler</code> at <code>DEBUG</code> and register it as the fourth root handler. New <code>app_workflow/tests/test_langfuse_logging.py</code> (+43 lines; 2 tests passing). Combined with <code>_TracingHandler</code> (ADR-064) and <code>operation_tracing.py</code>'s <code>instrument_namespace()</code> (ADR-068), every log line and every instrumented function call in <code>app_workflow/</code> now surfaces in whichever of the three tracing backends is enabled. <code>app/</code> is unaffected.
Impact	<code>app/llm_setup.py</code> only — <code>app_workflow/services/llm_setup.py</code> is unchanged and still fully on <code>CUSTOM_API_BASE</code>. <code>app/</code> now requires <code>GROQ_API_KEY</code>, <code>MODEL_NAME</code>, <code>HF_API_BASE</code>, <code>HF_TOKEN</code>, and <code>JSON_FIX_MODEL_NAME</code>; it no longer consumes <code>CUSTOM_API_BASE/CUSTOM_API_KEY/CUSTOM_API_MODEL_NAME</code> (those remain required for <code>app_workflow/</code>). <code>Architecture.md</code>'s description of <code>llm_caller.py</code> as "a thin wrapper around <code>ChatOpenAI</code> that routes to Groq's OpenAI-compatible endpoint" now only describes <code>json_fix_llm</code>'s HF-router traffic within <code>app/</code> — <code>llm/merge_llm/judge_llm</code> bypass <code>ChatOpenAI</code>-based routing entirely via native <code>ChatGroq</code>. <code>app/</code> and <code>app_workflow/</code> have diverging LLM provider configurations for the first time since the <code>610c4b8</code> sync.

ADR-069 · Run the Full Workflow Benchmark Against One Long-Lived API Process

Field	Detail
Decision	Use <code>useful_but_not_valuable_Files/run_all_workflow_batches.py</code> to start <code>app_workflow/api.py</code> once, wait for readiness through <code>GET /stats</code> , execute all configured query / feedback / stats / learn entries consecutively against that same process, summarize failures, and shut the server down through <code>POST /quit</code> with process-level fallbacks. Expand the catalog to 15 batches / 100 numbered scenarios while allowing targeted runs by commenting batches or entries in/out.
Date	2026-07-14
Context	The benchmark needed to exercise feedback refinement, learned-QA accumulation, forced distillation, tracing, and a broad range of retrieval/grounding edge cases under realistic sustained load. Restarting the API between batches would repeatedly reload models/services, fragment traces and logs, discard process-local state, and prevent later scenarios from observing earlier feedback/learning effects. A fixed startup sleep would be unreliable because model/vector-store initialization time varies by machine and cache state.
Options Considered	Restart per batch — stronger batch isolation but invalidates cross-batch learning/feedback tests and adds repeated startup cost. · Run every request in-process without HTTP — faster and easier to introspect, but bypasses FastAPI request validation, endpoint behavior, readiness, request IDs, and shutdown semantics. · One long-lived HTTP API with readiness polling (chosen) — exercises the deployed interface and preserves sequential runtime effects, at the cost of deliberate cross-scenario state and failure propagation.
Chosen Solution	The runner owns one subprocess, polls <code>/stats</code> every 250 ms for up to five minutes, maps string entries to <code>/query</code> and typed entries to <code>/feedback/bad</code> , <code>/stats</code> , or <code>/learn</code> , sends no artificial inter-request delay, records non-2xx/exception failures without silently losing the suite summary, then calls <code>/quit</code> ; if graceful shutdown fails it escalates locally to <code>terminate()</code> and <code>kill()</code> . The numbered catalog spans disambiguation, empty retrieval, multi-source reasoning, hallucination risk, dataset analysis, disparities, grounding, self-learning, feedback refinement, and edge cases.

Rationale	The purpose is a sustained system benchmark, not an isolated unit-test matrix. A single process exposes accumulated learned-QA and feedback behavior, long-tail provider latency, service degradation, and persistence failures that short isolated runs can hide. Readiness polling makes startup synchronization evidence-based rather than time-based.
Impact	Commit <code>cdb1f71</code> introduced the runner; the July 14 working expansion grew it to 15 batches / 100 scenarios. The run updated the shared learned-QA Chroma store as an expected side effect. Results are not statistically independent because earlier scenarios can affect later retrieval/learning; isolated regression tests remain necessary for causal comparisons.

ADR-070 · Resolve Workflow Feature Flags Per Request and Carry Them in GraphState

Field	Detail
Decision	Allow <code>POST /query</code> to accept any subset of 20 <code>ENABLE_*</code> boolean overrides (organized into nine functional groups), resolve them once against <code>app_workflow/config.py</code> defaults, store the complete effective mapping in <code>GraphState["switches"]</code> , and make every affected route/node/service read the request-scoped values rather than process-global constants.
Date	2026-07-14
Context	The expanded benchmark needs to compare workflow configurations without editing <code>config.py</code> and restarting the long-lived API between requests. Direct module-level constants are process-wide and cannot safely express consecutive requests using different combinations. The override surface spans graph routing and deeply nested helper calls, so partial propagation would produce misleading hybrid configurations.
Options Considered	Edit environment/config and restart per configuration — simple but slow, fragments sustained runs, and prevents per-request experiments. · Mutate imported module globals around each request — minimal code change but unsafe under concurrent requests and prone to leaked configuration after exceptions. · Pass individual booleans through every function — explicit but creates large, brittle signatures. · Resolve a typed API object into one state dictionary (chosen) — validates the

	public schema, centralizes fallback behavior, and gives routing plus nested services one consistent effective configuration.
Chosen Solution	<code>WorkflowSwitches</code> exposes optional Pydantic boolean fields; <code>build_initial_state()</code> excludes <code>None</code> , <code>resolve_switches()</code> overlays recognized booleans on <code>DEFAULT_SWITCHES</code> , and the result is stored in the initial state. <code>get_switches(state)</code> is the sole runtime accessor and falls back to defaults for CLI/legacy states. Routes consult it for graph-shape choices; nodes pass the same dictionary into validators, compression helpers, output repair, and self-learning functions that lack direct state access. Disabled fixed-topology stages become pass-through nodes where necessary.
Rationale	Request state is already the <code>LangGraph</code> execution contract and is isolated per invocation. Resolving once prevents different layers from interpreting omitted values differently, preserves <code>config.py</code> as the source of defaults, avoids shared mutable global state, and makes switch combinations traceable/reproducible from the request plus state.
Impact	Commit <code>1340c10</code> ; new <code>app_workflow/services/switches.py</code> ; <code>GraphState</code> gains optional <code>switches</code> ; <code>QueryRequest</code> gains optional <code>WorkflowSwitches</code> ; routing, query variants, retrieval validation/deduplication, NAC/DC/LBC, answer-quality checking, output repair, QA generation, commands, and auto-distillation now honor per-request values. Byte-compile, import, graph-build, default, override, and fallback smoke checks passed. The API docs at http://localhost:8001/docs expose the nested switch schema.

ADR-071 · Keep Docling, Unstructured, and Marker-PDF Converters as Evaluation Utilities; Do Not Replace Production Ingestion Yet

Field	Detail
Decision	Preserve all three converters under <code>useful_but_not_valuable_Files/document-loaders-codes/</code> as reproducible comparison utilities. Keep the existing production ingestion path unchanged. Do not promote raw Marker Markdown to the authoritative RAG representation or replace the current loader stack until candidates are compared under explicit retrieval-quality and structural-fidelity criteria and the chosen path includes asset validation, UTF-8 normalization, page-aware cleanup, heading repair, and complex-table handling.
Date	2026-07-15
Context	Docling, Unstructured, and Marker-PDF were explored as document-to-Markdown ingestion candidates. Batch-capable scripts now exercise each library over the same recursive <code>source/</code> tree and preserve outputs by relative path. The Marker evaluation found strong body-text recovery on conventional prose and useful results on some simple tables, but unreliable complex-table/form geometry, heading hierarchy, page furniture, page boundaries, nested-list structure, reading order, and visual semantics. Its tested output also contained 131 broken image references and mojibake in one prose-heavy document. Neither <code>app/</code> nor <code>app_workflow/</code> was changed to consume these Markdown outputs.
Options Considered	Replace the production loader immediately with Marker — rejected because structurally noisy Markdown would create misleading headings, chunks, metadata, and broken visual context. · Select Docling or Unstructured immediately from exploratory scripts alone — rejected because the preserved evidence does not include an equivalent, documented cross-loader quality comparison sufficient for adoption. · Discard the scripts after exploration — rejected because the batch adapters provide a reproducible basis for later evaluation.
Chosen Solution	Keep the three scripts and the Marker assessment as non-production research artifacts. Continue using the current <code>app/ingest.py</code> path unless a later bake-off demonstrates a better option against both structural and downstream RAG criteria. Treat Marker as a conditional first-stage text extractor whose

	output requires validation and post-processing.
Rationale	High text retention is necessary but insufficient for RAG ingestion. False headings, repeated page furniture, broken image links, lost table relationships, and encoding corruption can create bad chunks and misleading metadata even when most words survive. Deferring adoption preserves runtime stability while retaining a reproducible evaluation base.
Impact	Production behavior and dependencies remain unchanged. Future loader experiments can reuse consistent batch runners, but any adoption proposal must evaluate downstream retrieval quality as well as Markdown appearance and define richer fallbacks or manual review for layout-dependent documents. Commit <code>b75e70a</code> ; see Research topic 61.

ADR-072 · Run Marker as an Isolated GPU Microservice (Out-of-Process) Rather Than Installing marker-pdf Into app_workflow/

Field	Detail
Decision	Deploy Marker (marker-pdf 1.10.2 + surya-ocr 0.17.1 + a CUDA torch build) as a standalone GPU Docker container (marker_service/) exposing a POST /convert HTTP endpoint, and have app_workflow/ call it over HTTP via marker_loader.py, instead of importing marker-pdf into app_workflow/'s own Python 3.14 environment
Date	2026-07-24
Context	ADR-072 (2026-07-15) had deferred adopting Marker for production ingestion pending a quality bake-off, keeping it as an evaluation utility only. This session moved to actually wire a Marker-backed loader into app_workflow/. The blocker is a hard dependency conflict: marker-pdf's stack requires transformers 4.x and (on marker-pdf 2.0.0) pillow<11, but app_workflow/ runs Python 3.14 with sentence-transformers (the embedding backbone of the entire pipeline) pinned to a transformers 5.x floor, and pillow<11 has no Python-3.14 wheel and will not compile from source there. Installing Marker in-process would either break embeddings (transformers downgrade) or fail the build outright (pillow). A brief in-process attempt during the session did get marker-pdf installed behind a pillow>=12 override, but that leaves a fragile, override-dependent single environment carrying two mutually hostile ML stacks.
Options Considered	Install marker-pdf into app_workflow/'s venv behind dependency overrides — tried; works for a moment but forces transformers/pillow overrides that risk silently breaking sentence-transformers embeddings and must be re-justified on every dependency bump; rejected as unmaintainable. · Maintain a second Python 3.12 virtualenv (.venv-marker) on the host and shell out to it — avoids the container but is not portable (Windows wheels, baked paths), still couples two CUDA torch builds to one machine's driver, and offers no process/GPU isolation. · Isolated GPU Docker microservice with an HTTP boundary (chosen) — gives Marker its own Python 3.12 + CUDA-13.0 torch stack and its own model cache, fully decoupled from app_workflow/'s environment; the only integration surface is a stable POST /convert contract.
Chosen Solution	New top-level marker_service/: server.py (FastAPI, POST

	/convert + GET /health, one PdfConverter built at boot via asyncio.to_thread, conversions serialised behind a threading.Lock, pdftext_workers=1 — BUG-077); Dockerfile (python:3.12-slim, CUDA-13.0 torch==2.13.0+cu130, frozen requirements.txt carried from the validated .venv-marker, build-time download_font() pre-seed, late gcc/libc6-dev for Triton JIT); docker-compose.yml (nvidia GPU passthrough, persistent marker-models cache volume, port 8002). app_workflow/ingestion/marker_loader.py becomes a thin HTTP client POSTing each PDF to MARKER_SERVICE_URL/convert (default http://localhost:8002, config.py), keeping its load_documents(file_paths) -> list[Document] signature so callers are agnostic to the transport.
Rationale	The dependency conflict is intrinsic (two ML stacks with incompatible transformers/pillow floors), not a version-pin accident, so the only durable fix is environment isolation. A container additionally gives independent GPU-model caching, restart isolation (a Marker crash cannot take down the RAG API), and a portable, reproducible build — none of which an override-laden shared venv or a host-local second venv provides. The stable HTTP contract means app_workflow/ and Marker can each upgrade their ML stacks independently. This advances ADR-072's "don't productionise raw Marker yet" into "productionise the transport while keeping raw-Markdown-quality caveats (Research topic 61) unresolved" — the loader is now available and switchable, but still defaults off.
Impact	New marker_service/ component (server/Dockerfile/compose/requirements/README), independent of both pipelines. app_workflow/ingestion/marker_loader.py rewritten from an in-process converter to an HTTP client; new MARKER_SERVICE_URL/MARKER_SERVICE_TIMEOUT config + env vars. Running the Marker loader now requires the sidecar container to be up and GPU-reachable; when it is down or a PDF fails, the loader logs and skips that file so the rest of the batch still ingests. app/ is unaffected. See ADR-074 for the ingestion-pipeline refactor that consumes this loader, and Research topic 62 for the dependency-conflict analysis.

ADR-073 · Unified, Switchable `app_workflow/` Ingestion Pipeline (Loader × Splitter Matrix) With In-Memory Loading and a POST `/ingest` Endpoint

Field	Detail
Decision	Replace the monolithic <code>app_workflow/ingestion/ingest.py</code> with a coordinated five-module ingestion package driven by <code>ingestion_requests.run_ingestion(...)</code> , selecting the loader (Marker vs. Unstructured) and splitter (custom vs. recursive) from two <code>config.py</code> switches (<code>ENABLE_MARKER_LOADER</code> , <code>ENABLE_CUSTOM_SPLITTER</code>) that a POST <code>/ingest</code> request body can override per run; load everything in memory (no persisted intermediate Markdown/JSON)
Date	2026-07-24
Context	Introducing the Marker HTTP loader (ADR-073) meant <code>app_workflow/</code> needed a loader/splitter it could select at runtime rather than a single hardcoded path. The prior <code>ingest.py</code> bundled discovery, loading, splitting, and storage into one file with a fixed loader; the exploratory <code>custom_splitter.py</code> additionally had its own <code>discover_files()</code> and wrote Marker Markdown to disk. Both the earlier exploration (writing <code>.md/.json</code> intermediates) and the new microservice model made persisted intermediates unnecessary — Marker output arrives over HTTP and is chunked immediately. <code>app_workflow/</code> also needed an HTTP way to trigger ingestion for the benchmark/runner tooling.
Options Considered	Keep one hardcoded loader/splitter and swap by editing <code>config.py</code> + restart — simplest, but can't vary per request and forces a process restart to compare loaders. · Add ad-hoc boolean branches inside the existing <code>ingest.py</code> — less code churn but keeps discovery/loading/splitting/storage entangled and leaves <code>custom_splitter</code> 's disk-writing <code>discover_files()</code> as a second, divergent discovery path. · Factor into a five-module package with a single <code>run_ingestion</code> coordinator and shared <code>discover_files()</code> (chosen) — one recursive discovery, four interchangeable components (2 loaders × 2 splitters), switch resolution in one place, and a stable summary contract; components import lazily so importing the package (e.g. at API startup) doesn't pull Marker/Unstructured deps unless actually used.
Chosen Solution	<code>ingestion_requests.py</code> owns <code>discover_files()</code> (recursive <code>rglob</code> over <code>data_files/</code> for <code>SUPPORTED_EXT</code> , order-preserving de-dup) and

	run_ingestion(enable_marker_loader=None, enable_custom_splitter=None), which resolves each switch via resolve() (None → config.py default, explicit bool → override), lazily imports and runs the chosen loader then splitter, embeds via EmbeddingManager, upserts into the documents collection via VectorStore.add_documents_in_batches, and returns a summary dict. marker_loader.py (HTTP, PDF-only) and unstructure_loader.py (Unstructured + in-memory tabular→JSON + JSONLoader) are the two loaders; custom_splitter.py (Marker-Markdown-aware) and recursive_splitter.py (RecursiveCharacterTextSplitter + per-source chunk_seq) are the two splitters — both consume/return LangChain Documents. custom_splitter lost its own discover_files(), Markdown reader, local Document dataclass, and main(). POST /ingest in api.py accepts an optional IngestRequest (the two switch overrides) and runs run_ingestion off-thread. write_chunk_run() + the tiktoken encoder moved to services/logger_config.py; ingest.py and marker_logger_config.py deleted.
Rationale	Centralizing discovery and switch resolution removes the divergent second discovery path and makes the four loader×splitter combinations a single, testable matrix. Loading in memory (Marker over HTTP, tabular converted in-RAM) drops the disk-intermediate step the earlier exploration used, so nothing needs cleanup and re-ingestion is idempotent from source. Lazy component imports keep the heavy Marker/Unstructured dependency trees out of the API's import path unless a run selects them. The POST /ingest endpoint gives the benchmark tooling and future callers a uniform, off-thread trigger consistent with the pipeline's other endpoints. Defaults stay False so behavior is unchanged until a caller opts in — consistent with this project's config-flag conventions (ADR-021).
Impact	app_workflow/ingestion/ reorganized (net +952 / -511 across the session): new ingestion_requests.py, unstructure_loader.py, recursive_splitter.py; rewritten marker_loader.py; trimmed custom_splitter.py; deleted ingest.py, marker_logger_config.py. api.py gains POST /ingest; config.py gains the ingestion section (switches, DATA_FILES_DIR, CHUNK_RUNS_DIR, extension sets, chunk sizes, TOKEN_ENCODING); services/logger_config.py gains write_chunk_run + the token encoder. data_files/ becomes the recursive ingestion root. A known caveat carries forward: the Marker loader silently ingests only PDFs, so a mixed corpus under ENABLE_MARKER_LOADER=True drops non-PDFs — use

	the Unstructured loader or a separate pass. <code>app/ingest.py</code> is untouched and remains <code>app/</code> 's ingestion path. See ADR-073 and Research topic 62.
--	---